

RĪGAS VALSTS TEHNIKUMS

DATORIKAS NODAĻA

Izglītības programma: Programmēšana

KVALIFIKĀCIJAS DARBS

"Futbola statistikas apskates vietne"

Paskaidrojošais raksts 74 lpp.

Audzēknis:

Rainers Čubars DP4-4

Prakses vadītājs:

Gundars Rēbuks

Nodaļas vadītājs:

Normunds Barbāns

Rīga 2026

ANOTĀCIJA

Kvalifikācijas darba tēma ir futbola statistikas tīmekļa lietojumprogrammas "Footbalyze" izstrāde. Sistēma nodrošina piekļuvi desmit futbola sacensībām: deviņām līgām un UEFA Čempionu līgai. Lietotāji var aplūkot statistiku, pārvaldīt personīgo izlasi, iesniegt spēļu prognozes un atstāt komentārus. Klienta puse uzrakstīta ar Vue 3, servera puse ar Node.js un Express.js, datubāze ir MySQL. Futbola dati nāk no football-data.org API; lokāli glabājas tikai lietotāju kontu informācija. Paroles šifrē bcrypt, sesijas nodrošina JWT, komentārus attīra sanitize-html.

Ievadā aprakstīts problēmas konteksts un esošo risinājumu trūkumi. Uzdevuma nostādņē definēti sistēmas lietotāji, to piekļuves tiesības un ievaddati. Prasību specifikācijā uzskaitītas astoņas funkcionālās prasības (PR-01 līdz PR-08) un piecas nefunkcionālās (NFR-01 līdz NFR-05), kā arī parādītas četras lietotāja saskarnes skices. Tehnoloģiju izvēles pamatojumā izskaidrota katra tehnoloģijas izvēle ar alternatīvu salīdzinājumu. Sistēmas modelēšanas nodaļā aprakstītas piecas apakšsistēmas, entītiju–attiecību (ER) modelis Čena notācijā un astoņas datu plūsmu diagrammas (DFD). Datu struktūru aprakstā raksturoti visi seši datubāzes tabulu lauki. Lietotāja ceļvedī aprakstītas sistēmas prasības, instalācija un katrs saskarnē redzamais elements ar ekrānuzņēmumiem. Testēšanas nodaļā dokumentēti melnās kastes testa gadījumi piecām prasībām.

Paskaidrojošais raksts satur 74 lappuses, 41 attēlu, 13 tabulas, 4 pielikumus.

ANNOTATION

The subject of this qualification work is the development of a football statistics web application called "Footbalyze". The system covers ten football competitions: nine domestic leagues and the UEFA Champions League. Users can browse statistics, manage a favourites list, submit predictions, and comment on matches. The client is built with Vue 3, the server with Node.js and Express.js, and the database runs on MySQL. Football data comes from the football-data.org API; only user account data is stored locally. Passwords are hashed with bcrypt, sessions use JWT, and comments are sanitized with sanitize-html.

The introduction outlines the problem context and shortcomings of existing solutions. The problem statement defines user roles, access rights, and input/output data. The requirements specification lists eight functional requirements (PR-01 to PR-08) and five non-functional requirements (NFR-01 to NFR-05), with four wireframes. The technology justification chapter explains each tool choice with alternative comparisons. The modelling chapter describes five subsystems, the entity–relationship (ER) model in Chen notation, and eight data flow diagrams (DFD). The data structure chapter details all six database tables. The user guide covers system requirements, setup, and a screen-by-screen walkthrough with screenshots. The testing chapter documents black-box test cases for five requirements.

The explanatory report contains 74 pages, 41 figures, 13 tables, 4 appendices.

SATURS

IEVADS	7
1. UZDEVUMA NOSTĀDNE	9
2. PRASĪBU SPECIFIKĀCIJA	12
2.1. Ieejas un izejas informācijas apraksts	12
2.1.1. Ieejas informācijas apraksts	12
2.1.2. Izejas informācijas apraksts	13
2.1.3. Ārējās informācijas apraksts	13
2.2. Funkcionālās prasības	14
2.3. Nefunkcionālās prasības.....	16
3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS	21
3.1. Frontend tehnoloģijas.....	21
3.1.1. Vue 3 (v3.5.13).....	21
3.1.2. Vue Router (v4.5.0).....	21
3.1.3. Vuex (v4.0.2).....	21
3.1.4. Vite (v6.2.4)	21
3.2. Backend tehnoloģijas	22
3.2.1. Node.js (v22.14.0).....	22
3.2.2. Express (v5.1.0)	22
3.2.3. MySQL 8.3 un mysql2 (v3.14.1)	22
3.2.4. jsonwebtoken (v9.0.2)	22
3.2.5. bcryptjs (v3.0.2).....	23

3.2.6. <i>sanitize-html</i> (v2.17.2).....	23
3.2.7. <i>multer</i> (v2.1.1)	23
3.3. Ārējie pakalpojumi	23
3.3.1. <i>football-data.org API</i>	23
3.3.2. <i>Palīgbibliotēkas</i>	23
3.4. Aizgūto moduļu saraksts	24
4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA	26
4.1. Sistēmas arhitektūra	26
4.2. Sistēmas ER modelis.....	27
4.3. Funkcionālais sistēmas modelis.....	28
4.3.1. <i>Lietotāja pieslēgšanās modulis</i>	29
4.3.2. <i>Proгноzes iesniegšanas modulis</i>	31
4.3.3. <i>Komentāra pievienošanas modulis</i>	33
4.3.4. <i>Reģistrācijas modulis</i>	35
4.3.5. <i>Profila rediģēšanas modulis</i>	36
4.3.6. <i>Izlases pārvaldības modulis</i>	36
4.3.7. <i>Avatāra augšupielādes modulis</i>	38
4.3.8. <i>Lietotāja bloķēšanas modulis</i>	40
5. DATU STRUKTŪRU APRAKSTS	42
6. LIETOTĀJA CEĻVEDIS	46
6.1. Sistēmas prasības aparatūrai un programmatūrai	46
6.2. Sistēmas instalācija un palaišana	46

6.3. Programmas apraksts	47
7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA	70
NOBEIGUMS	72
INFORMĀCIJAS AVOTI	73
PIELIKUMI.....	75

IEVADS

Futbols ir viens no skatītākajiem sportiem Eiropā. Fans, kurš seko vairākiem čempionātiem vienlaikus, parasti izmanto vairākas vietnes: rezultāti vienā, tabulas citā, spēlētāju statistiku bieži nākas meklēt vēl citur. Vienots risinājums, kas apkopotu datus, ļautu saglabāt personīgo izlasi un aktīvi līdzdarboties, tīmeklī praktiski nav pieejams bez maksas abonēšanas.

Tirgū pastāv vairāki plaši izmantoti risinājumi, taču katram ir būtiski ierobežojumi. Sofascore ir labi pazīstama lietotne ar reāllaika datiem un detalizētu statistiku, taču daudzas funkcijas (prognozes, pilna spēlētāju statistika, reklāmu brīva pieredze) pieejamas tikai ar maksas abonementu, un platforma galvenokārt orientēta uz mobilajām ierīcēm. FlashScore darbojas tīmeklī un aptver plašu līgu klāstu, taču reklāmas aizņem lielu daļu saskarnes; izlasi saglabāt, sekot konkrētam spēlētājam vai komentēt spēles nav iespējas. ESPN.com ir plašs sporta portāls ar labu pārklājumu, taču Eiropas futbols tur nav prioritāte un bez abonēšanas vairākas sadaļas ir daļēji bloķētas.

Kopīgais trūkums visos minētajos risinājumos ir tas, ka bezmaksas lietotājs nevar vienā vietā gan aplūkot statistiku no vairākām līgām, gan saglabāt izlasi, gan iesniegt prognozes, gan komentēt. Footbalyze mērķis ir novērst šo robu: piedāvāt tīmekļa platformu, kur visas šīs darbības ir pieejamas bez maksas, vienā saskarnes vietā.

Footbalyze aptver desmit sacensības: Anglijas Premjerlīgu, Spānijas La Ligu, Vācijas Bundeslīgu, Itālijas Sēriju A, Francijas Ligue 1, Anglijas Čempionātu, Portugāles Primeira Liga, Nīderlandes Eredivisie, Brazīlijas Série A un UEFA Čempionu līgu. Futbola dati (komandu sastāvi, spēlētāju statistika, spēļu grafiki un rezultāti) nāk no football-data.org API, tāpēc sistēmai nav jāuztur sava sporta datubāze. Lokāli glabājas tikai lietotāju kontu informācija.

Reģistrēts lietotājs var saglabāt vienu mīļāko komandu un vienu spēlētāju izlasē, pievienot spēlētājus vērošanas sarakstam, iesniegt spēļu rezultātu prognozes un komentēt spēles. Lietotāju aktivitāte tiek apkopota līderpaneļa reitingā. Administrators var bloķēt lietotājus un dzēst ziņotos komentārus.

Mērķauditorija ir futbola interesenti, kuri vēlas sekot vairākām sacensībām un aktīvi līdzdarboties. Platforma der gan tam, kurš reizi nedēļā pārbauda rezultātus un tabulas, gan tam, kurš regulāri iesniedz prognozes un piedalās diskusijās komentāros.

Sistēma izstrādāta kā kvalifikācijas darbs. Tehniskais steks balstīts uz JavaScript ekosistēmu: Vue 3 klienta pusē, Node.js ar Express servera pusē un MySQL datubāzē. Izstrādes

procesā ievērotas drošības prasības: bcrypt parolu šifrēšanai, JWT sesiju pārvaldībai, sanitize-html komentāru attīrīšanai.

Paskaidrojošajā rakstā aprakstīts viss izstrādes process: uzdevuma nostādne, prasību specifikācija, tehnoloģiju pamatojums, sistēmas projektēšana, datubāzes struktūra, lietotāja ceļvedis un testēšanas rezultāti.

1. UZDEVUMA NOSTĀDNE

Kvalifikācijas darba uzdevums ir izstrādāt futbola statistikas tīmekļa lietojumprogrammu "Footbalyze", kas nodrošina piekļuvi desmit futbola sacensību datiem, kā arī lietotāju personalizācijas funkcijām: izlasei, prognozēm un komentāriem.

Footbalyze ir futbola statistikas tīmekļa lietojumprogramma, kas ļauj sekot līdzi desmit futbola sacensībām: deviņām līgām un UEFA Čempionu līgai. Dati tiek iegūti no football-data.org API reāllaikā; lokālajā datubāzē glabājas tikai lietotāju kontu informācija un izlases ieraksti. Sistēmā darbojas trīs lomas: viesis, autorizēts lietotājs un administrators.

Viesis (nepieslēdzies apmeklētājs) var:

- aplūkot sākumlapu ar Premjerlīgas statistikas kopsavilkumu;
- pārlūkot komandu sarakstu no jebkuras pieejamās līgas;
- aplūkot komandas detaļu lapu ar sastāvu, treneri (ja pieejams) un pēdējo spēļu formu;
- aplūkot spēlētāju sarakstu ar pozīcijas filtru;
- aplūkot spēļu grafiku un rezultātus;
- aplūkot līgu pārskata lapu un tabulas;
- izmantot H2H (no angļu val. head-to-head – divu komandu tiešais savstarpējais salīdzinājums) rīku;
- aplūkot vārtu guvēju līderu tabulu;
- reģistrēties, izveidojot jaunu kontu;
- pieslēgties ar esošu kontu.

Autorizēts lietotājs (pieslēdzies) papildus viesim var:

- saglabāt izlasē vienu iecienīto komandu;
- saglabāt izlasē vienu iecienīto spēlētāju;
- aplūkot personīgo paneli ar iecienīto komandu un spēlētāju statistiku;
- mainīt iecienīto komandu un spēlētāju no paneļa izvēles logiem;
- iesniegt spēles rezultāta prognozi pirms spēles sākuma;
- atjaunināt iepriekš iesniegtu prognozi;

- pievienot teksta komentāru pie spēles;
- ziņot par cita lietotāja komentāru;
- rediģēt savu profilu (mainīt lietotājevārdu vai paroli);
- augšupielādēt profila attēlu (avatāru);
- izrakstīties no sistēmas.

Administrators papildus autorizētam lietotājam var:

- aplūkot visu reģistrēto lietotāju sarakstu ar lomām un statusiem;
- bloķēt jebkuru lietotāju; bloķētam lietotājam pieslēgšanās tiek liegta;
- atbloķēt bloķētu lietotāju, atjaunojot piekļuvi sistēmai;
- mainīt lietotāja lomu (lietotājs vai administrators);
- dzēst lietotāja kontu;
- aplūkot ziņotos komentārus un tos dzēst.

Lietojumgadījumu diagrammā (skat. 1.1.att.) attēlotas visas trīs lietotāju lomas un to galvenās mijiedarbības ar sistēmu.



1.1.att. Footbalyze lietojumgadījumu diagramma

2. PRASĪBU SPECIFIKĀCIJA

2.1. Ieejas un izejas informācijas apraksts

2.1.1. Ieejas informācijas apraksts

Sistēmā tiek nodrošināta šādas ieejas informācijas apstrāde, ko lietotājs ievada caur ekrānveidlapām vai failu augšupielādi. Ievaddati ir grupēti pa datu kopām atbilstoši lokālās datubāzes entītijām: lietotāji, izlase, prognozes un komentāri.

1. Informācija par **lietotājiem** sastāv no šādiem datiem.

- Lietotājvārds – unikāls lietotāja identifikators; teksts, 3 līdz 20 rakstzīmes, atļauti latīņu burti (A–Z, a–z), cipari (0–9) un pasvītrojums (_) (piem. Lietotājs_1).
- E-pasta adrese – pieslēgšanās identifikators; derīgs e-pasta formāts ar "@" un domēnu, kas satur punktu (piem. epasts@example.com).
- Parole – autentifikācijas atslēga; teksts, vismaz 8 rakstzīmes, vismaz viens lielais burts un viens cipars (piem. Parole123).
- Profila attēls (avatārs) – attēla fails; atļautie formāti JPEG, PNG, WebP, maksimālais izmērs 3 MB (piem. foto.jpg).

2. Informācija par **izlasi** sastāv no šādiem datiem.

- Izvēlētā komanda – komanda, ko lietotājs izvēlas no pieejamo komandu saraksta, nospiežot "Pievienot izlasei" (piem. Liverpool).
- Izvēlētais spēlētājs – spēlētājs, ko lietotājs izvēlas no pieejamo spēlētāju saraksta (piem. Mohamed Salah).

3. Informācija par **prognozēm** sastāv no šādiem datiem.

- Mājas komandas vārti – paredzētais vārtu skaits; vesels skaitlis diapazonā no 0 līdz 20 (piem. 2).
- Viesas komandas vārti – paredzētais vārtu skaits; vesels skaitlis diapazonā no 0 līdz 20 (piem. 1).

4. Informācija par **komentāriem** sastāv no šādiem datiem.

- Komentāra teksts – brīva teksta komentārs par spēli; 1 līdz 500 rakstzīmes. Visi HTML tagi tiek automātiski noņemti, lai novērstu XSS uzbrukumus (piem. "Šodien Liverpool bija pārliecinoši labāki.").

2.1.2. Izejas informācijas apraksts

Izejas informācija tiek attēlota ekrānā kā HTML lapas vai JSON atbildes no API. Galvenās izejas informācijas vienības:

1. Līgas tabula: komandu saraksts, sakārtots pēc punktiem dilstošā secībā. Katrai komandai tiek uzrādīts nosaukums, aizvadīto spēļu skaits, uzvaru, neizšķirtu un zaudējumu skaits, gūtie un ielaistie vārti, kā arī kopējais punktu skaits. Piemērs: Manchester City, 30 spēles, 20U 5N 5Z, vārti 65:30, 65 punkti.

2. Spēlētāja profils: informācija par konkrētu spēlētāju no ārējā API. Tiek uzrādīts vārds, komanda, pozīcija, valstspiederība un aktuālās sezonas statistika (gūtie vārti, piespēles). Piemērs: Mohamed Salah, Liverpool, labais uzbrucējs, Ēģipte, 22 vārti, 10 piespēles.

3. Spēļu grafiks ar rezultātiem: spēļu saraksts ar mājas un viesas komandu, datumu un laiku, kā arī rezultātu vai statusu. Iespējamie statusi: SCHEDULED (gaidāma), LIVE (notiek), FINISHED (pabeigta). Piemērs: Arsenal vs Chelsea, 15.04.2026. 21:00, 2:1 FINISHED.

4. Lietotāja panelis: apkopota informācija par autorizēta lietotāja darbību sistēmā. Redzama izlasē saglabātā komanda un spēlētājs, kā arī iesniegtās prognozes ar to iznākumu (precīzs/pareizs/nepareizs/gaida). Piemērs: izlase Liverpool, Mohamed Salah; prognoze Arsenal vs Chelsea 2:1, pareizs.

5. Kļūdas ziņojums: sistēmas atbilde uz nederīgu vai nepilnīgu pieprasījumu. Formāts: JSON objekts ar lauku "message" un atbilstošu HTTP statusa kodu. Piemēri: 400 "Username must be between 3 and 20 characters.", 401 "No account found with that email.", 403 "Your account has been banned."

2.1.3. Ārējās informācijas apraksts

Sistēma izmanto football-data.org kā ārējo datu avotu. Savienojums notiek caur REST API ar API atslēgu HTTP galvenē (X-Auth-Token). Servera puse kešo (uz laiku saglabā atmiņā) atbildes, lai nepārsniegtu bezmaksas plāna limitu (10 pieprasījumi minūtē).

No API tiek iegūti: komandu saraksti, spēlētāju dati, spēļu grafiks, rezultāti, līgas tabula un divu komandu tiešā salīdzinājuma (H2H) statistika. Šie dati netiek saglabāti lokālajā datubāzē.

2.2. Funkcionālās prasības

Sistēmas galvenās funkcionālās prasības ir apkopotas tabulās, katrai norādot identifikatoru, ievadu (mērķi), ievaddatus, apstrādi, izvaddatus, kļūdas paziņojumus un izsaucamās prasības. Prasību identifikatori (PR-01 līdz PR-08) tiek izmantoti arī testēšanas tabulā.

2.1. tabula

Funkcionālā prasība PR-01 – lietotāja reģistrācija

Identifikators	PR-01
Ievads	Funkcija ļauj viesim izveidot jaunu lietotāja kontu, lai piekļūtu autorizēta lietotāja funkcijām.
Ievaddati	1. Lietotājvārds – 3 līdz 20 rakstzīmes, latīņu burti, cipari un pasvītrojums. 2. E-pasta adrese – derīgs e-pasta formāts, vēl neregistrēts datubāzē. 3. Parole – vismaz 8 rakstzīmes, vismaz viens liels burts un viens cipars.
Apstrāde	1. Sistēma validē katru lauku klienta un servera pusē. 2. Parole tiek šifrēta ar bcrypt (10 salt rounds). 3. Ja visi lauki derīgi, konts tiek ierakstīts users tabulā.
Izvaddati	1. Veiksmes gadījumā konts izveidots, lietotājs novirzīts uz pieslēgšanās lapu. 2. Kļūdas gadījumā zem attiecīgā lauka parādās paziņojums; veidlapa netiek iesniegta.
Kļūdas paziņojumi	1. "Lietotājvārdam jābūt no 3 līdz 20 rakstzīmēm." 2. "E-pasts jau tiek izmantots." 3. "Parolei jābūt vismaz 8 rakstzīmes garai, ar lielo burtu un ciparu."
Izsaucamās prasības	Pieejama jebkuram viesim (neregistrētam apmeklētājam).

2.2. tabula

Funkcionālā prasība PR-02 – lietotāja pieslēgšanās

Identifikators	PR-02
Ievads	Funkcija ļauj reģistrētam lietotājam pieslēgties sistēmai un izmantot autorizēta lietotāja funkcijas.
Ievaddati	1. E-pasta adrese – reģistrēta konta e-pasts. 2. Parole – konta parole.
Apstrāde	1. Sistēma pārbauda e-pasta esamību un salīdzina paroli ar bcrypt hash. 2. Sistēma pārbauda, vai konts nav bloķēts. 3. Veiksmes gadījumā serveris izsniedz JWT tokenu.
Izvaddati	1. Veiksmes gadījumā tokens saglabāts localStorage, lietotājs pieslēgts. 2. Kļūdas gadījumā parādās paziņojums.
Kļūdas paziņojumi	1. "Nepareizs lietotājvārds vai parole." 2. "Konts ir bloķēts."
Izsaucamās prasības	Pieejama jebkuram reģistrētam lietotājam.

Funkcionālā prasība PR-04 – izlases pārvaldība

Identifikators	PR-04
Ievads	Funkcija ļauj autorizētam lietotājam saglabāt izlasē vienu komandu un vienu spēlētāju vai nomainīt tos uz citiem.
Ievaddati	Lietotāja izvēlēta komanda vai spēlētājs no pieejamo ierakstu saraksta.
Apstrāde	1. Pieprasījums tiek autorizēts ar JWT tokenu. 2. Sistēma saglabā izvēli favourites tabulā; ja izlasē jau ir komanda vai spēlētājs, tas tiek aizstāts ar jauno (upsert loģika).
Izvaddati	1. Atjaunināta izlase redzama lietotāja panelī. 2. Neautorizēta pieprasījuma gadījumā serveris atgriež 401.
Kļūdas paziņojumi	401 Unauthorized – pieprasījums bez derīga tokena.
Izsaucamās prasības	Pieejama tikai autorizētam lietotājam.

Funkcionālā prasība PR-05 – spēles rezultāta prognoze

Identifikators	PR-05
Ievads	Funkcija ļauj autorizētam lietotājam iesniegt vai atjaunināt spēles rezultāta prognozi pirms spēles sākuma.
Ievaddati	1. Mājas komandas vārti – vesels skaitlis, 0 līdz 20. 2. Viesas komandas vārti – vesels skaitlis, 0 līdz 20.
Apstrāde	1. Sistēma validē, ka vērtības ir diapazonā 0–20. 2. Ja prognoze spēlei vēl nav, tā tiek ierakstīta (INSERT). 3. Ja prognoze jau eksistē, tā tiek atjaunināta (UPDATE) – apvienotā ievietošanas/atjaunināšanas jeb upsert loģika.
Izvaddati	Saglabāta prognoze ar apstiprinājumu saskarnē.
Kļūdas paziņojumi	"Vārtu skaitlis ārpus diapazona." – ja vērtība ir mazāka par 0 vai lielāka par 20.
Izsaucamās prasības	Pieejama tikai autorizētam lietotājam.

Funkcionālā prasība PR-06 – komentāru pievienošana

Identifikators	PR-06
Ievads	Funkcija ļauj autorizētam lietotājam pievienot teksta komentāru pie spēles un ziņot par citu lietotāju komentāriem.
Ievaddati	Komentāra teksts – 1 līdz 500 rakstzīmes.
Apstrāde	1. Sistēma apstrādā tekstu ar sanitize-html, noņemot visus HTML tagus (aizsardzība pret XSS uzbrukumiem). 2. Komentārs tiek ierakstīts comments tabulā.
Izvaddati	1. Komentārs nekavējoties parādās spēles lapā bez atkārtotas ielādes. 2. Ziņotie komentāri nonāk administratora sarakstā.
Kļūdas paziņojumi	"Komentārs nevar būt tukšs." – ja teksta lauks ir tukšs.
Izsaucamās prasības	Pieejama tikai autorizētam lietotājam.

Pārējās funkcionālās prasības seko tādai pašai struktūrai. Tālāk sniegts to kopsavilkums.

- PR-03. Futbola statistikas apskate (viesis, autorizēts lietotājs): komandu, spēlētāju, spēļu, līgas tabulu un vārtu guvēju datu iegūšana no football-data.org API ar atbilžu kešošanu; komandu filtrēšana pēc līgas un nosaukuma, spēlētāju filtrēšana pēc pozīcijas; divu komandu tiešais salīdzinājums (H2H).
- PR-07. Profila pārvaldība (autorizēts lietotājs): lietotājvārda maiņa (3 līdz 20 rakstzīmes, latīņu burti, cipari un pasvītrojums), paroles maiņa (vismaz 8 rakstzīmes ar lielo burtu un ciparu), kā arī profila attēla augšupielāde (JPEG, PNG, WebP; līdz 3 MB).
- PR-08. Administratora funkcijas (administrators): reģistrēto lietotāju saraksta apskate, lietotāju bloķēšana un atbloķēšana, lietotāja lomas maiņa (lietotājs/administrators), lietotāja konta dzēšana, kā arī ziņoto komentāru apskate un dzēšana; administrators nevar bloķēt, mainīt lomu vai dzēst pats sevi.

2.3. Nefunkcionālās prasības

Nefunkcionālās prasības attiecas uz sistēmas drošību, veiktspēju, uzticamību un uzturamību.

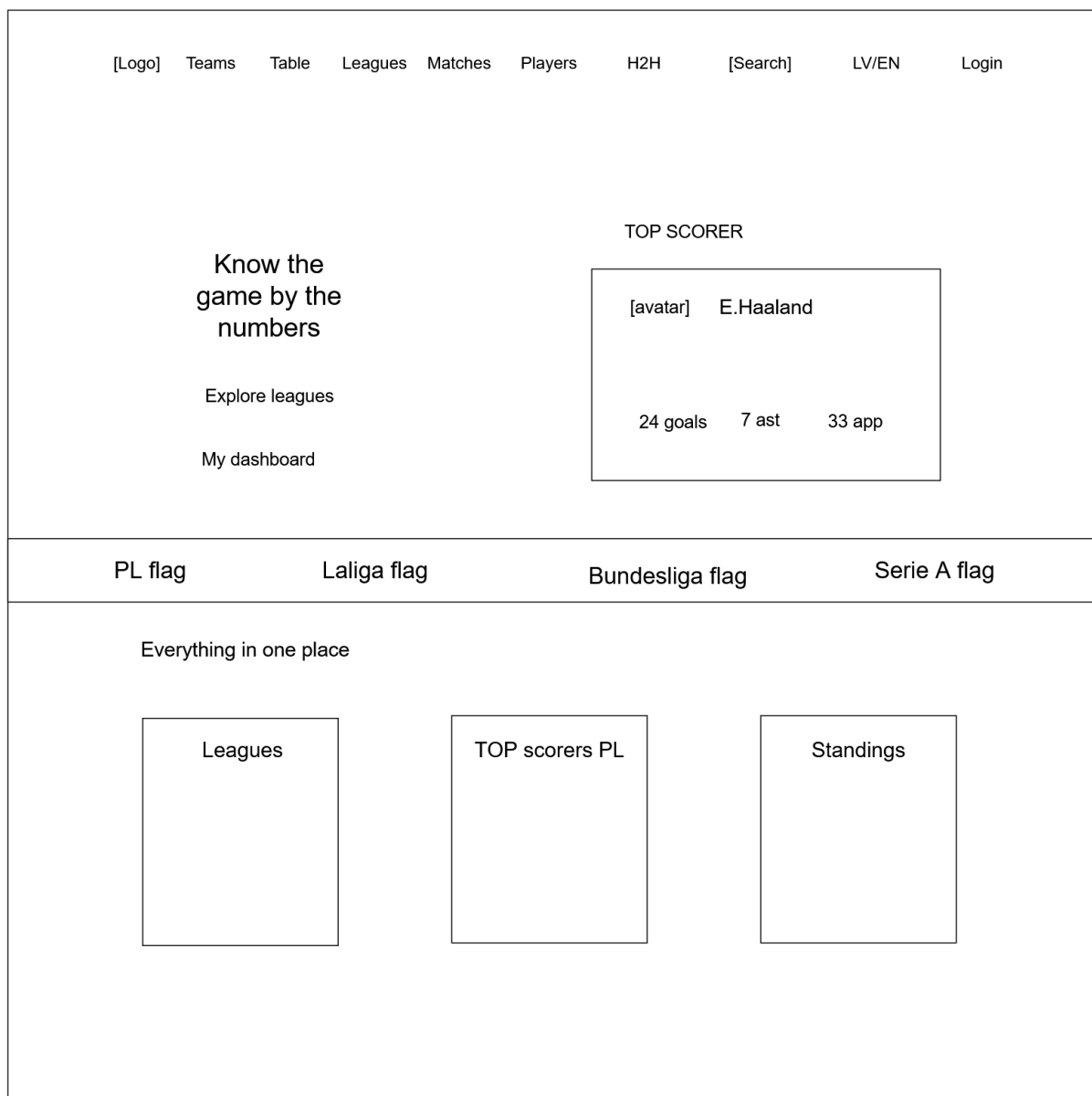
2.6. tabula

Nefunkcionālo prasību saraksts

Prasības ID	Kategorija	Apraksts
NFR-01	Drošība	Paroles glabātas kā bcrypt hash (10 salt rounds). Komentāri sanitizēti ar sanitize-html. Aizsargātie maršruti pārbauda JWT tokenu.
NFR-02	Veiktspēja	Football-data.org API atbildes tiek kešotas atmiņā, samazinot ārējo pieprasījumu skaitu. Lapas sākotnējā ielāde nedrīkst pārsniegt 3 sekundes.
NFR-03	Lietojamība	Saskarne darbojas jebkurā modernā desktopu pārlūkprogrammā. Kļūdu ziņojumi ir saprotami un precīzi norāda uz problēmu.
NFR-04	Uzticamība	Visi lietotāja ievaddati tiek validēti gan klienta, gan servera pusē. Sistēma atgriež saprotamus HTTP kļūdu kodus (400, 401, 403, 404, 500).
NFR-05	Uzturamība	Kods organizēts pēc MVC principiem: maršruti, loģika un datubāzes piekļuve ir atdalītas. Vides mainīgie glabāti .env failā.

Sistēmas lietotāja saskarnes projektēšanas procesā tika izstrādātas ekrānlogu skices (wireframes) galvenajām lapām. Skices ataino saskarnes izkārtojumu un elementu izvietojumu pirms vizuālā dizaina izstrādes.

Sākumlapas skice (skat. 2.1.att.) parāda divdaļīgo izkārtojumu: kreisajā pusē teksts un darbības pogas, labajā pusē statistikas bloks, kā arī navigācijas joslu augšā un informatīvo bloku ar trim kartītēm zemāk.



2.1.att. Sākumlapas ekrānloga skice

Spēļu lapas skice (skat. 2.2.att.) parāda spēļu saraksta izkārtojumu ar komandu nosaukumiem, datumu, laiku un rezultātu vai statusu katrai spēlei.

[Logo]

Teams

Table

Leagues

Matches

Players

H2H

[Search]

LV/EN

Login

All leagues

[PL flag] Premier League

927 goals

2.7 avg.

143 HW

90 D

106 AW

BIGGEST WIN Arsenal FC 5 - 0 Leeds United FC

Search by team

Newest first

All 339

Home win 143

Draw 90

Away win 106

All teams

AFC Bournemouth

Arsenal

. . .

Matchday 34

21 apr - 24 apr

27 apr

Man United 2 - 1 Brentford FC [H]

25 apr

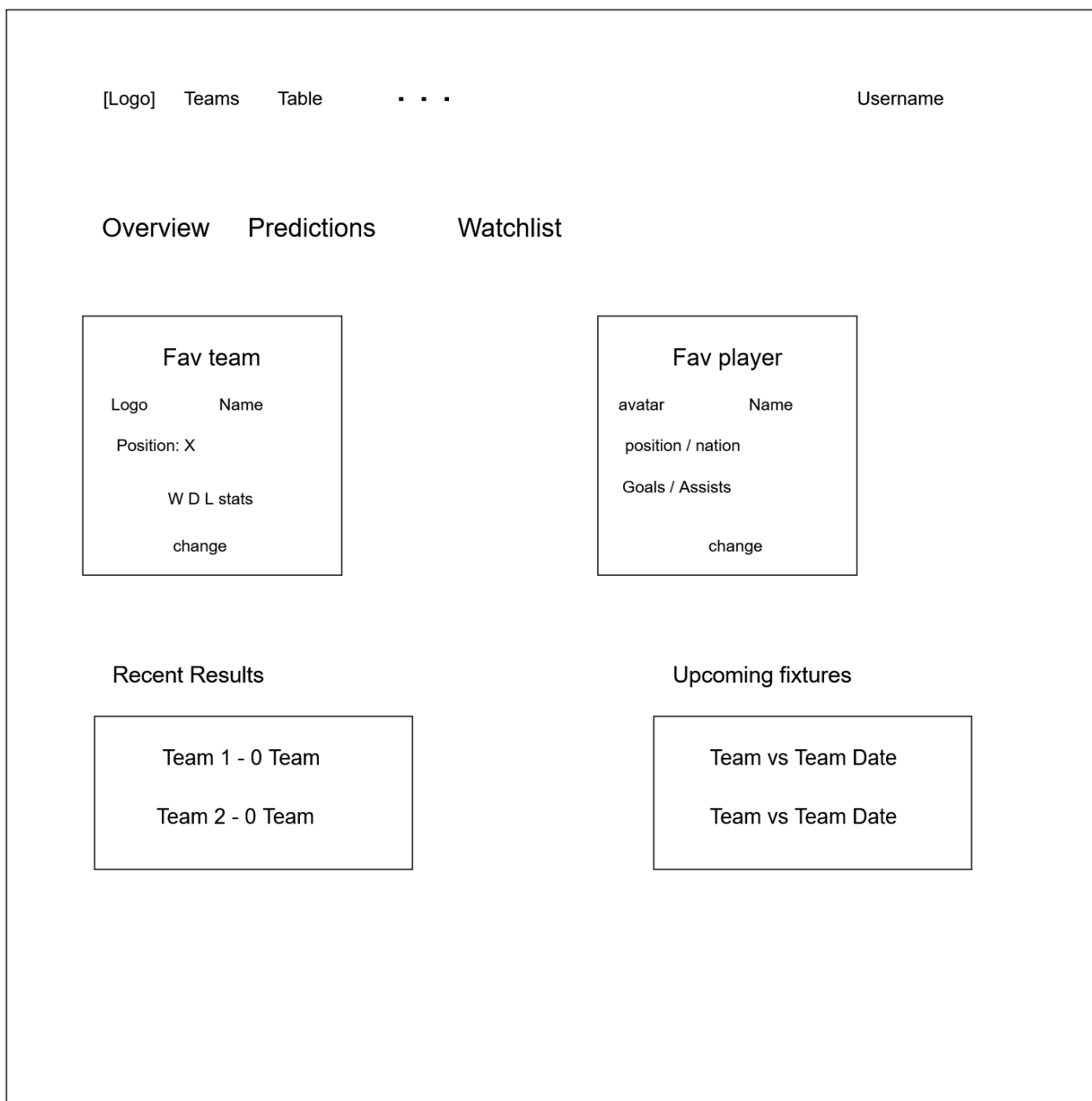
Arsenal FC 1 - 0 Newcastle FC [H]

25 apr

Liverpool FC 3 - 1 Crystal Palace [H]

2.2.att. Spēļu lapas ekrānloga skice

Lietotāja paneļa skice (skat. 2.3.att.) parāda autorizēta lietotāja skatu ar cilnēm "Overview", "Predictions" un "Watchlist". Pārskata cilnē redzamas divas kartītes (ieciņītā komanda un ieciņītais spēlētājs) un spēļu rezultātu saraksti zemāk.



2.3.att. Lietotāja paneļa ekrānloga skice

Reģistrācijas lapas skice (skat. 2.4.att.) parāda centrētu veidlapu ar laukiem lietotājvārdam, e-pastam un parolei, kā arī reāllaika validācijas norādēm zem katra lauka.

[Logo] Teams Table . . . Login / Register

Register

Username

[]

3-20 rakstzīmes

Email

[]

Password

[]

min. 8 rakstzīmes

[Register]

Already have an account?

2.4.att. Reģistrācijas lapas ekrānloga skice

3. UZDEVUMA RISINĀŠANAS LĪDZEKĻU IZVĒLES PAMATOJUMS

Viss steks balstīts uz JavaScript, kas nozīmē vienu valodu frontendā un backendā, samazina valodu pārslēgšanas izmaksas un atkarību skaitu. Katrā apakšsadaļā norādīta versija, izvēles iemesls un konkrēts pielietojums sistēmā.

3.1. Frontend tehnoloģijas

3.1.1. *Vue 3 (v3.5.13)*

Vue 3 ir JavaScript ietvars lietotāja saskarnes veidošanai [14]. Viena faila komponentes (Single File Components, .vue) apvieno veidni (template), skriptu (script) un stilu (style) vienā failā ar skaidru struktūru, kas atvieglo orientēšanos lielākā koda bāzē. Composition API (loģikas organizēšanas pieeja, kas grupē kodu pēc funkcionalitātes, nevis pēc komponentes opcijām) padara loģiku pārskatāmāku. Alternatīvi tika apsvērts React, taču Vue vienkāršākā veidņu sintakse un mazāks atkārtotā standartkoda (boilerplate) apjoms padara to piemērotāku viena izstrādātāja projektam bez lielākas komandas.

Visas Footbalyze frontend lapas (komandu apskate, spēlētāju profils, līgu tabula, lietotāja panelis) ir Vue komponentes.

3.1.2. *Vue Router (v4.5.0)*

Vue Router pārvalda klienta puses navigāciju bez lapas pārlādēšanas [13]. Navigācijas sardzes aizsargā privātus maršrutus un neautorizētus lietotājus novirza uz pieslēgšanās lapu.

3.1.3. *Vuex (v4.0.2)*

Vuex glabā lietotnes globālo stāvokli [15]. Sistēmā to izmanto autentifikācijai: JWT tokena saglabāšanai localStorage un lietotāja profila datu (lietotājevārds, loma, avatārs) atjaunināšanai. Vue ekosistēmā jaunāka alternatīva ir Pinia, taču projekts tika sākts ar Vuex, un tā API ir pilnībā pietiekams autentifikācijas stāvokļa pārvaldībai bez papildu sarežģītības.

3.1.4. *Vite (v6.2.4)*

Vite izstrādes režīmā neapkopo pakotnes, bet izmanto pārlūka iebūvēto ES moduļu atbalstu, tāpēc serveris palaižas tūlītēji [12]. Ražošanas versijas kompilēšanai izmanto Rollup.

Salīdzinājumā ar Vue CLI (kas balstās uz webpack rīka), Vite nodrošina ievērojami ātrāku pirmo palaišanu (cold start) un karsto moduļu nomaiņu (Hot Module Replacement, HMR – koda izmaiņu tūlītēju atspoguļošanu pārlūkā bez lapas pārlādēšanas), kas saīsina izstrādes iterācijas laiku.

3.2. Backend tehnoloģijas

3.2.1. Node.js (v22.14.0)

Node.js ir JavaScript izpildlaika vide, kas balstās uz Google V8 dzinēja [9]. Tā ļauj izpildīt JavaScript kodu ārpus pārlūka – servera pusē. Vienāda valoda klienta un servera pusē samazina valodu pārslēgšanas izmaksas. Programmas ieejas punkts ir backend/server.js.

3.2.2. Express (v5.1.0)

Express ir Node.js ietvars HTTP pieprasījumu maršrutēšanai [3]. Express v5 automātiski pārsūta nenokertās kļūdas no asinhronām apstrādes funkcijām uz kļūdu apstrādes starpprogrammatūru (middleware – funkcijas, kas apstrādā pieprasījumu pirms galīgās atbildes), kas samazina atkārtotajā standartkoda apjomu. Sistēmā tas pārvalda visus API maršrutus: autentifikāciju, futbola datus, izlasi un administrāciju.

3.2.3. MySQL 8.3 un mysql2 (v3.14.1)

MySQL ir relāciju datubāzu pārvaldības sistēma [8]. mysql2 draiveris nodrošina sagatavotos vaicājumus (prepared statements – vaicājumus, kuru vērtības tiek nodotas atsevišķi no SQL teksta), kas aizsargā pret SQL injekciju (uzbrukumu, kur ievaddatos ievieto ļaunprātīgu SQL kodu). Lokālajā datubāzē glabāti tikai lietotāju dati: tabulas users, favourites, predictions un comments. Futbola dati tiek iegūti no ārējā API. PostgreSQL būtu tikpat derīga alternatīva, taču MySQL ir plašāk izmantota studentu hostinga vidēs un Coolify izvietojumā, kas samazināja konfigurācijas sarežģītību.

3.2.4. jsonwebtoken (v9.0.2)

JWT (RFC 7519) tokens satur lietotāja datus (ID, lietotājvārds, loma) un ir parakstīts ar servera slepeno atslēgu [6]. Serveris validē tokenu katram aizsargātam pieprasījumam bez papildu datubāzes vaicājuma. Alternatīva būtu sesiju autentifikācija ar servera puses sesiju glabāšanu (express-session + Redis), taču tā prasa papildu infrastruktūru. JWT ir bezstāvokļa (serveris

neglabā sesijas datus atmiņā) un piemērots REST API [1], kur klienta un servera puse darbojas atsevišķos procesos.

3.2.5. *bcryptjs* (v3.0.2)

bcrypt izmanto adaptīvu jaucējfunkciju (hash) ar pievienotu nejaušu virkni (salt) un skaitļošanas sarežģītības faktoru (work factor), kas padara brutāla spēka uzbrukumus laukietilpīgus [2]. Reģistrējoties, parole tiek šifrēta ar 10 salt kārtām. Atklātā teksta (plaintext) parole netiek saglabāta datubāzē.

3.2.6. *sanitize-html* (v2.17.2)

sanitize-html ar konfigurāciju { allowedTags: [], allowedAttributes: {} } izfiltrē visus HTML tagus no lietotāju komentāriem, saglabājot tikai parastu tekstu [11]. Tas novērš XSS uzbrukumu iespēju [10].

3.2.7. *multer* (v2.1.1)

multer apstrādā multipart/form-data pieprasījumus failu augšupielādei [7]. Konfigurēts pieļaut attēlu failus (JPEG, PNG, WebP) līdz 3 MB. Faili tiek saglabāti backend/uploads/ ar unikālu nosaukumu.

3.3. Ārējie pakalpojumi

3.3.1. *football-data.org* API

football-data.org nodrošina piekļuvi futbola statistikai par 10 Eiropas līgām [5]. Bezmaksas plāns atļauj 10 pieprasījumus minūtē. Sistēmā šis API ir vienīgais futbola datu avots. Backend ievieš atmiņas kešatmiņu (footballApi.js), lai nepārsniegtu API limitu.

3.3.2. *Palīgbibliotēkas*

dotenv (v16.5.0): vides mainīgo ielāde no .env faila. Sensitīvie dati (datubāzes parole, JWT atslēga, API atslēga) tiek glabāti ārpus versiju kontroles.

cors (v2.8.5): Cross-Origin Resource Sharing galveņu iestatīšana, kas ļauj frontend (ports 5173) sazināties ar backend (ports 3000).

express-rate-limit (v8.3.2): pieprasījumu ātruma ierobežošana, kas aizsargā API pret pārslodzi un brutāla spēka uzbrukumiem [4].

Jest (v30.3.0) un Supertest (v7.2.2): backend API integrācijas testēšana. Testi atrodas backend/tests/ direktorijā.

3.4. Aizgūto moduļu saraksts

Projektā izmantotās aizgūtās bibliotēkas ir standarta npm pakotnes, kuras ievietotas kā atkarības (dependencies/devDependencies) un nav tieši iekļautas projekta pirmkodā. Tās neveido vairāk par vienu trešdaļu no kopējā koda apjoma.

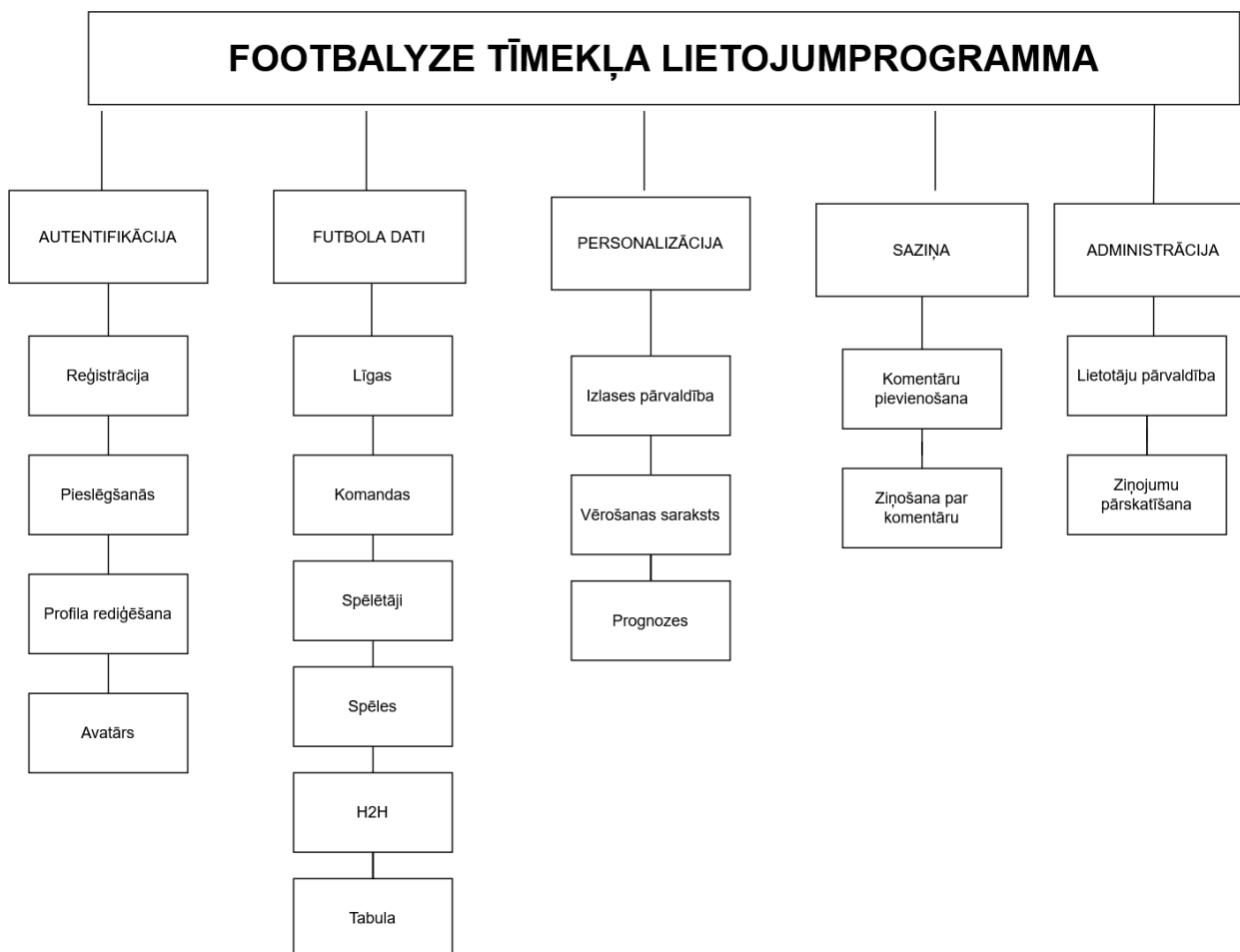
- bcryptjs (v3.0.2) – paroles šifrēšana; npm package (<https://www.npmjs.com/package/bcryptjs>)
- cors (v2.8.5) – CORS galveņu apstrāde; npm package (<https://www.npmjs.com/package/cors>)
- dotenv (v16.5.0) – vides mainīgo ielāde; npm package (<https://www.npmjs.com/package/dotenv>)
- express (v5.1.0) – HTTP servera ietvars; npm package (<https://www.npmjs.com/package/express>)
- express-rate-limit (v8.3.2) – pieprasījumu ierobežošana; npm package (<https://www.npmjs.com/package/express-rate-limit>)
- jsonwebtoken (v9.0.2) – JWT izsniegšana un verifikācija; npm package (<https://www.npmjs.com/package/jsonwebtoken>)
- multer (v2.1.1) – failu augšupielāde; npm package (<https://www.npmjs.com/package/multer>)
- mysql2 (v3.14.1) – MySQL savienojums; npm package (<https://www.npmjs.com/package/mysql2>)
- sanitize-html (v2.17.2) – HTML sanitizācija; npm package (<https://www.npmjs.com/package/sanitize-html>)
- axios (v1.15.1) – HTTP pieprasījumi no frontend; npm package (<https://www.npmjs.com/package/axios>)
- vue (v3.5.13) – UI ietvars; npm package (<https://www.npmjs.com/package/vue>)
- vue-i18n (v9.14.5) – internacionalizācija; npm package (<https://www.npmjs.com/package/vue-i18n>)
- vue-router (v4.5.0) – klienta puses maršrutēšana; npm package (<https://www.npmjs.com/package/vue-router>)

- vuex (v4.0.2) – globālā stāvokļa pārvaldība; npm package
(<https://www.npmjs.com/package/vuex>)
- vite (v6.2.4) – frontend būvēšanas rīks; npm package
(<https://www.npmjs.com/package/vite>)

4. PROGRAMMATŪRAS PRODUKTA MODELĒŠANA UN PROJEKTĒŠANA

4.1. Sistēmas arhitektūra

Footbalyze ir sadalīta piecās apakšsistēmās: autentifikācija, futbola dati, personalizācija, saziņa un administrācija. Sistēmas funkcionālā sadalījuma hierarhija attēlota 4.1.att.



4.1.att. Footbalyze funkcionālās dekompozīcijas diagramma

Autentifikācijas apakšsistēma pārvalda lietotāju piekļuvi sistēmai. Tajā ietilpst četri moduļi: reģistrācija, pieslēgšanās, profila rediģēšana un avatāra augšupielāde.

Futbola datu apakšsistēma nodrošina piekļuvi ārējā API datiem. Moduļi aptver līgu sarakstu, komandu statistiku, spēlētāju profilus, spēļu grafiku un rezultātus, divu komandu tiešo salīdzinājumu un līgas tabulu.

Personalizācijas apakšsistēma ļauj lietotājam pārvaldīt personīgo saturu. Tā ietver izlases pārvaldību, vērošanas sarakstu spēlētājiem un rezultātu prognozes.

Saziņas apakšsistēma nodrošina lietotāju mijiedarbību. Tajā ietilpst komentāru pievienošana pie spēlēm un ziņošana par neatbilstošiem komentāriem.

Administrācijas apakšsistēma paredzēta administratora darbībām. Tā ietver lietotāju pārvaldību (bloķēšanu, atbloķēšanu, lomas maiņu un kontu dzēšanu) un ziņoto komentāru pārskatīšanu un dzēšanu.

4.2. Sistēmas ER modelis

Lokālā datubāze satur piecas galvenās entītijas: LIETOTĀJS, IZLASE, PROGNOZE, KOMENTĀRS un VĒROJUMI, kā arī vienu saišu tabulu ZIŅOJUMS. Entītijū–attiecību (ER) modelis Čena notācijā attēlots 1. pielikumā. Katra saite zemāk aprakstīta abos virzienos.

LIETOTĀJS – IZLASE (1:1). Viens LIETOTĀJS glabā vienu IZLASI, bet viena IZLASE pieder tieši vienam LIETOTĀJAM. Izlasē var saglabāt vienu komandu un vienu spēlētāju.

LIETOTĀJS – PROGNOZE (1:N). Viens LIETOTĀJS var iesniegt daudzas PROGNOZES, bet katra PROGNOZE pieder tikai vienam LIETOTĀJAM. Vienai spēlei lietotājs var iesniegt tikai vienu prognozi.

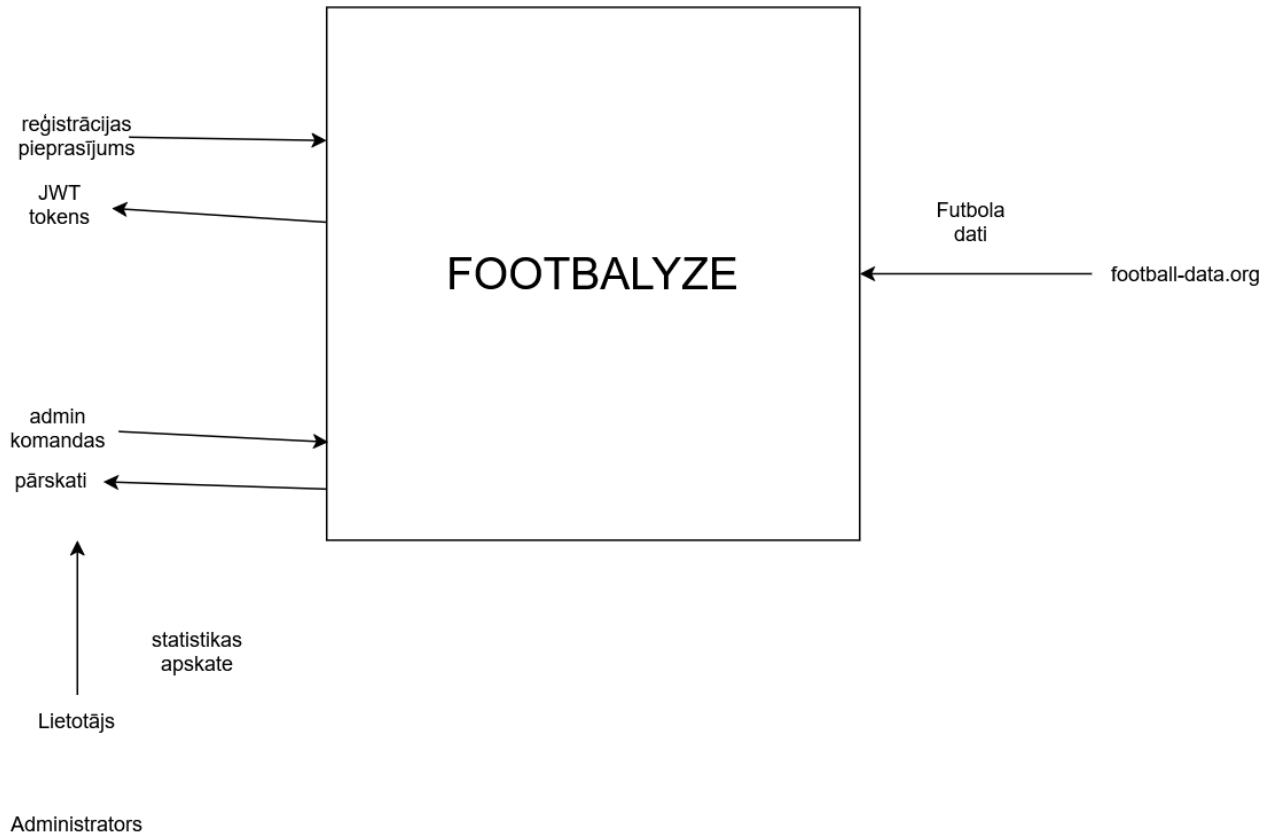
LIETOTĀJS – KOMENTĀRS (1:N). Viens LIETOTĀJS var rakstīt daudzus KOMENTĀRUS, bet vienam KOMENTĀRAM var būt tikai viens autors.

LIETOTĀJS – VĒROJUMI (1:N). Viens LIETOTĀJS var uzturēt daudzus VĒROJUMUS, bet katrs VĒROJUMU ieraksts pieder tikai vienam LIETOTĀJAM. Vērojumi ir spēlētāji, kuriem lietotājs seko.

LIETOTĀJS – KOMENTĀRS, ziņošana (N:M). Vienu KOMENTĀRU var ziņot daudzi LIETOTĀJI, un viens LIETOTĀJS var ziņot daudzus KOMENTĀRUS. Šo daudzi pret daudziem saiti realizē saišu tabula ZIŅOJUMS (comment_reports), kas satur Comment_ID un Reporter_ID ar unikālu ierobežojumu, kas novērš atkārtotu ziņošana.

4.3. Funkcionālais sistēmas modelis

Funkcionālais sistēmas modelis ir attēlots ar datu plūsmu diagrammām (DFD). Konteksta līmeņa DFD (skat. 4.2.att.) parāda sistēmu kā vienu vienību un tās mijiedarbību ar ārējiem aktoriem: reģistrētu lietotāju, administratoru un football-data.org API.

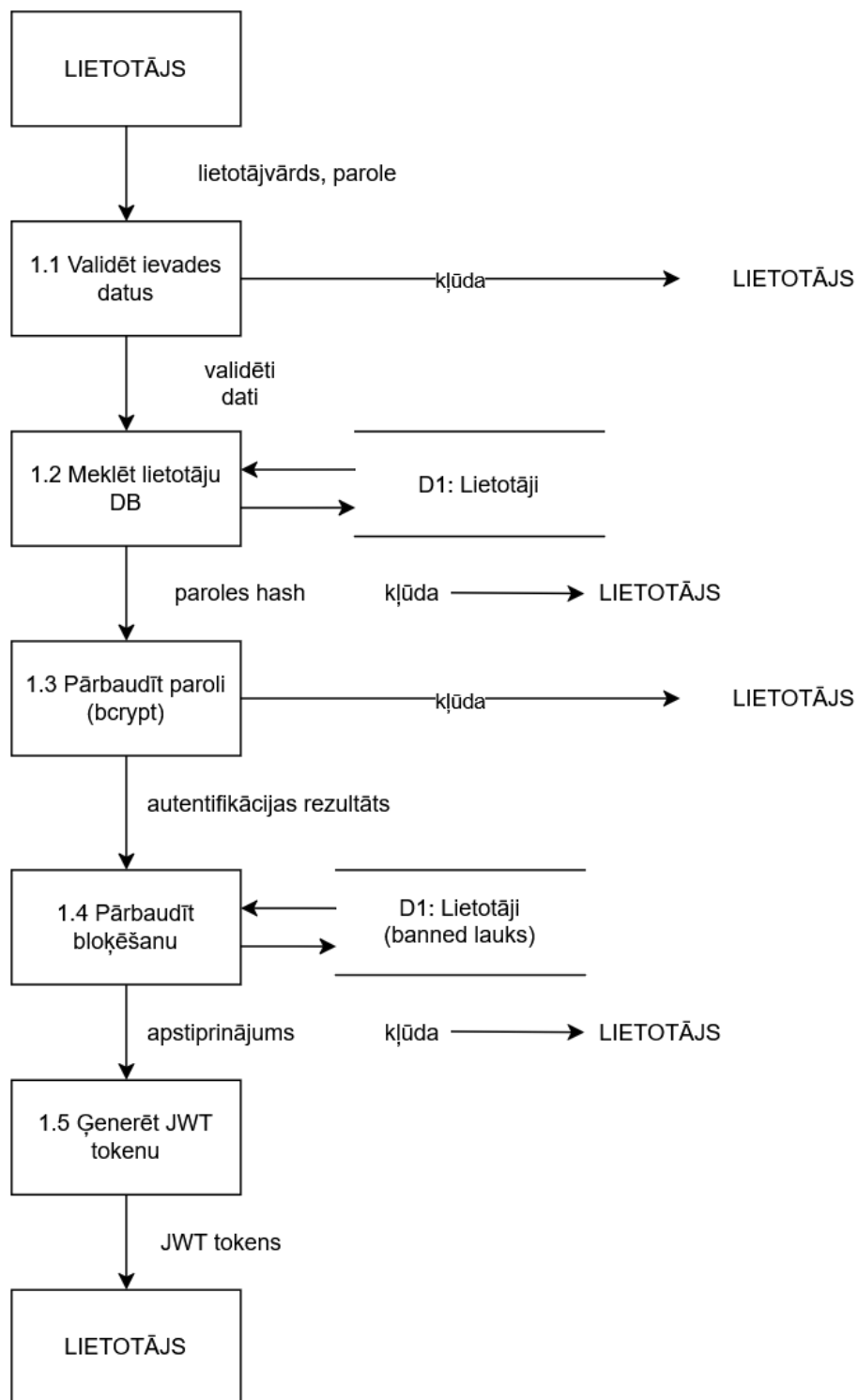


4.2.att. Footbalyze konteksta līmeņa datu plūsmas diagramma

Tika izvēlēti astoņi sistēmas moduļi. Katram izstrādāta DFD diagramma apakšējā (detalizētajā) līmenī.

4.3.1. Lietotāja pieslēgšanās modulis

Modulis nodrošina lietotāja autentifikāciju sistēmā, pārbaudot akreditācijas datus un izsniedzot JWT tokenu (skat. 4.3.att.).



4.3.att. Lietotāja pieslēgšanās moduļa datu plūsmas diagramma

Datu apstrādes algoritma apraksts – Lietotāja pieslēgšanās modulis

Lietotājs ievada e-pastu un paroli pieslēgšanās formā.

1.1. process – Validēt ievades datus. Sistēma pārbauda, vai abi lauki ir aizpildīti. Tukšu lauku gadījumā apstrāde apstājas ar kļūdas ziņojumu.

1.2. process – Meklēt lietotāju datubāzē. Tiek veikts SELECT vaicājums tabulā users pēc ievadītā lietotājvārda. Ja netiek atrasts, kļūdas ziņojums.

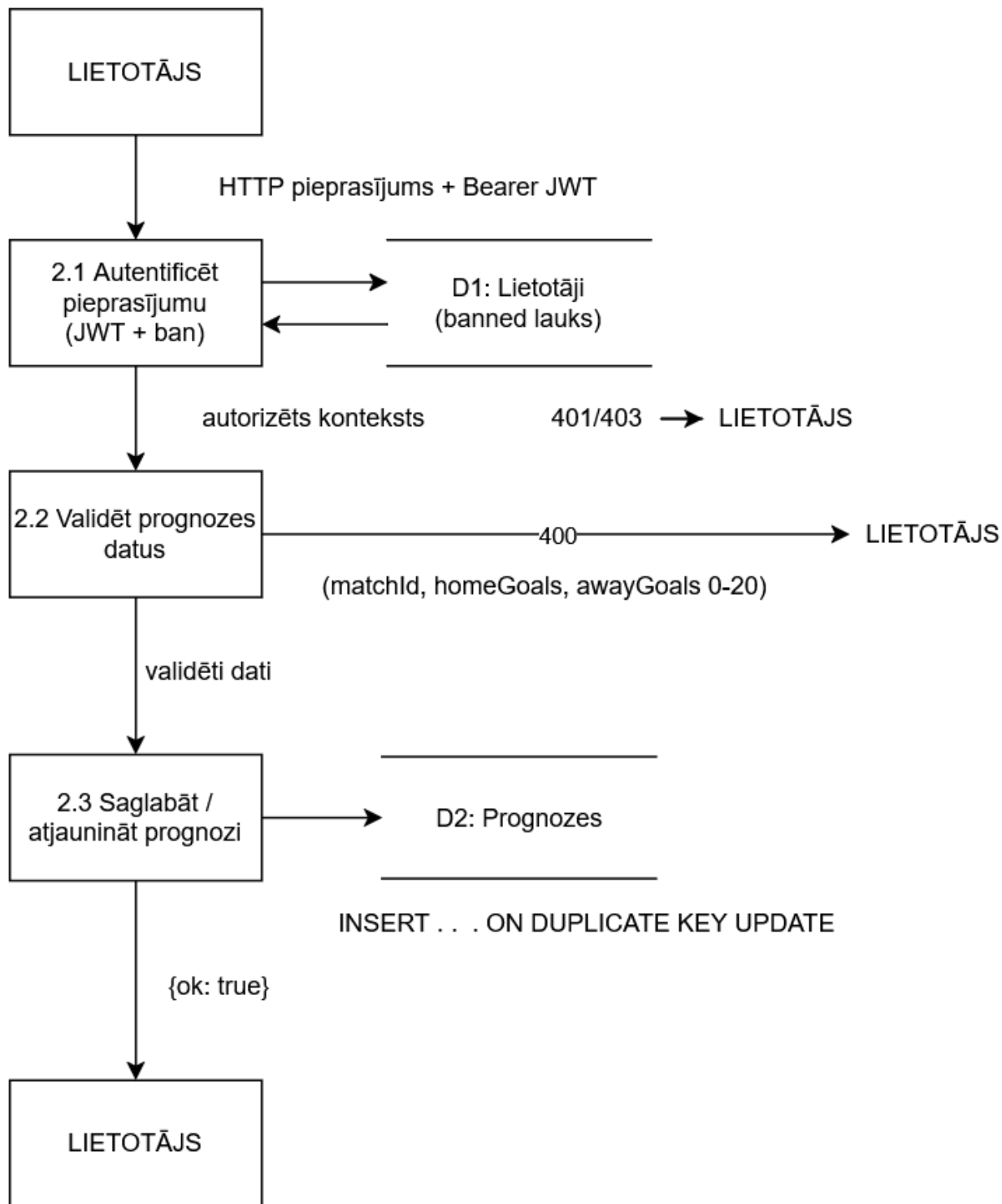
1.3. process – Pārbaudīt paroli. Datubāzes paroles hash tiek salīdzināts ar ievadīto paroli, izmantojot bcrypt.compare(). Nesakritības gadījumā tiek atgriezta kļūda.

1.4. process – Pārbaudīt bloķēšanas statusu. Tiek nolasīts banned lauks. Ja vērtība ir 1, tiek atgriezts "Konts ir bloķēts".

1.5. process – Ģenerēt JWT tokenu. Tiek ģenerēts JWT ar lietotāja ID, lietotājvārdu un lomu. Tokens tiek nosūtīts lietotājam un saglabāts localStorage.

4.3.2. Prognozes iesniegšanas modulis

Modulis nodrošina autorizētam lietotājam iespēju iesniegt rezultātu prognozi. Ja ieraksts šai spēlei jau eksistē, tas tiek atjaunināts (upsert loģika) (skat. 4.4.att.).



4.4.att. Prognozes iesniegšanas moduļa datu plūsmas diagramma

Datu apstrādes algoritma apraksts – Prognozes iesniegšanas modulis

Autorizēts lietotājs ievada prognozēto mājas un viesu vārtu skaitu.

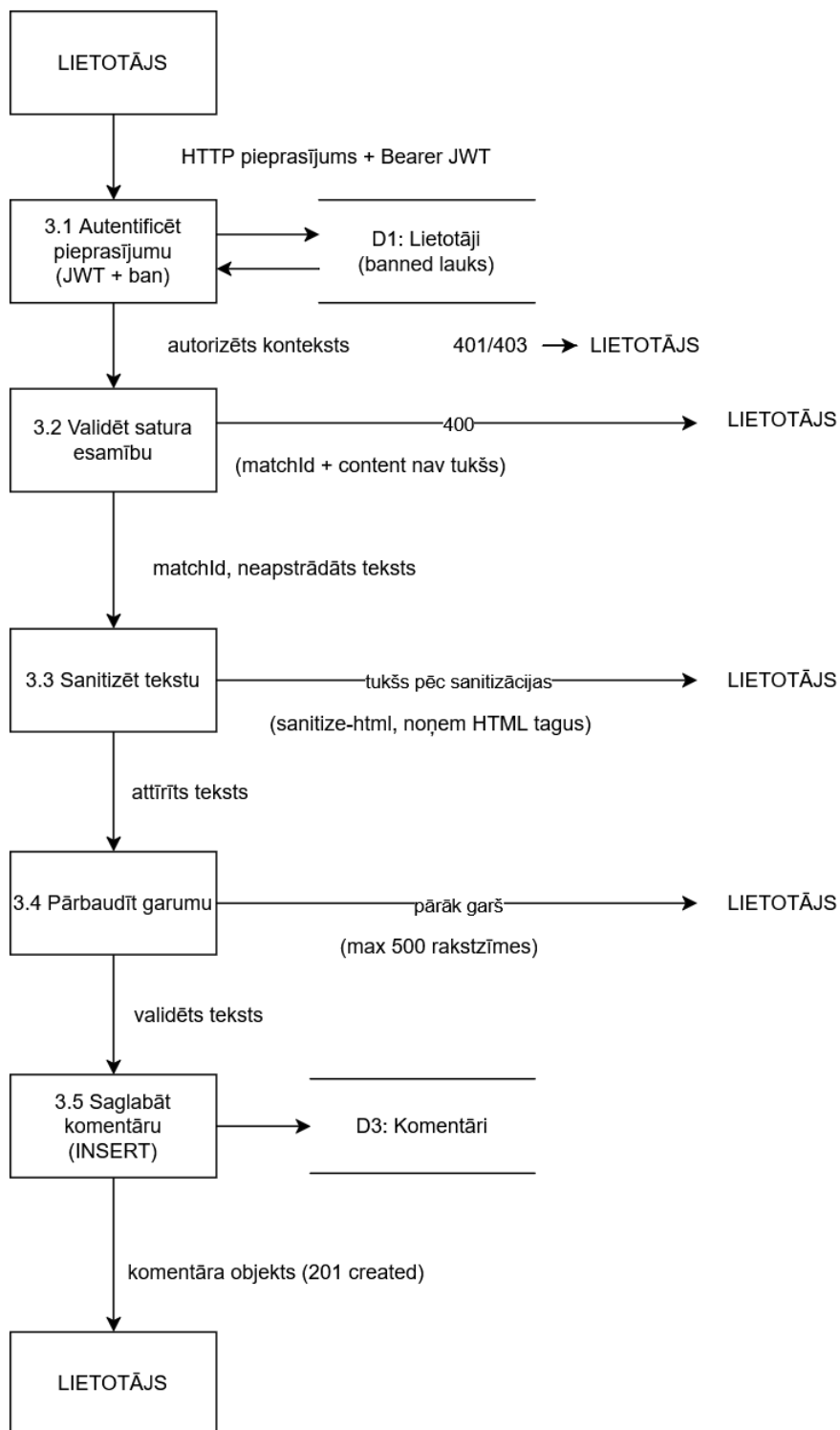
2.1. process – Autenticēt pieprasījumu. Serveris verificē Bearer JWT tokenu un pārbauda, vai konts nav bloķēts.

2.2. process – Validēt prognozes datus. Pārbauda matchId, homeGoals un awayGoals esamību. Vārtu skaitļi tiek pārvērsti veselos skaitļos un pārbaudīti diapazonā 0–20.

2.3. process – Saglabāt vai atjaunināt prognozi. Tabulā predictions tiek izpildīts INSERT ... ON DUPLICATE KEY UPDATE. Unikālais ierobežojums (User_ID, Match_ID) nodrošina vienu prognozi vienai spēlei.

4.3.3. Komentāra pievienošanas modulis

Modulis nodrošina autorizētam lietotājam iespēju pievienot komentāru. Pirms saglabāšanas saturs tiek sanitizēts pret XSS ievainojamībām (skat. 4.5.att.).



4.5.att. Komentāra pievienošanas moduļa datu plūsmas diagramma

Datu apstrādes algoritma apraksts – Komentāra pievienošanas modulis

Lietotājs ievada komentāra tekstu un nospiež "Nosūtīt".

3.1. process – Autenticēt pieprasījumu. Serveris verificē JWT tokenu un pārbauda bloķēšanas statusu.

3.2. process – Validēt satura esamību. Pārbauda matchId un content lauku esamību; content nedrīkst būt tukšs.

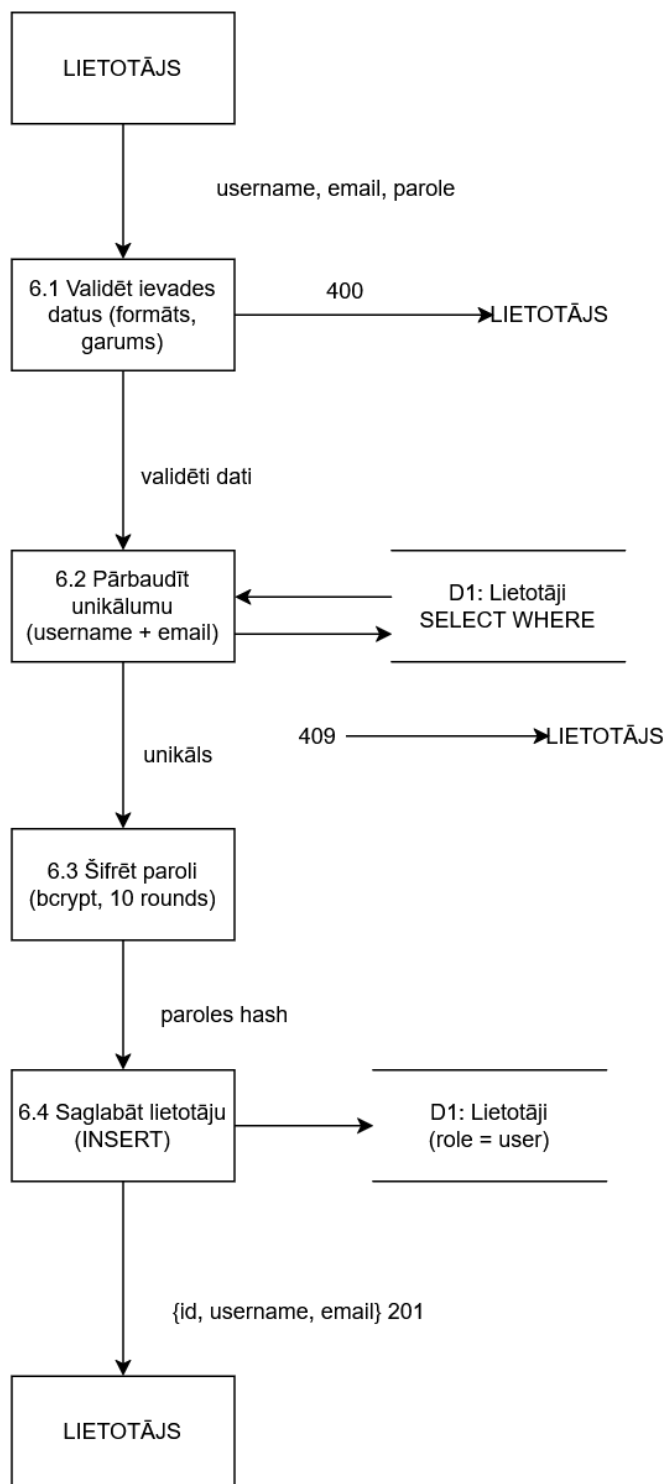
3.3. process – Sanitizēt tekstu. sanitize-html ar { allowedTags: [], allowedAttributes: {} } noņem visus HTML tagus. Pēc sanitizācijas tukšs teksts tiek noraidīts.

3.4. process – Pārbaudīt garumu. Sanitizētā teksta garums nedrīkst pārsniegt 500 rakstzīmes.

3.5. process – Saglabāt komentāru. Tabulā comments tiek ierakstīts jauns ieraksts. Serveris atgriež jaunā komentāra objektu ar HTTP 201.

4.3.4. Reģistrācijas modulis

Modulis nodrošina jauna lietotāja konta izveidi. Pirms saglabāšanas tiek validēti ievaddati, pārbaudīts e-pasta unikālums un šifrēta parole (skat. 4.6.att.).



4.6.att. Reģistrācijas moduļa datu plūsmas diagramma

Datu apstrādes algoritma apraksts – Reģistrācijas modulis

Lietotājs ievada reģistrācijas veidlapā lietotājvārdu, e-pastu un paroli.

4.1. process – Validēt ievades datus. Pārbauda lauku aizpildītību un paroles garumu (≥ 8 rakstzīmes).

4.2. process – Pārbaudīt e-pasta unikālumu. SELECT vaicājums tabulā users. Ja e-pasts jau eksistē, tiek atgriezts kļūdas ziņojums.

4.3. process – Šifrēt paroli. bcryptjs.hash() ar 10 salt rounds ģenerē paroles hash.

4.4. process – Saglabāt lietotāju. INSERT tabulā users ar lietotājvārdu, e-pastu, hash un noklusēto lomu "user". Veiksmīgi saglabājot, lietotājs saņem apstiprinājumu.

4.3.5. Profila rediģēšanas modulis

Modulis ļauj autorizētam lietotājam mainīt lietotājvārdu vai paroli. Pirms izmaiņām tiek verificēts JWT tokens un validēti jaunie dati. Moduļa datu plūsmas diagramma attēlota 2. pielikumā.

Datu apstrādes algoritma apraksts – Profila rediģēšanas modulis

Autorizēts lietotājs profila lapā ievada jaunus datus.

5.1. process – Autentificēt pieprasījumu. JWT tokens tiek verificēti; neautorizētiem: 401.

5.2. process – Validēt jaunos datus. Pārbauda lauku esamību, garuma prasības un lietotājvārda unikālumu. Jaunā parole tiek šifrēta ar bcrypt.

5.3. process – Atjaunināt lietotāja ierakstu. UPDATE tabulā users pēc lietotāja ID no JWT. Veiksmīgi; lietotājs saņem apstiprinājumu.

4.3.6. Izlases pārvaldības modulis

Modulis nodrošina autorizētam lietotājam iespēju pievienot vai noņemt komandas un spēlētājus no izlases. Dati glabājas tabulā favourites. Moduļa datu plūsmas diagramma attēlota 2. pielikumā.

Datu apstrādes algoritma apraksts – Izlases pārvaldības modulis

Autorizēts lietotājs nospiež "Pievienot izlasei" vai "Noņemt no izlases".

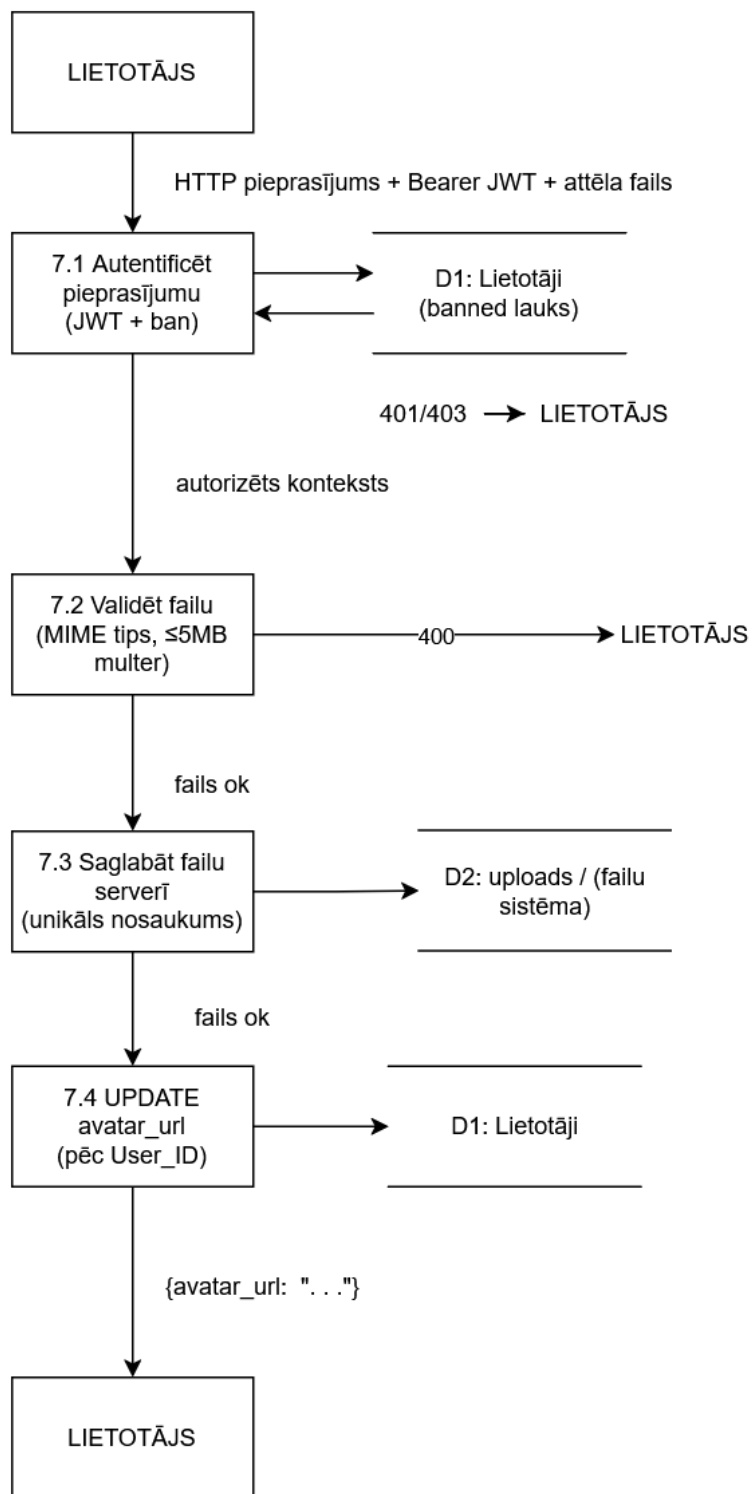
6.1. process – Autentificēt pieprasījumu. JWT tokens tiek verificēts. Neautorizētiem: 401.

6.2. process – Validēt darbību. Pārbauda Team_ID vai Player_ID esamību un darbības veidu.

6.3. process – Pievienot vai noņemt no izlases. INSERT vai DELETE tabulā favourites. Unikālais ierobežojums novērš dublikātus. Atgriež apstiprinājumu.

4.3.7. Avatāra augšupielādes modulis

Modulis ļauj lietotājam augšupielādēt profila attēlu. Fails tiek validēts, saglabāts serverī un lietotāja ierakstā tiek atjaunināts avatar_url (skat. 4.7.att.).



4.7.att. Avatāra augšupielādes moduļa datu plūsmas diagramma

Datu apstrādes algoritma apraksts – Avatāra augšupielādes modulis

Autorizēts lietotājs izvēlas attēla failu un apstiprina augšupielādi.

7.1. process – Autenticēt pieprasījumu. JWT tokens verificēts. Neautorizētiem: 401.

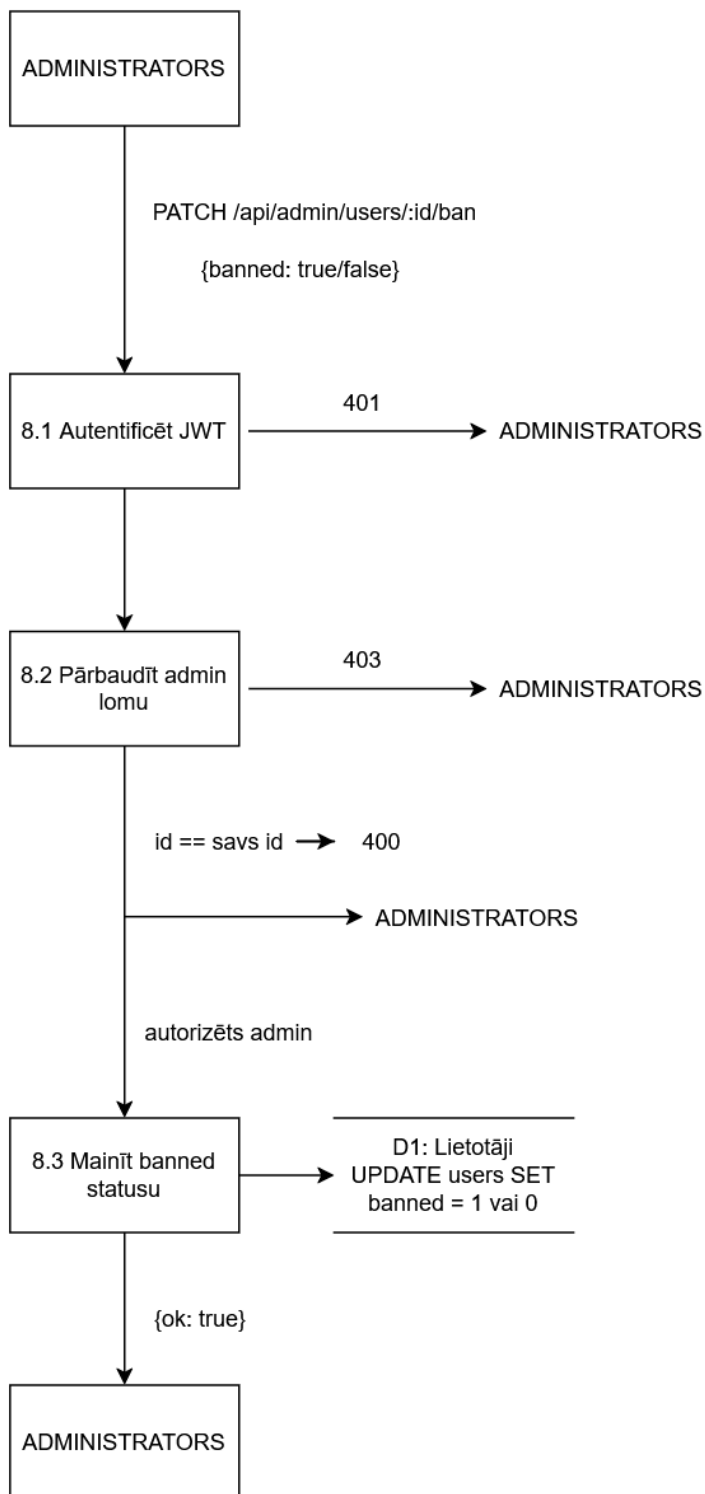
7.2. process – Validēt failu. multer pārbauda MIME tipu (JPEG/PNG/WebP) un izmēru (≤ 3 MB).

7.3. process – Saglabāt failu serverī. Fails saglabāts uploads/ ar unikālu nosaukumu.

7.4. process – Atjaunināt avatar_url. UPDATE tabulā users. Serveris atgriež jauno avatar_url.

4.3.8. Lietotāja bloķēšanas modulis

Modulis nodrošina administratoram iespēju bloķēt vai atbloķēt lietotāja kontu (skat. 4.8.att.).



4.8.att. Lietotāja bloķēšanas moduļa datu plūsmas diagramma

Datu apstrādes algoritma apraksts – Lietotāja bloķēšanas modulis

Administrators administrācijas panelī atlasa lietotāju un nospiež "Bloķēt".

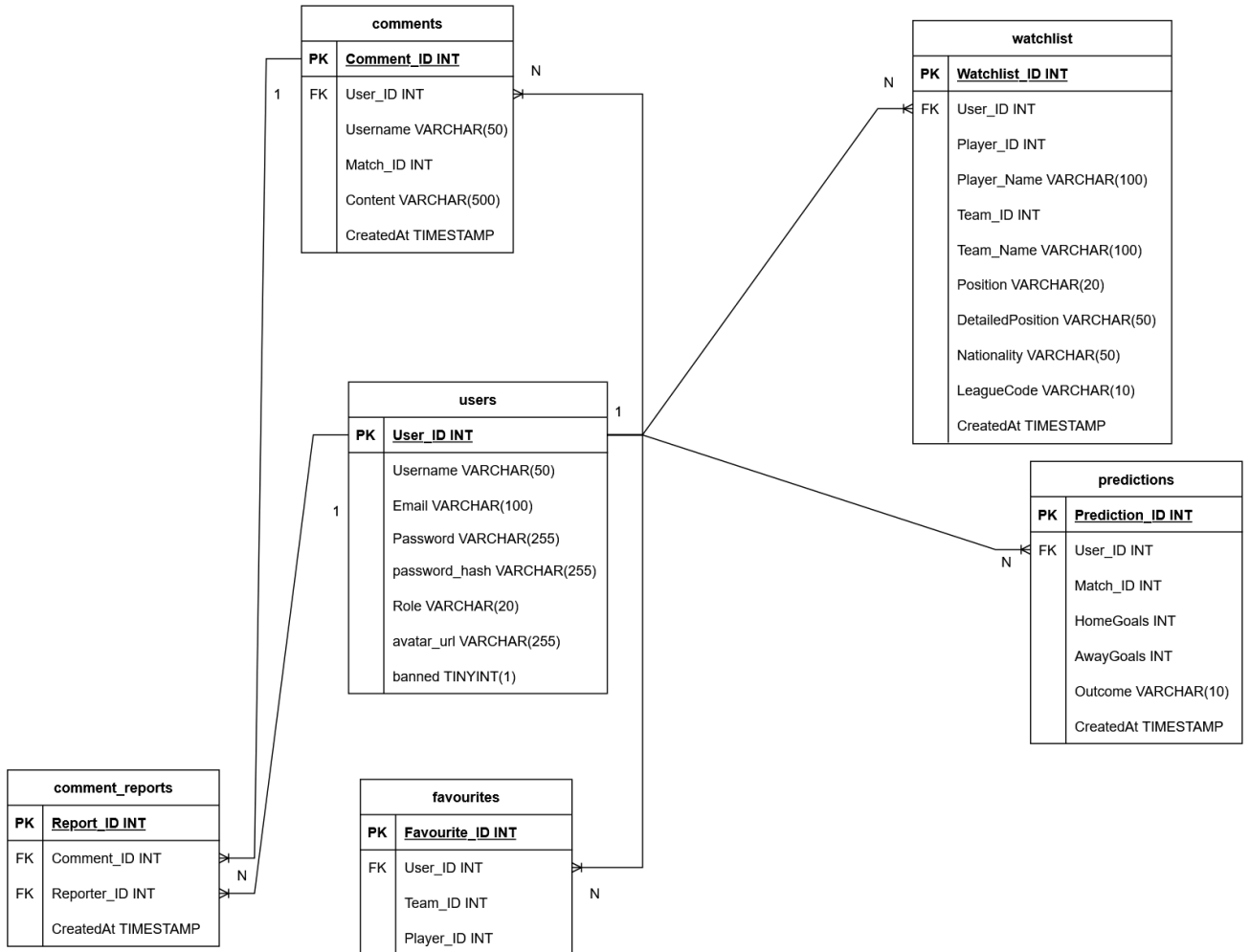
8.1. process – Autenticēt un pārbaudīt lomu. JWT tokens verificēts un pārbaudīts, vai Role ir "admin". Citiem: 403.

8.2. process – Atrast mērķa lietotāju. SELECT tabulā users pēc URL parametra User_ID. Nav atrasts: 404.

8.3. process – Atjaunināt bloķēšanas statusu. UPDATE tabulā users: banned = 1 vai 0. Bloķētais lietotājs saņems "Konts ir bloķēts" pie nākamās pieslēgšanās.

5. DATU STRUKTŪRU APRAKSTS

Footbalyze lokālā datubāze sastāv no sešām tabulām: users, favourites, predictions, comments, comment_reports un watchlist. Tabulu saišu shēma attēlota 5.1.att.



5.1.att. Datubāzes tabulu saišu shēma

Tabula users glabā visu reģistrēto lietotāju datus: autentifikācijai, lomu pārvaldībai un profila attēlam. Katra rinda atbilst vienam lietotāja kontam.

Tabulas users lauku apraksts

Nr.	Lauka nosaukums	Datu tips	Atslēga	Ierobežojumi	Apraksts
1	User_ID	INT	PK, AI	NOT NULL	Unikāls lietotāja identifikators, primārā atslēga
2	Username	VARCHAR(50)	–	NOT NULL, UNIQUE	Lietotāja izvēlētais vārds (3–20 rakstzīmes)
3	Email	VARCHAR(100)	–	NOT NULL, UNIQUE	E-pasta adrese, izmantota pieslēgšanās
4	password_hash	VARCHAR(255)	–	NOT NULL	Paroles bcrypt hash (10 salt rounds)
5	Role	VARCHAR(20)	–	NOT NULL	Loma sistēmā: "user" vai "admin"
6	avatar_url	VARCHAR(255)	–	NULL	Profila attēla faila ceļš serverī
7	banned	TINYINT(1)	–	NOT NULL	Bloķēšanas statuss: 0 = aktīvs, 1 = bloķēts

Tabula favourites glabā katra lietotāja iecienītās komandas un spēlētāja identifikatorus no football-data.org API. Katram lietotājam var būt ne vairāk kā viena komanda un viens spēlētājs.

Tabulas favourites lauku apraksts

Nr.	Lauka nosaukums	Datu tips	Atslēga	Ierobežojumi	Apraksts
1	Favourite_ID	INT	PK, AI	NOT NULL	Unikāls izlases ieraksta identifikators
2	User_ID	INT	FK → users.User_ID	NOT NULL	Atsauce uz lietotāju, kuram pieder izlase
3	Team_ID	INT	–	NULL	Iecienītās komandas ID no football-data.org
4	Player_ID	INT	–	NULL	Iecienītā spēlētāja ID no football-data.org
5	Player_Name	VARCHAR(100)	–	NULL	Spēlētāja vārds, saglabāts pievienošanas brīdī
6	Player_Team_ID	INT	–	NULL	Spēlētāja komandas ID pievienošanas brīdī
7	Player_Team_Name	VARCHAR(100)	–	NULL	Spēlētāja komandas nosaukums
8	Position	VARCHAR(20)	–	NULL	Vispārīgā pozīcija (Goalkeeper, Defender utt.)
9	DetailedPosition	VARCHAR(50)	–	NULL	Detalizētā pozīcija (Centre-Back utt.)
10	Nationality	VARCHAR(50)	–	NULL	Spēlētāja tautība
11	LeagueCode	VARCHAR(10)	–	NULL	Līgas kods (PL, PD, BL1 utt.)

Tabula predictions glabā autorizētu lietotāju iesniegtas spēļu rezultātu prognozes. Katram lietotājam var būt ne vairāk kā viena prognoze uz katru spēli (upsert loģika: atkārtota iesniegšana atjaunina esošo ierakstu).

5.3. tabula

Tabulas predictions lauku apraksts

Nr.	Lauka nosaukums	Datu tips	Atslēga	Ierobežojumi	Apraksts
1	Prediction_ID	INT	PK, AI	NOT NULL	Unikāls prognozes identifikators
2	User_ID	INT	FK → users.User_ID	NOT NULL	Atsauce uz lietotāju, kurš iesniedza prognozi
3	Match_ID	INT	–	NOT NULL	Spēles ID no football-data.org API
4	HomeGoals	INT	–	NOT NULL	Prognozētie mājas komandas vārti (0–20)
5	AwayGoals	INT	–	NOT NULL	Prognozētie viesas komandas vārti (0–20)
6	Outcome	VARCHAR(10)	–	NULL	Prognozes iznākums: "exact" (precīzs rezultāts, 3 p.), "correct" (pareizs iznākums, 1 p.), "wrong" vai NULL (gaida)
7	CreatedAt	TIMESTAMP	–	NOT NULL	Prognozes iesniegšanas laiks

Tabula comments glabā autorizētu lietotāju teksta komentārus, kas pievienoti pie konkrētām spēlēm. Komentāra saturs tiek sanitizēts ar sanitize-html, lai novērstu XSS uzbrukumus.

5.4. tabula

Tabulas comments lauku apraksts

Nr.	Lauka nosaukums	Datu tips	Atslēga	Ierobežojumi	Apraksts
1	Comment_ID	INT	PK, AI	NOT NULL	Unikāls komentāra identifikators
2	User_ID	INT	FK → users.User_ID	NOT NULL	Atsauce uz komentāra autoru
3	Username	VARCHAR(50)	–	NOT NULL	Autora lietotājvārds komentāra pievienošanas brīdī
4	Match_ID	INT	–	NOT NULL	Spēles ID, pie kuras pievienots komentārs
5	Content	VARCHAR(500)	–	NOT NULL	Sanitizēts komentāra teksts (1–500 rakstzīmes)
6	CreatedAt	TIMESTAMP	–	NOT NULL	Komentāra pievienošanas laiks

Tabula comment_reports reģistrē lietotāju ziņojumus par nevēlamiem komentāriem. Katrs ieraksts saista ziņoto komentāru ar ziņotāju; administrators var apskatīt šo sarakstu un dzēst attiecīgos komentārus.

5.5. tabula

Tabulas comment_reports lauku apraksts

Nr.	Lauka nosaukums	Datu tips	Atslēga	Ierobežojumi	Apraksts
1	Report_ID	INT	PK, AI	NOT NULL	Unikāls ziņojuma identifikators
2	Comment_ID	INT	FK → comments.Comment_ID	NOT NULL	Atsauce uz ziņoto komentāru
3	Reporter_ID	INT	FK → users.User_ID	NOT NULL	Atsauce uz lietotāju, kurš ziņoja
4	CreatedAt	TIMESTAMP	–	NOT NULL	Ziņojuma iesniegšanas laiks

Tabula watchlist glabā papildu informāciju par spēlētājiem, kurus lietotājs pievienojis vērojumu sarakstam. Tā kā football-data.org API neatgriež spēlētāja datus pēc ID vien, tabula kešo svarīgākos laukus, lai izvairītos no atkārtotiem API pieprasījumiem.

5.6. tabula

Tabulas watchlist lauku apraksts

Nr.	Lauka nosaukums	Datu tips	Atslēga	Ierobežojumi	Apraksts
1	Watchlist_ID	INT	PK, AI	NOT NULL	Unikāls watchlist ieraksta identifikators
2	User_ID	INT	FK → users.User_ID	NOT NULL	Atsauce uz lietotāju
3	Player_ID	INT	–	NOT NULL	Spēlētāja ID no football-data.org
4	Player_Name	VARCHAR(100)	–	NULL	Spēlētāja pilnais vārds
5	Team_ID	INT	–	NULL	Komandas ID no football-data.org
6	Team_Name	VARCHAR(100)	–	NULL	Komandas nosaukums
7	Position	VARCHAR(20)	–	NULL	Vispārīgā pozīcija (Goalkeeper, Defender utt.)
8	DetailedPosition	VARCHAR(50)	–	NULL	Detalizētā pozīcija (Centre-Back, Left Winger utt.)
9	Nationality	VARCHAR(50)	–	NULL	Spēlētāja tautība
10	LeagueCode	VARCHAR(10)	–	NULL	Līgas kods (PL, PD, BL1 utt.)
11	CreatedAt	TIMESTAMP	–	NOT NULL	Ieraksta pievienošanas laiks

6. LIETOTĀJA CEĻVEDIS

6.1. Sistēmas prasības aparatūrai un programmatūrai

Footbalyze ir tīmekļa lietojumprogramma, kas darbojas pārlūkprogrammā bez papildu instalācijas. Lietotājam nepieciešama moderna pārlūkprogramma: Google Chrome versija 110 vai jaunāka, Mozilla Firefox versija 110 vai jaunāka, vai Microsoft Edge versija 110 vai jaunāka. Tā kā futbola statistika tiek ielādēta no ārēja API reāllaikā, nepieciešams aktīvs interneta savienojums. Ieteicamā ekrāna izšķirtspēja ir vismaz 1280×720 pikseļi; šaurākās ierīcēs saskarne var nebūt pilnībā optimizēta. Papildu programmatūra, spraudņi vai konti nav nepieciešami.

6.2. Sistēmas instalācija un palaišana

Sistēma ir izvietota publiskā serverī un pieejama bez instalēšanas. Lai lietotu Footbalyze, pārlūkprogrammā jāatver adrese:

<https://footbalyze.rvtdev.tech>

Apmeklētājs var aplūkot futbola statistiku, komandu un spēlētāju datus bez reģistrācijas. Lai piekļūtu personalizētajām funkcijām (izlasei, prognozēm un komentāriem), jāizveido konts vai jāpiesakās ar esošajiem akreditācijas datiem. Testēšanai pieejami šādi demo konti:

- Viesis: reģistrācija nav nepieciešama, pietiek atvērt vietnes adresi
- Reģistrēts lietotājs: e-pasts test@test.com, parole Test123123
- Administrators: e-pasts admin@admin.com, parole Admin123

Administrators konts nodrošina piekļuvi administratora panelim, kurā var pārvaldīt lietotājus un veikt bloķēšanu.

Izstrādātājiem, kuri vēlas palaist sistēmu lokāli, nepieciešams instalēt Node.js (v18 vai jaunāku versiju) un MySQL 8. Tālāk jāveic šādi soļi.

1. Klonēt repozitoriju: `git clone <repozitorija adrese>`
2. Aizpildīt backend/.env failu: DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, JWT_SECRET, FOOTBALL_DATA_API_KEY (football-data.org atslēga), PORT (noklusējums: 3000)
3. Izveidot tukšu MySQL datubāzi un norādīt tās nosaukumu DB_NAME. Tabulas tiek izveidotas automātiski, startējot serveri (runMigrations()).
4. Palaist backend: `cd backend → npm install → node server.js`
5. Palaist frontend jaunā terminālī: `cd frontend → npm install → npm run dev`

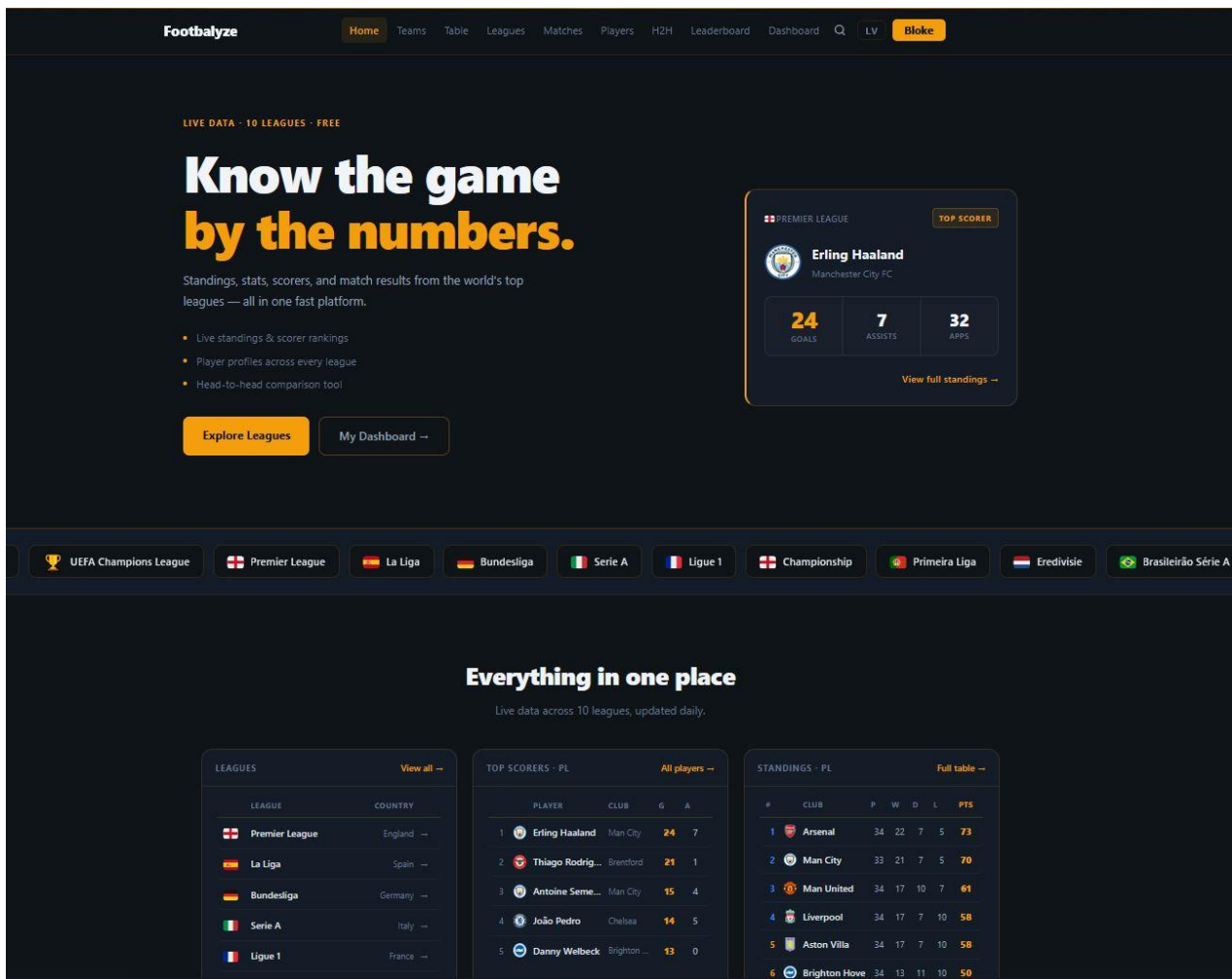
- 6. Atvērt pārlūkprogrammā: <http://localhost:5173>

6.3. Programmas apraksts

Šajā sadaļā aprakstīti visi sistēmas lietotāja saskarnes elementi un to darbības iespējas katrā lapā. Navigācijas josla ir redzama visās lapās. Tā satur Footbalyze logotipu (noklikšķinot atgriežas sākumlapā) un saites uz sadaļām: Teams, Table, Leagues, Matches, Players, H2H, Leaderboard un Dashboard. Labajā pusē atrodas meklēšanas ikona, valodas pārslēgšanas poga un lietotāja poga. Nepieslēdzies lietotājs redz pogas "Pieslēgties" un "Reģistrēties". Pieslēdzies lietotājs redz savu lietotājvārdu. Noklikšķinot uz tā, tiek atvērta profila izvēlne ar opcijām "Mans profils" un "Iziet". Poga "Iziet" dzēš JWT tokenu no localStorage un novirza uz sākumlapu.

Sākumlapa (skat. 6.1.att.) sastāv no titulsadaļas un informācijas bloka. Navigācijas joslā augšā redzamas saites uz visām sadaļām, meklēšanas ikona un valodas pārslēgšanas poga. Titulsadaļas kreisajā pusē redzams virsraksts "Know the game by the numbers", īss sistēmas apraksts un divas darbības pogas: "Explore Leagues" un "My Dashboard →". Labajā pusē redzams Premjerlīgas labākā vārtu guvēja informācijas bloks. Zem titulsadaļas atrodas horizontāli ritināms

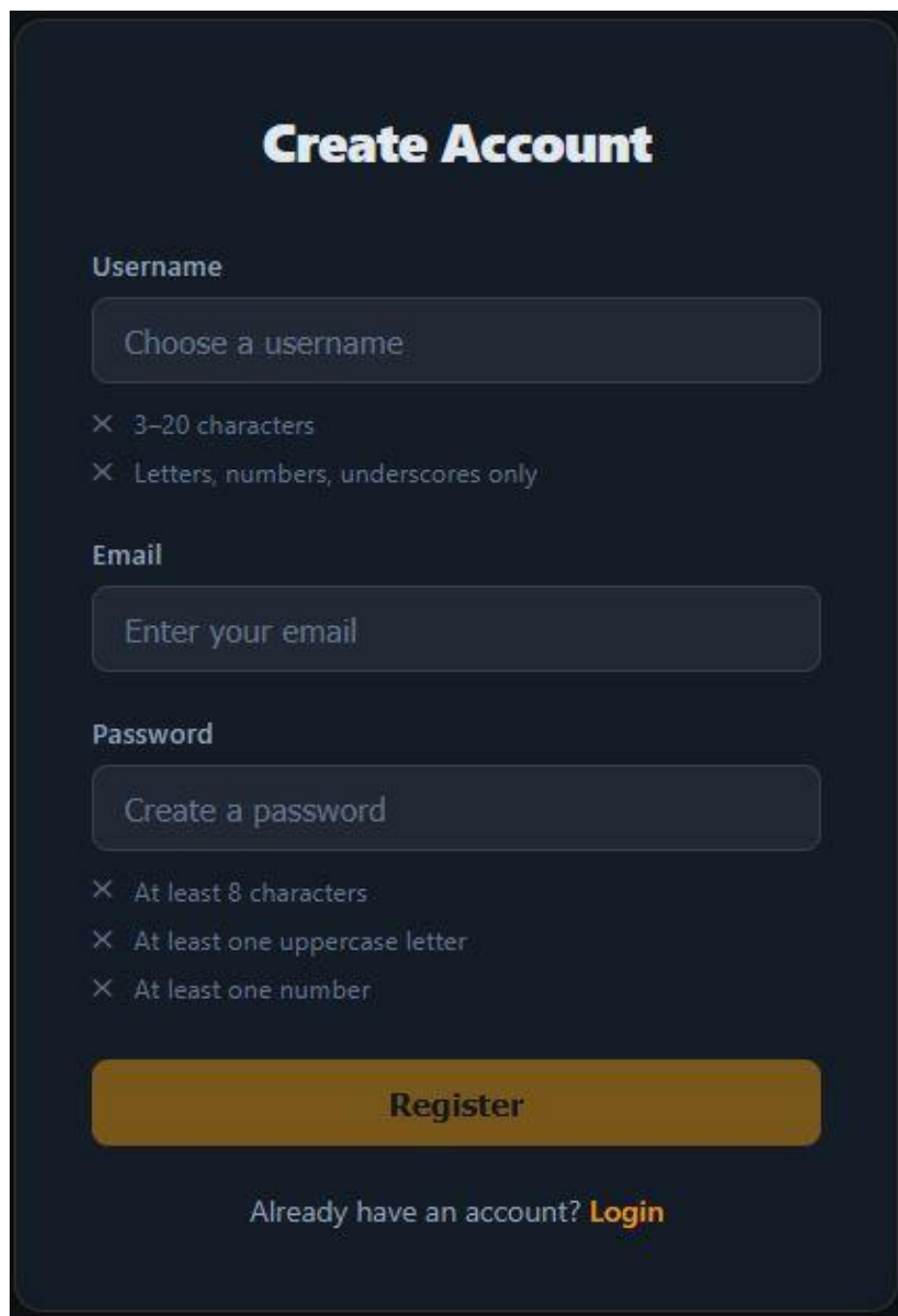
līgu joslas elements ar karogu ikonām un līgu nosaukumiem. Lapas apakšā atrodas aicinājuma bloks ar pogām "Get Started, It's Free" un "Browse as Guest →".



6.1.att. Sākumlapa

Reģistrācijas lapa (skat. 6.2.att.) satur veidlapu ar trim ievadlaukiem: "Username", "Email" un "Password". Zem katra lauka redzami validācijas nosacījumi, kas atjauninās reāllaikā. Poga

"Register" iesniedz veidlapu; ja visi lauki ir derīgi, konts tiek izveidots un lietotājs tiek novirzīts uz pieslēgšanās lapu.



The image shows a 'Create Account' registration form on a dark blue background. The form is titled 'Create Account' in white bold text. It contains three input fields: 'Username' with the placeholder 'Choose a username', 'Email' with the placeholder 'Enter your email', and 'Password' with the placeholder 'Create a password'. Below the 'Username' field are two error messages: 'X 3-20 characters' and 'X Letters, numbers, underscores only'. Below the 'Password' field are three error messages: 'X At least 8 characters', 'X At least one uppercase letter', and 'X At least one number'. At the bottom of the form is a large orange 'Register' button. Below the button is the text 'Already have an account? Login', where 'Login' is a link in orange.

Create Account

Username

X 3-20 characters
X Letters, numbers, underscores only

Email

Password

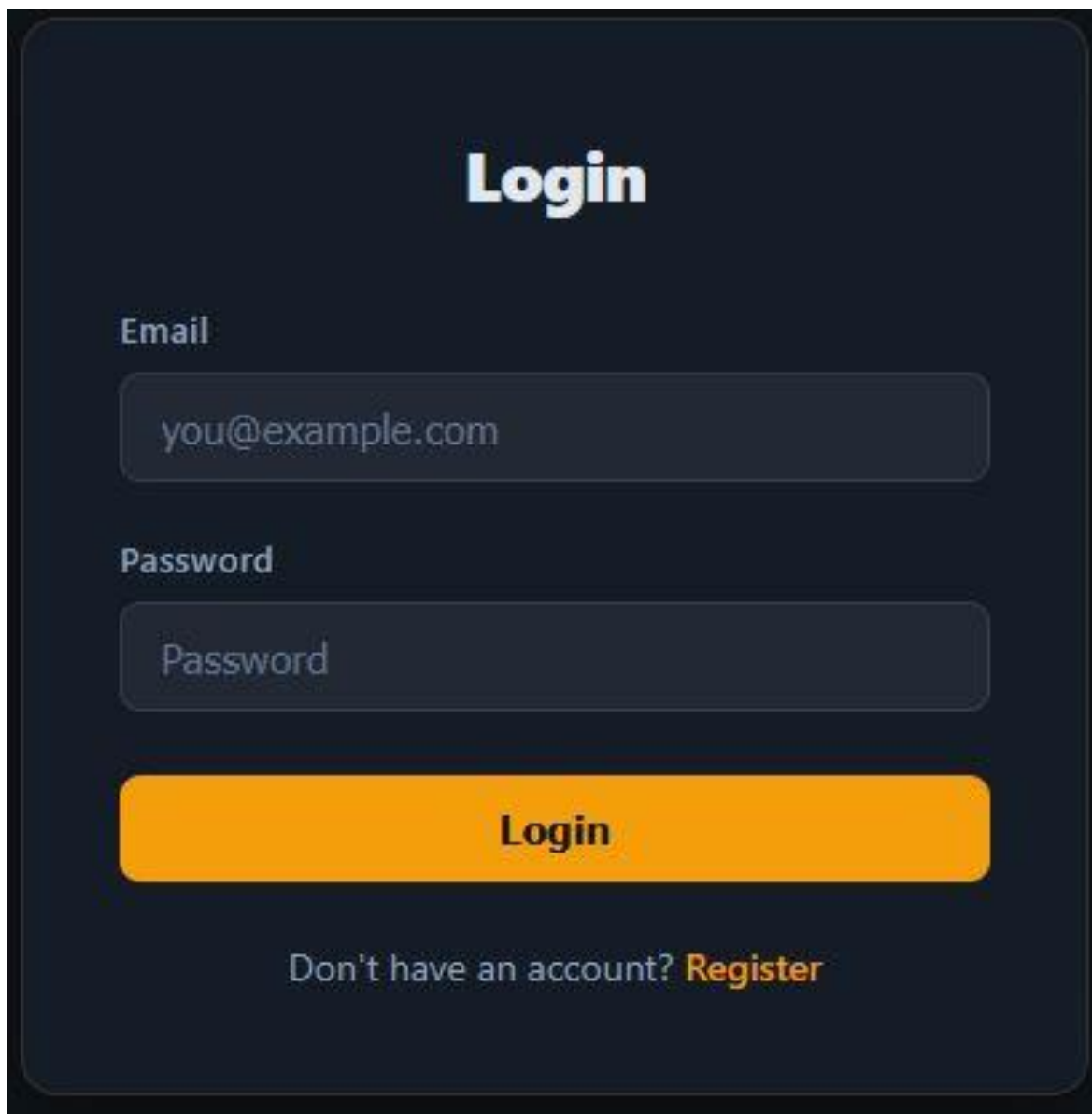
X At least 8 characters
X At least one uppercase letter
X At least one number

Register

Already have an account? [Login](#)

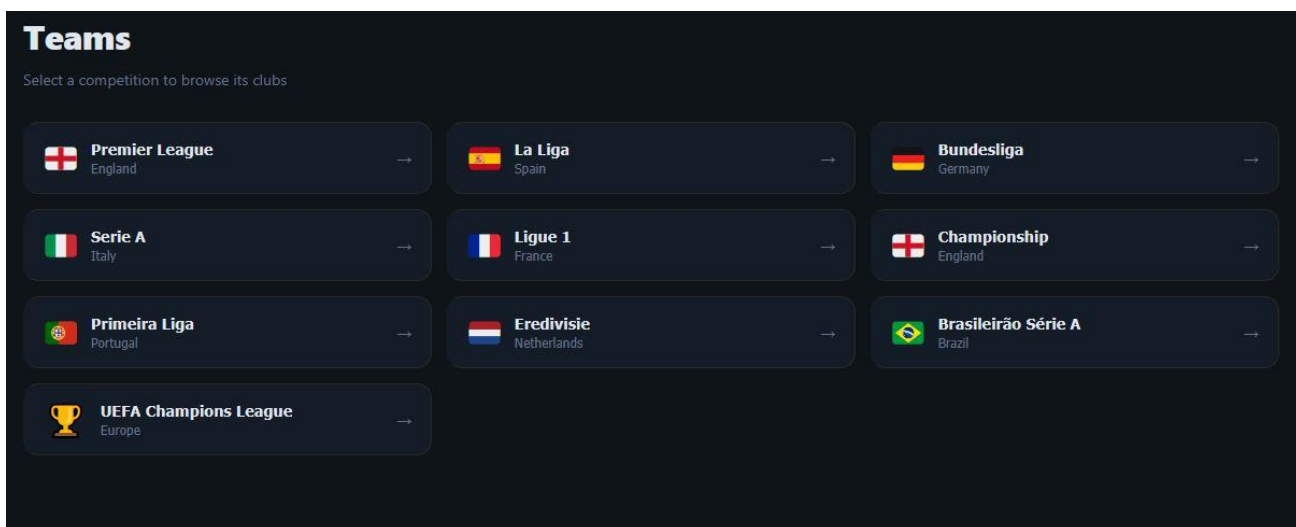
6.2.att. Reģistrācijas lapa

Pieslēgšanās lapa (skat. 6.3.att.) satur divus ievadlaukus: "Email" un "Password". Veiksmīgas autentifikācijas gadījumā serveris atgriež JWT tokenu, kas tiek saglabāts localStorage. Nepareizu datu vai bloķēta konta gadījumā tiek parādīts attiecīgs kļūdas paziņojums.

The image shows a login form with a dark blue background. At the top, the word "Login" is written in a large, bold, white font. Below it, there are two input fields. The first is labeled "Email" in a smaller white font, and it contains the text "you@example.com". The second is labeled "Password" in a smaller white font, and it contains the text "Password". Below these fields is a large, rounded orange button with the word "Login" in a bold black font. At the bottom of the form, there is a line of text that says "Don't have an account? Register", where "Register" is in a bold orange font.

6.3.att. Pieslēgšanās lapa

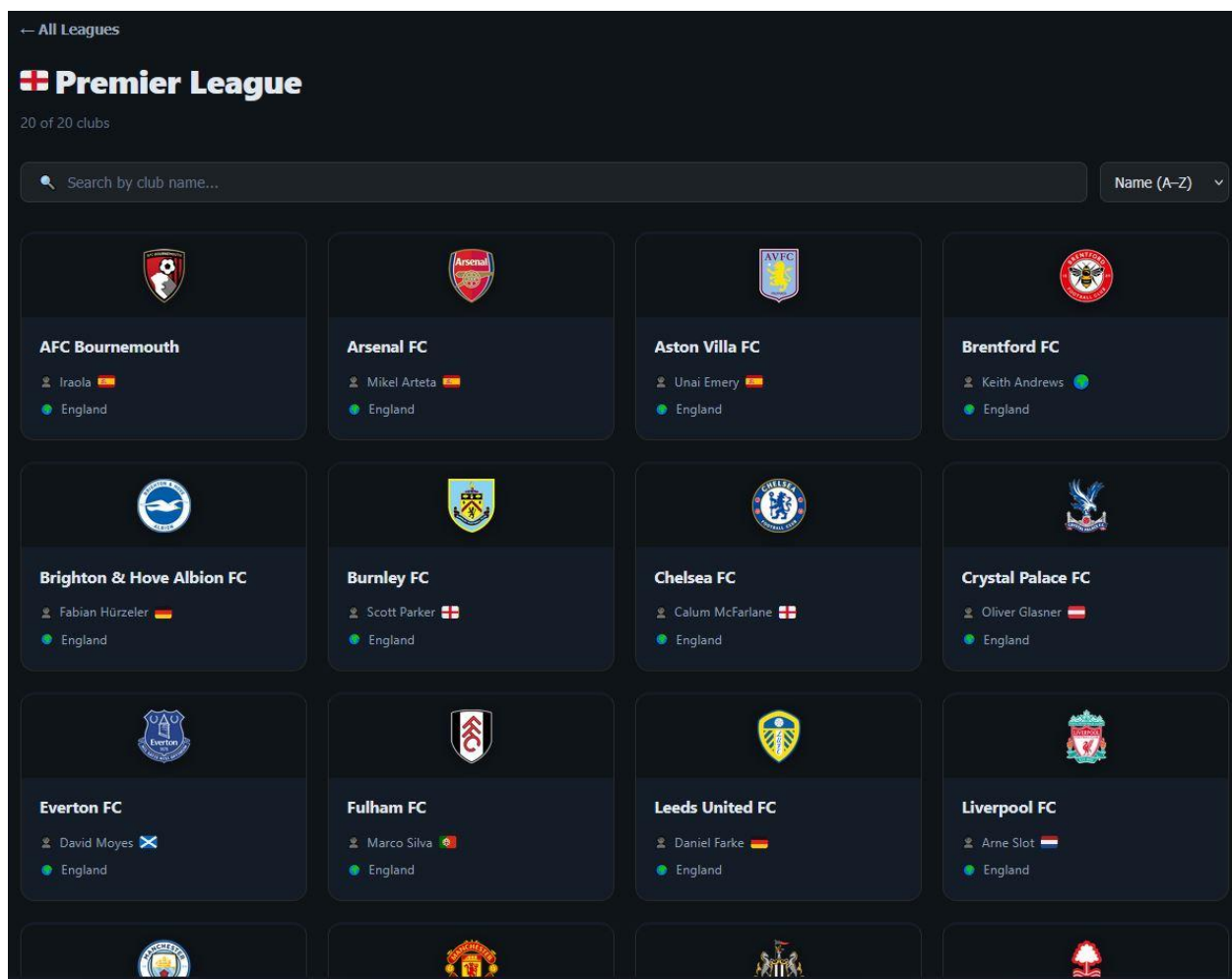
Sadaļā Teams (skat. 6.4.att.) lietotājs vispirms redz visu pieejamo līgu sarakstu ar katras līgas nosaukumu un karogu. Noklikšķinot uz līgas, tiek atvērta attiecīgās līgas komandu lapa.



6.4.att. Līgas izvēles lapa sadaļā Teams

Komandu saraksta lapā (skat. 6.5.att.) redzams izvēlētās līgas nosaukums ar karogu un kopējo komandu skaitu. Atrodas meklēšanas lauks "Search by club name...", kas reāllaikā filtrē komandu sarakstu, un kārtošanas izvēlne "Name (A–Z)". Komandas tiek rādītas četru kolonnu

režģī. Katrs elements satur komandas emblēmu, nosaukumu, trenera vārdu (ja pieejams) ar tautības karogu un valsti.



6.5.att. Komandu saraksta lapa

Komandas detaļu lapā (skat. 6.6.att.) redzama komandas emblēma, nosaukums, treneris (ja pieejams), līga un formas josla ar pēdējo piecu spēļu rezultātiem. Cilne "Squad" rāda spēlētāju

sarakstu ar pozīciju nozīmītēm, tautību un vecumu. Cilne "Stats" rāda komandas statistiku. Zemāk atrodas pēdējo spēļu saraksts ar rezultātiem.

[← All Teams](#)



AFC Bournemouth

 Iraola England29 players

FORM D D W W D

SquadStats

RECENT MATCHES

22 Apr	AFC Bournemouth	2-2	Leeds United FC
18 Apr	Newcastle United FC	1-2	AFC Bournemouth
11 Apr	Arsenal FC	1-2	AFC Bournemouth
20 Mar	AFC Bournemouth	2-2	Manchester United FC
14 Mar	Burnley FC	0-0	AFC Bournemouth

All 29

Goalkeepers 3

Defenders 9

Midfielders 7

Forwards 10

GOALKEEPERS

F	Fraser Forster	GK	 38
C	Christos Mandas	GK	 24
D	Đorđe Petrović	GK	 26

DEFENDERS

6.6.att. Komandas detaļu lapa

Tabulas lapā (skat. 6.7.att.) redzama izvēlētās līgas pašreizējā vietu tabula ar kolonnām: vieta, komanda, aizvadītās spēles (P), uzvaras (W), neizšķirti (D), zaudējumi (L), gūtie vārti (GF), ielaistie vārti (GA), vārtu starpība (GD), punkti (PTS) un formas josla (FORM).

← All Leagues

Premier League

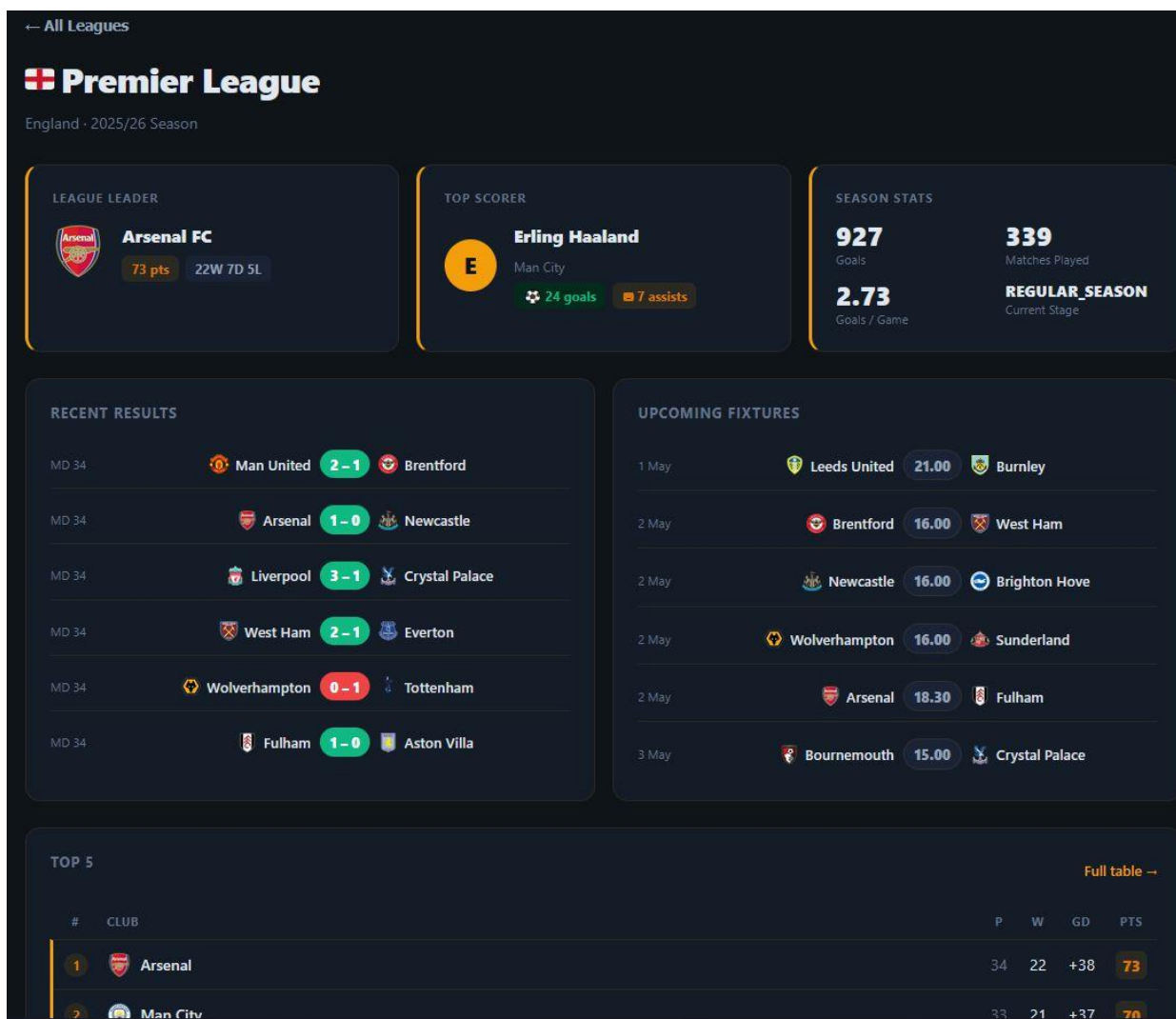
2025/26 Season Standings

#	CLUB	P	W	D	L	GF	GA	GD	PTS	FORM
1	Arsenal FC	34	22	7	5	64	26	+38	73	●●●●●
2	Manchester City FC	33	21	7	5	66	29	+37	70	●●●●●
3	Manchester United FC	34	17	10	7	60	46	+14	61	●●●●●
4	Liverpool FC	34	17	7	10	57	44	+13	58	●●●●●
5	Aston Villa FC	34	17	7	10	47	42	+5	58	●●●●●
6	Brighton & Hove Albion FC	34	13	11	10	48	39	+9	50	●●●●●
7	AFC Bournemouth	34	11	16	7	52	52	0	49	●●●●●
8	Chelsea FC	34	13	9	12	53	45	+8	48	●●●●●
9	Brentford FC	34	13	9	12	49	46	+3	48	●●●●●
10	Fulham FC	34	14	6	14	44	46	-2	48	●●●●●
11	Everton FC	34	13	8	13	41	41	0	47	●●●●●
12	Sunderland AFC	34	12	10	12	36	45	-9	46	●●●●●
13	Crystal Palace FC	33	11	10	12	36	39	-3	43	●●●●●
14	Newcastle United FC	34	12	6	16	46	50	-4	42	●●●●●
15	Leeds United FC	34	9	13	12	44	51	-7	40	●●●●●

6.7.att. Tabulas lapā

Līgas pārskata lapā (skat. 6.8.att.) redzami trīs informācijas bloki: līgas līderis ar punktiem un W/D/L bilanci, labākais vārtu guvējs ar gūto vārtu un piespēļu skaitu, un sezonas kopsavilkums

ar kopējo vārtu skaitu un aizvadīto spēļu skaitu. Zemāk atrodas pēdējo rezultātu un gaidāmo spēļu saraksti, kā arī top 5 tabulas izvilkums.



6.8.att. Līgas pārskata lapa

Spēļu lapā (skat. 6.9.att.) redzams spēļu saraksts ar mājas un viesas komandu nosaukumiem, emblēmām un rezultātiem. Lapas augšdaļā atrodas statistikas josla ar kopējo vārtu skaitu, vidējo

vārtu skaitu spēlē, mājas uzvarām, neizšķirtiem un viesas uzvarām. Iespējams filtrēt spēles pēc rezultāta veida un meklēt pēc komandas nosaukuma.

The screenshot displays the Premier League website interface. At the top, there's a navigation bar with '← All Leagues' and the 'Premier League' logo. Below the logo, it says '339 of 339 results'. The main section features a 'Results' tab (highlighted) and a 'Fixtures' tab. A summary box shows key statistics: 927 Total Goals, 2.7 Avg / Game, 143 Home Wins, 90 Draws, and 106 Away Wins. Below this, a 'BIGGEST WIN' section highlights Arsenal FC's 5-0 victory over Leeds United FC. A search bar with the placeholder 'Search by team...' and a 'Newest First' dropdown menu are present. Filter buttons for 'All' (339), 'Home Win' (143), 'Draw' (90), and 'Away Win' (106) are shown. A list of team buttons includes AFC Bournemouth, Arsenal FC, Aston Villa FC, Brentford FC, Brighton & Hove Albion FC, Burnley FC, and Chelsea FC. The 'MATCHDAY 34' section, dated '21 Apr – 27 Apr', lists several matches with their dates, teams, scores, and status (Home, Away, or Draw). The matches listed are: Manchester United FC vs Brentford FC (27 Apr, 2-1, Home), Arsenal FC vs Newcastle United FC (25 Apr, 1-0, Home), Liverpool FC vs Crystal Palace FC (25 Apr, 3-1, Home), West Ham United FC vs Everton FC (25 Apr, 2-1, Home), Wolverhampton Wanderers FC vs Tottenham Hotspur FC (25 Apr, 0-1, Away), and Fulham FC vs Aston Villa FC (25 Apr, 1-0, Home).

6.9.att. Spēļu lapa

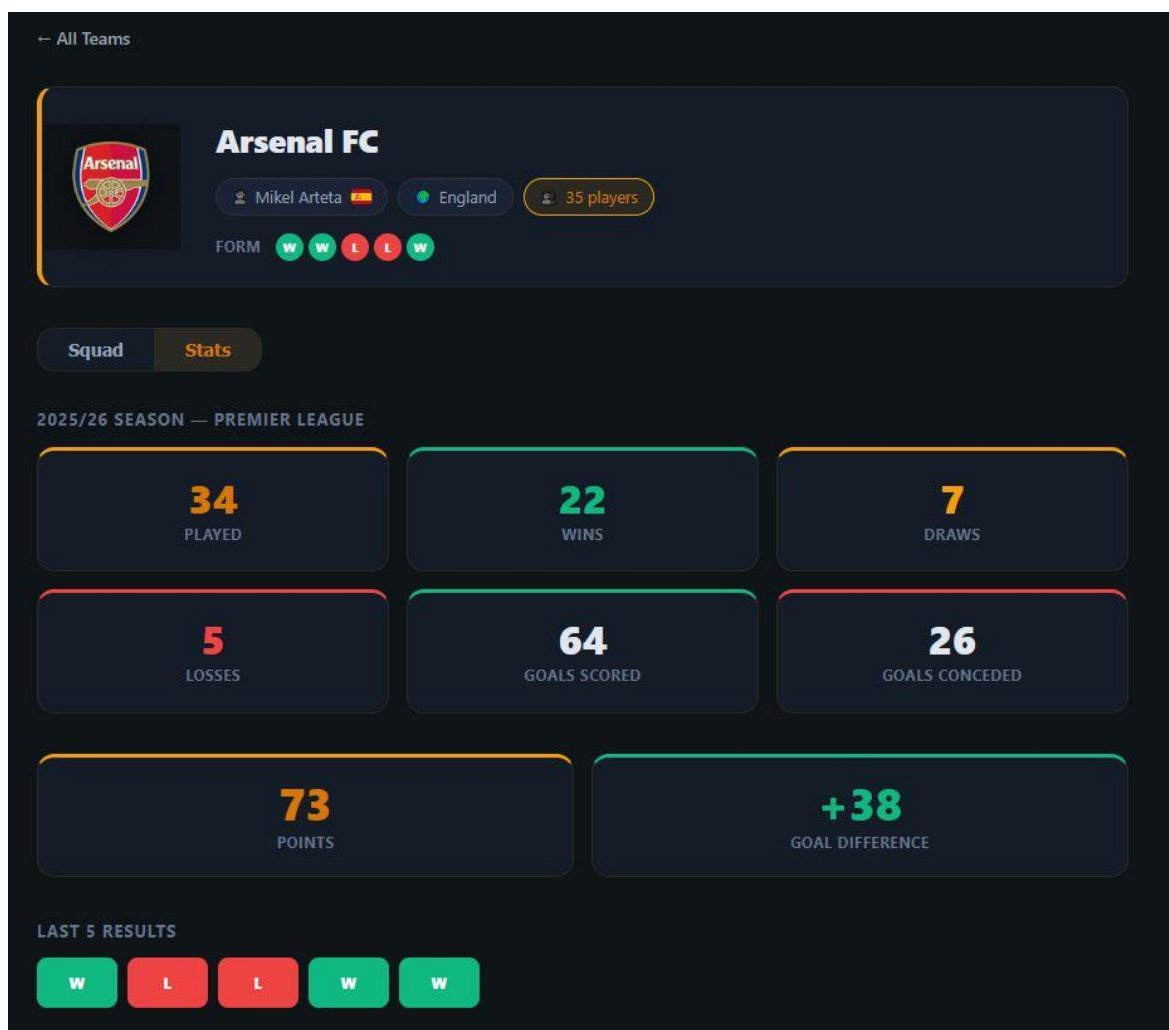
Noklikšķinot uz komentāru ikonai blakus spēlei, tiek atvērts komentāru panelis (skat. 6.10.att.). Redzami visi komentāri ar lietotārvārdu, datumu un tekstu. Katram komentāram blakus

atrodas karodziņa ikona ziņošanai par neatbilstošu saturu. Apakšā atrodas teksta lauks "Write a comment..." un poga "Post" jauna komentāra iesniegšanai.



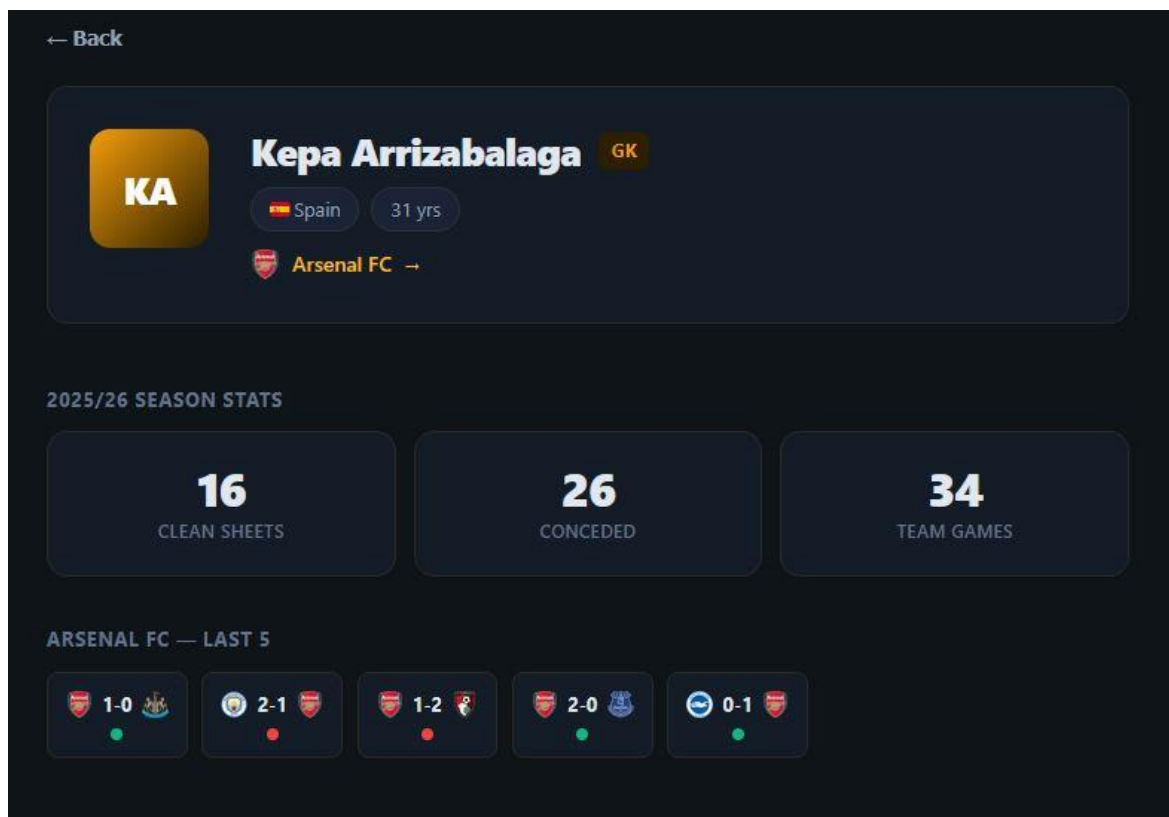
6.10.att. Spēles komentāri

Spēlētāju sadaļā, atverot komandas statistikas skatu (skat. 6.11.att.), redzama komandas spēlētāju pašreizējās sezonas statistika: gūtie vārti, rezultatīvās piespēles, sodi un aizvadītās spēles katram spēlētājam.



6.11.att. Komandas statistikas skats

Spēlētāja detaļu skatā (skat. 6.12.att.) redzams spēlētāja vārds, pozīcijas nozīmīte, tautība un vecums. Vārtusargiem tiek rādīti tīrie vārti, ielaistie vārti un aizvadīto spēļu skaits. Zemāk redzami komandas pēdējo piecu spēļu rezultāti.



6.12.att. Spēlētāja detaļu skats

Lietotāja panelis (skat. 6.13.att.) ir pieejams tikai autorizētiem lietotājiem. Lapas augšdaļā atrodas trīs cilnes: "Overview", "Predictions" un "Watchlist". Pārskata cīlnē kreisajā pusē redzama iecienītās komandas karte ar emblēmu, nosaukumu, līgas pozīciju, sezonu statistiku un formas

joslu. Labajā pusē redzama iecienītā spēlētāja karte ar vārdu, pozīciju, tautību un statistiku. Zemāk atrodas komandas pēdējo rezultātu un gaidāmo spēļu saraksti.

Welcome, Bloke

Your personal football hub.

[Overview](#)[Predictions 3](#)[Watchlist 2](#)

Favourite Team

Change

**Borussia Dortmund**

Bundesliga2nd

67PTS

20W

7D

4L

+34GD

FORM●●●●●●

View Squad →

Favourite Player

Change

**Benjamin Šeško**

✓ Saved

CF

Manchester United FC

Slovenia

Age 22

10GOALS

1ASSISTS

0PENS

31APPS

FORM●●●●●●

View Squad →

Recent Results

Borussia Dortmund

W

26 Apr

Borussia Dortmund

4-0

SC Freiburg

L

18 Apr

TSG 1899 Hoffenheim

2-1

Borussia Dortmund

L

11 Apr

Borussia Dortmund

0-1

Bayer 04 Leverkusen

W

4 Apr

VfB Stuttgart

0-2

Borussia Dortmund

W

21 Mar

Borussia Dortmund

3-2

Hamburger SV

Upcoming Fixtures

Borussia Dortmund

3 May
17:30

Borussia Mönchengladbach

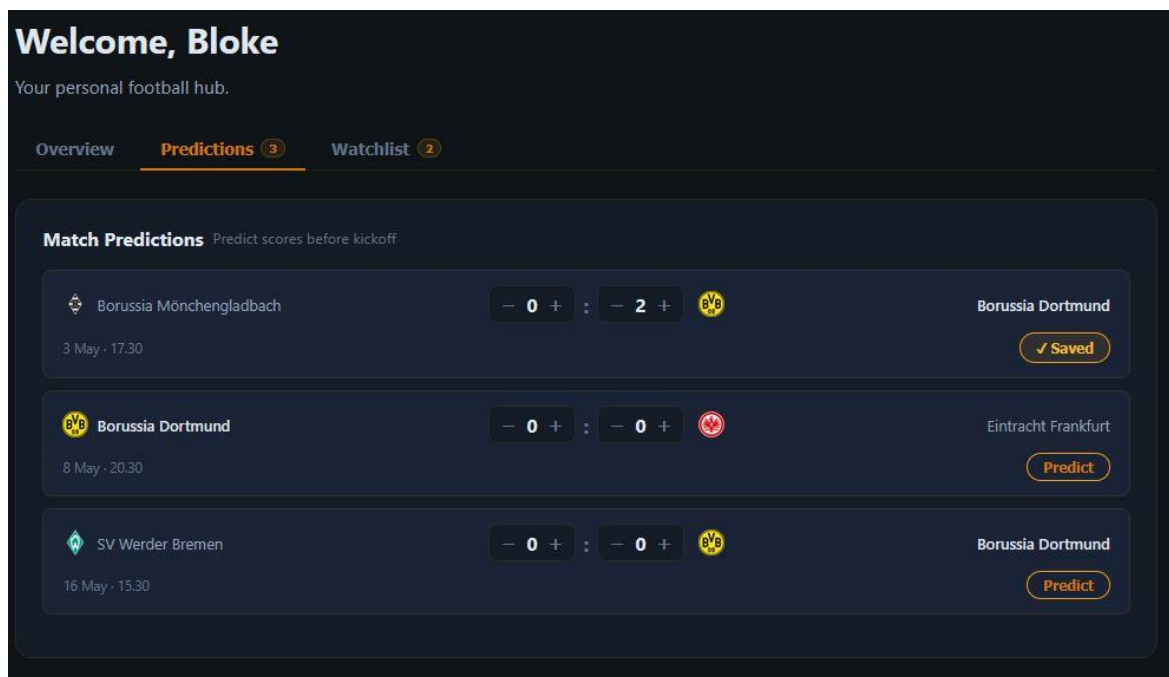
VS

Borussia Dortmund

Upcoming

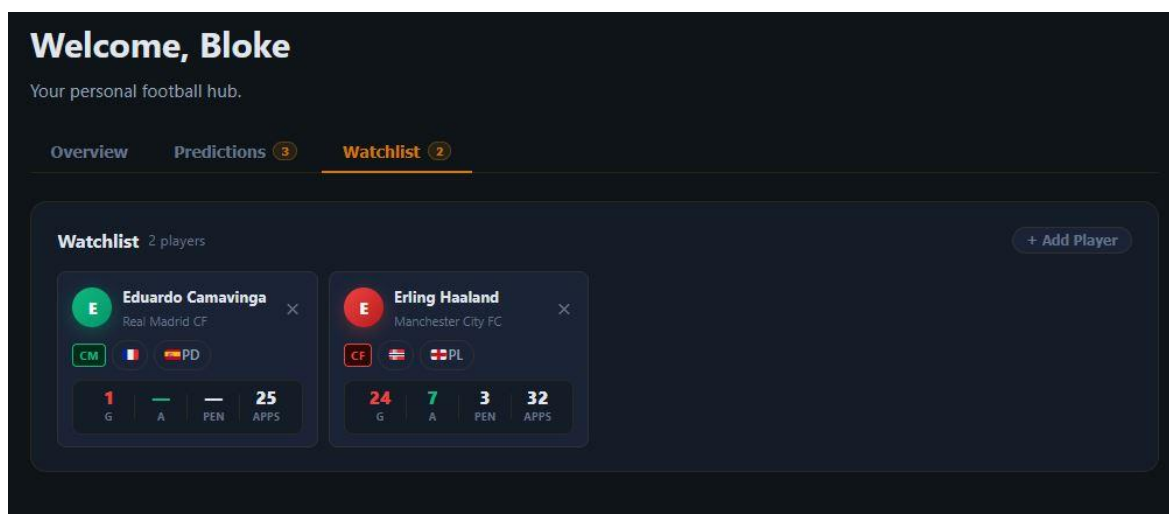
6.13.att. Lietotāja panelis – Pārskats

Prognozes cilnē (skat. 6.14.att.) redzamas visas lietotāja iesniegtās spēļu prognozes ar rezultātu un izpildes statusu: "exact" (precīzs rezultāts, 3 punkti), "correct" (pareizs iznākums, 1 punkts) vai "wrong" (nepareizi).



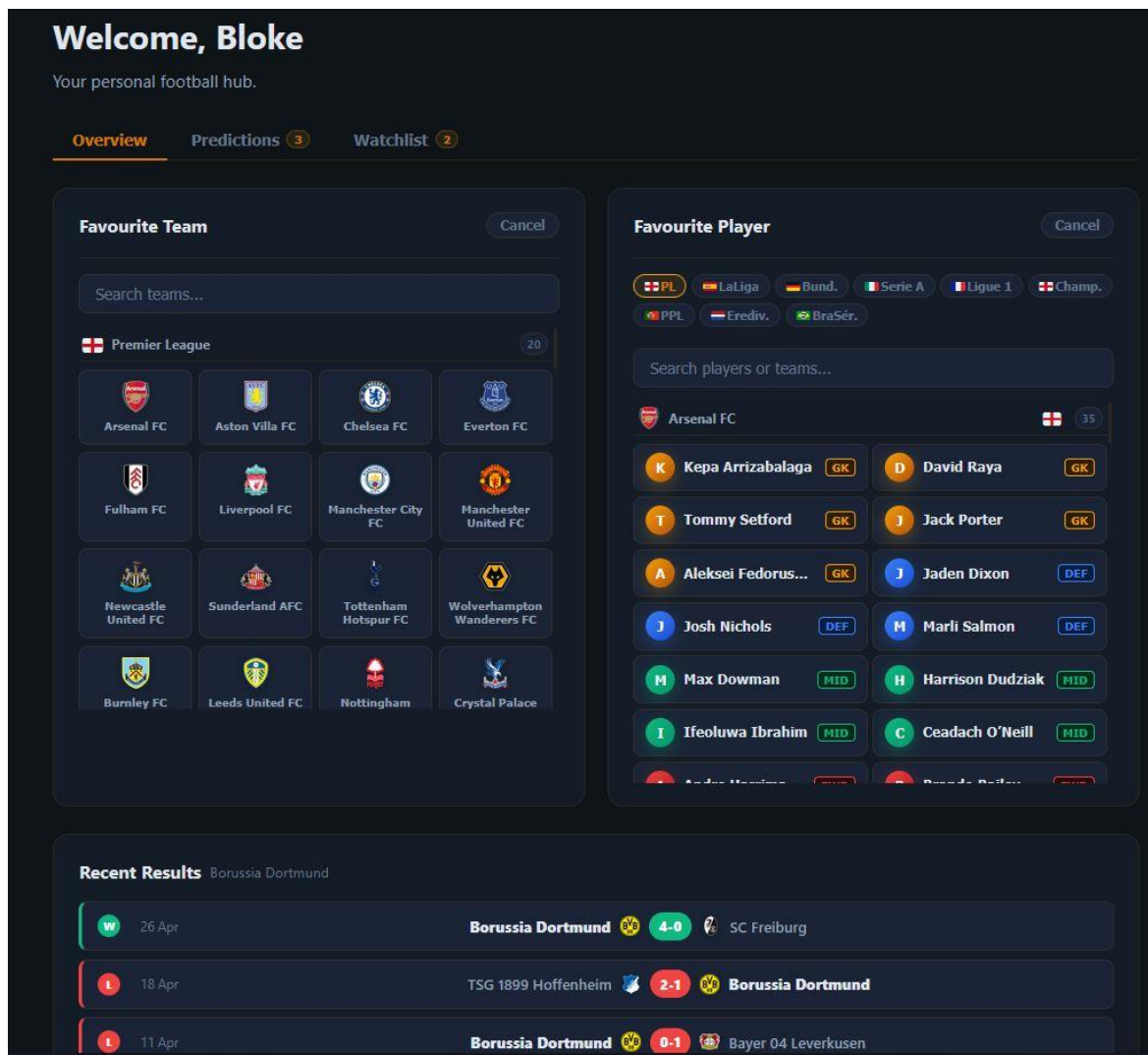
6.14.att. Lietotāja panelis – Prognozes

Vērošanas saraksta cilnē (skat. 6.15.att.) redzami visi spēlētāji, kuriem lietotājs seko. Katram spēlētājam redzama pozīcijas nozīmīte, tautības karogs, komanda un pamatstatistika. Spēlētāju var noņemt no saraksta ar attiecīgu pogu.



6.15.att. Lietotāja panelis – Vērošanas saraksts

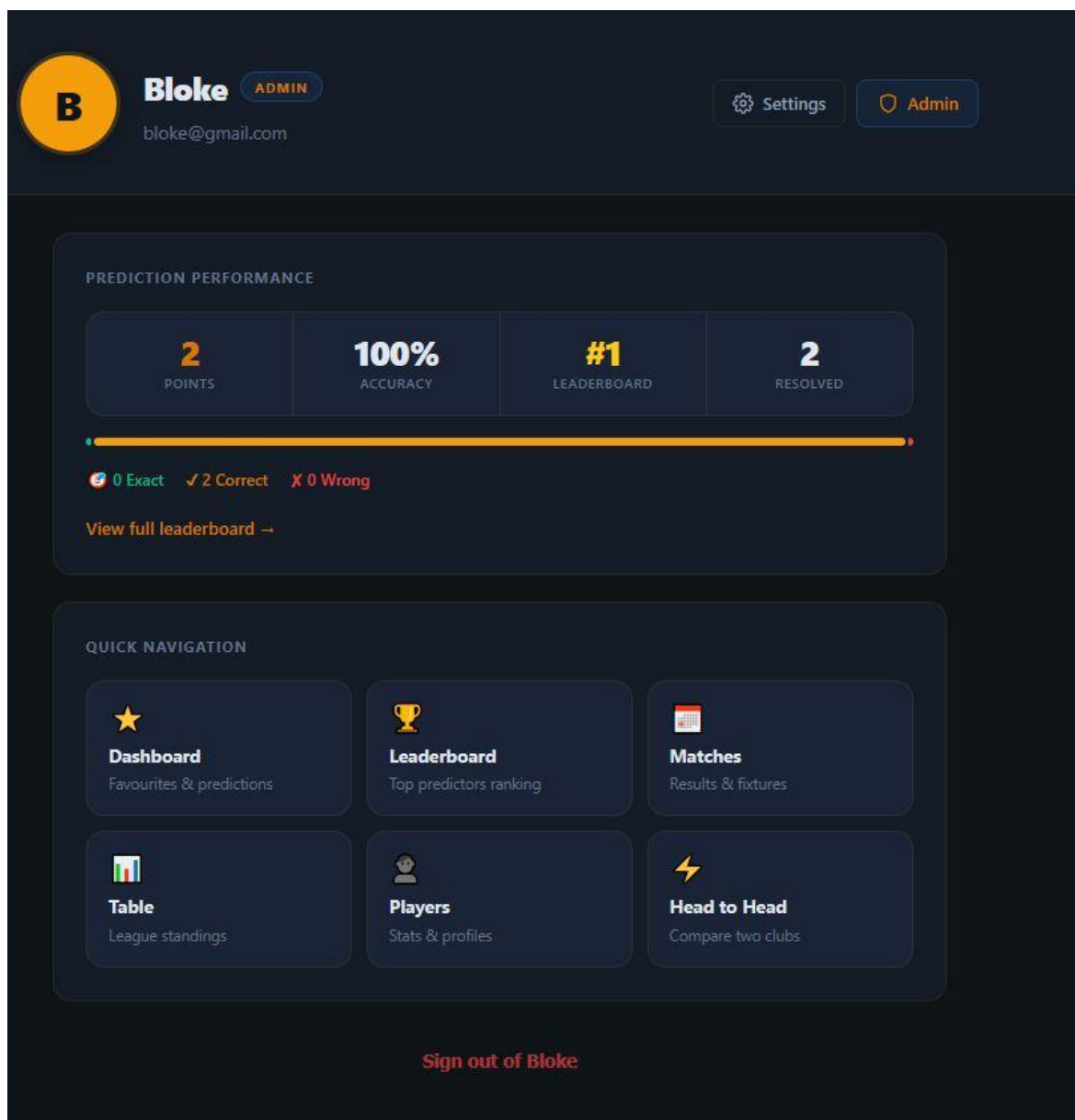
Komandas un spēlētāja izvēles logi (skat. 6.16.att.) tiek atvērti ar pogām "Change" attiecīgajās kartītēs. Komandas logā atrodas meklēšanas lauks un komandu režģis, grupēts pēc līgas. Spēlētāja logā atrodas līgas cilnes un spēlētāju saraksts, grupēts pēc kluba. Abi logi var būt atvērti vienlaikus.



6.16.att. Komandas un spēlētāja izvēles logi

Profila lapa (skat. 6.17.att.) tiek atvērta noklikšķinot uz lietotājvārda navigācijā. Lapā redzams avatārs ar iniciāļiem vai augšupielādētu attēlu, lietotājvārds un e-pasta adrese, prognozes

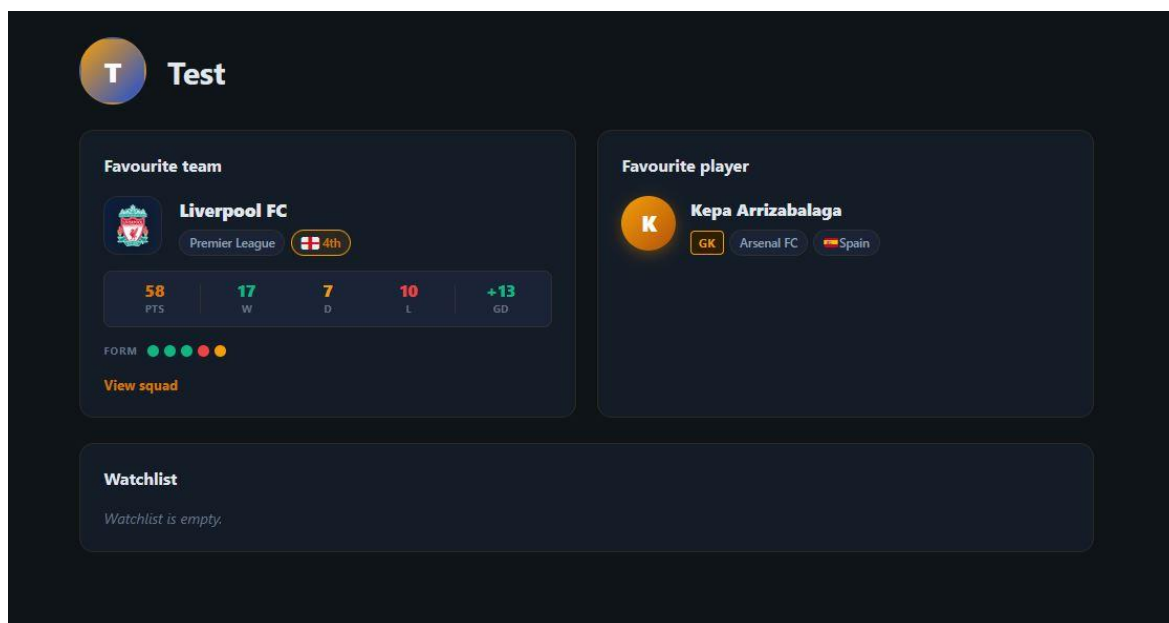
veiktspējas bloks ar punktiem, precizitāti un vietu līdertablā, un ātrās navigācijas pogas uz galvenajām sadaļām. Lapas apakšā atrodas poga "Sign out".



6.17.att. Profila lapa

Publiskais lietotāja profils (skat. 6.18.att.) ir pieejams jebkurai vietnes apmeklētājam pēc adreses /users/:lietotājvārds. Lapā redzams lietotāja avatārs ar iniciāļiem, lietotājvārds, iecienītā

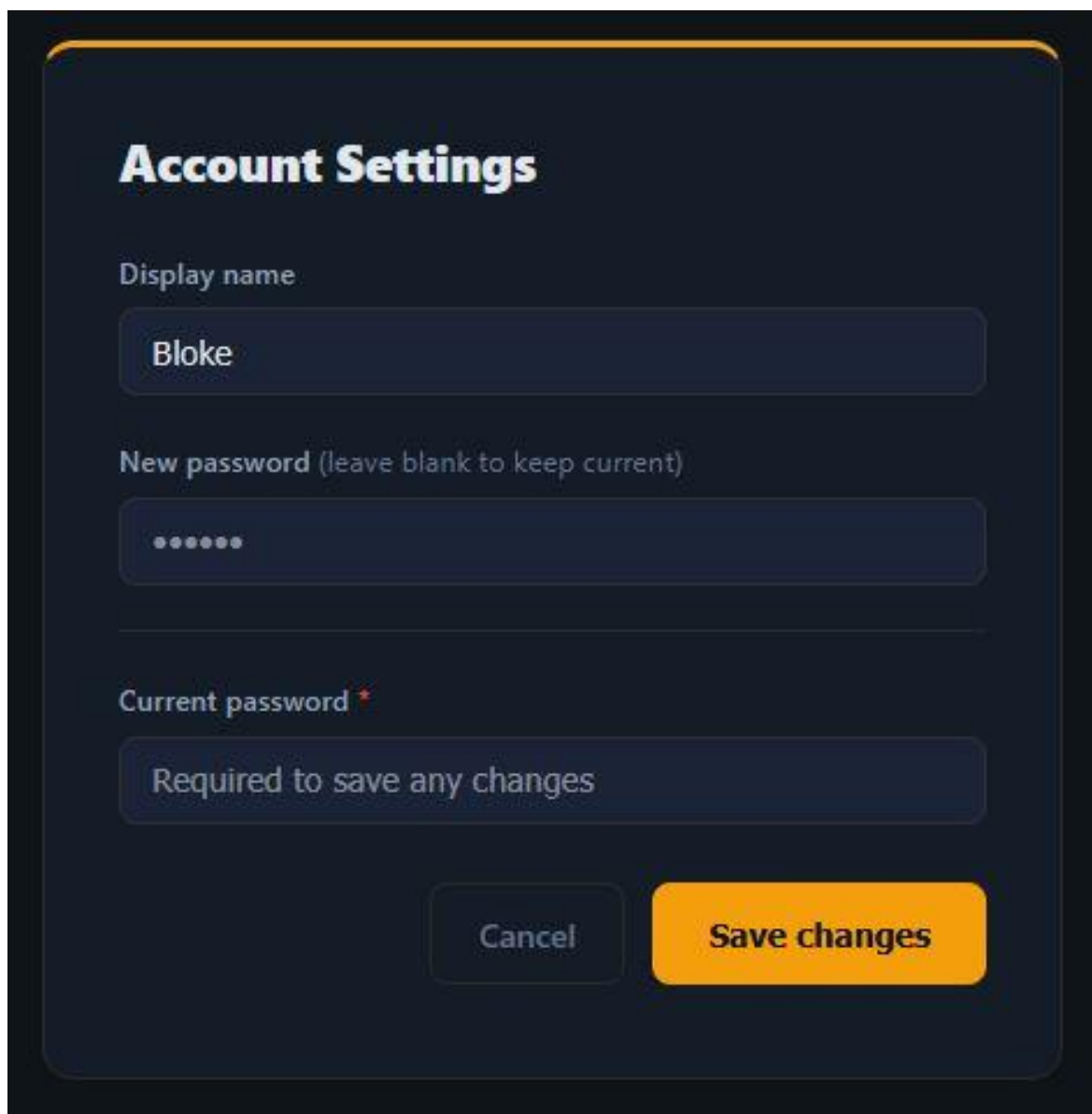
komanda ar līgas pozīciju un statistiku, iecienītais spēlētājs ar pozīciju un tautību, un vērošanas saraksts.



6.18.att. Publiskais lietotāja profils

Konta iestatījumu logs (skat. 6.19.att.) tiek atvērts ar pogu "Settings" profila lapā. Logs satur lauku "Display name" pašreizējā lietotājevārda maiņai, lauku "New password" jaunas paroles

ievadei un lauku "Current password", kas ir obligāts jebkuru izmaiņu saglabāšanai. Poga "Save changes" iesniedz veidlapu.



Account Settings

Display name

Bloke

New password (leave blank to keep current)

••••••

Current password *

Required to save any changes

Cancel Save changes

6.19.att. Konta iestatījumu logs

Administratora panelis (skat. 6.20.att.) ir pieejams tikai lietotājiem ar lomu "admin". Cīlnē "Users" redzama visu reģistrēto lietotāju tabula ar kolonnām: ID, lietotājvārds, e-pasts, loma un darbību pogas. Katrai rindai pieejamas pogas "Promote" (paaugstināt uz admin), "Ban" (bloķēt) un

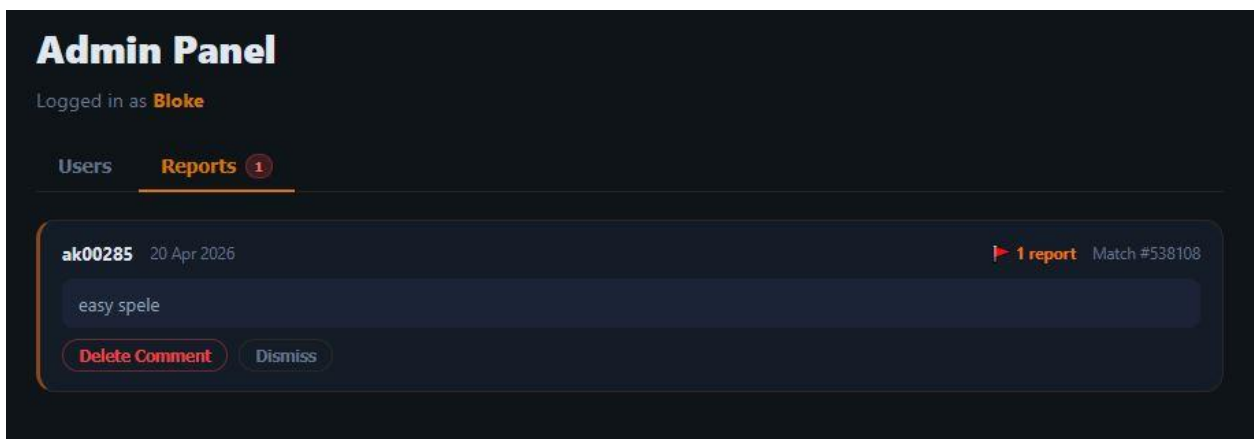
"Delete" (dzēst). Paneļa augšdaļā redzama kopsavilkuma josla ar kopējo lietotāju, adminu, parasto lietotāju un bloķēto lietotāju skaitu.

The screenshot displays an 'Admin Panel' interface. At the top, it shows the user is logged in as 'Bloke'. Below this, there are two tabs: 'Users' (active) and 'Reports'. A summary bar contains four statistics: 8 Total Users, 3 Admins, 5 Regular, and 0 Banned. A search bar is provided below the statistics. The main section is a table with columns: ID, Username, Email, Role, and Actions. The table lists nine users, including the current user 'Bloke' (ADMIN) and several other users with roles of ADMIN or USER. Each user entry has specific action buttons like 'Promote', 'Ban', 'Delete', or 'Demote'.

ID	USERNAME	EMAIL	ROLE	ACTIONS
1	Bloke you	bloke@gmail.com	ADMIN	—
2	aleksis13k	Aleksis.kahanovksis@gmail.com	USER	Promote Ban Delete
3	ak00285	ak00285@rvt.lv	ADMIN	Demote Ban Delete
4	kozilans	rkozilans@gmail.com	USER	Promote Ban Delete
6	krabis1	krabis1@krabis1.lv	USER	Promote Ban Delete
7	Admin	admin@admin.com	ADMIN	Demote Ban Delete
8	Test	Test@test.com	USER	Promote Ban Delete
9	Maris43	al.hackett@example.com	USER	Promote Ban Delete

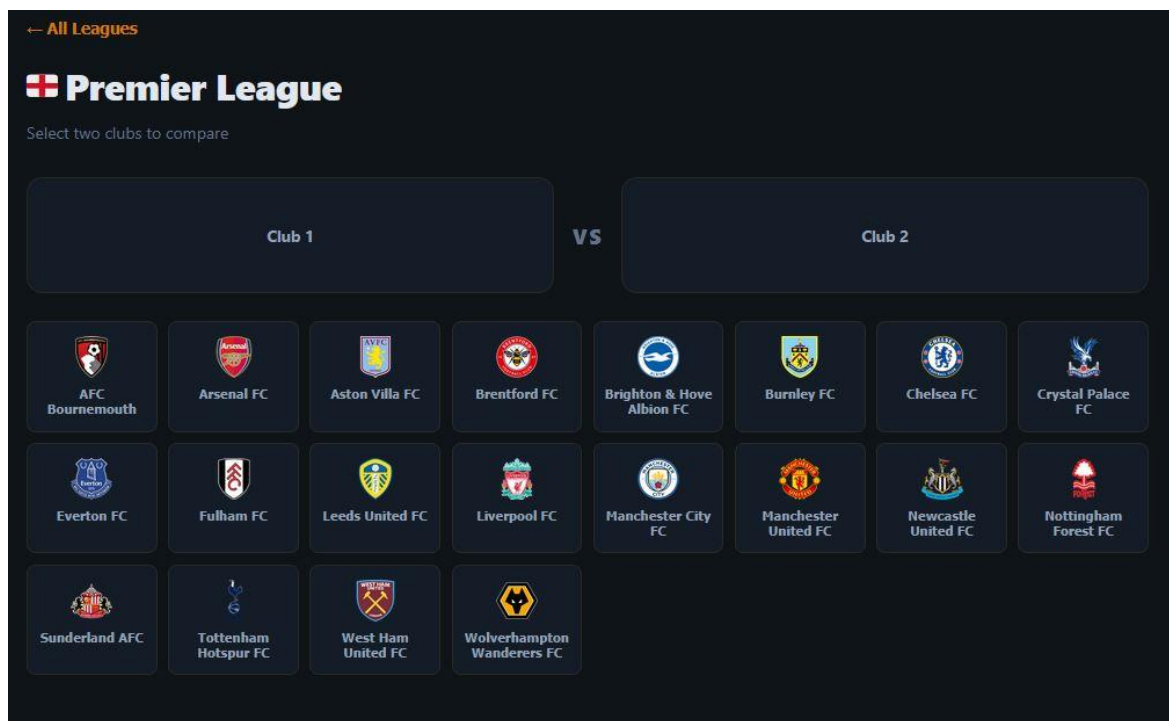
6.20.att. Administratora panelis – lietotāju pārvaldība

Ziņojumu cilnē (skat. 6.21.att.) redzami visi ziņotie komentāri ar autora lietotārvārdu, datumu, komentāra tekstu un ziņojumu skaitu. Katram ziņojumam pieejamas pogas "Delete Comment" (dzēst komentāru) un "Dismiss" (noraidīt ziņojumu bez dzēšanas).



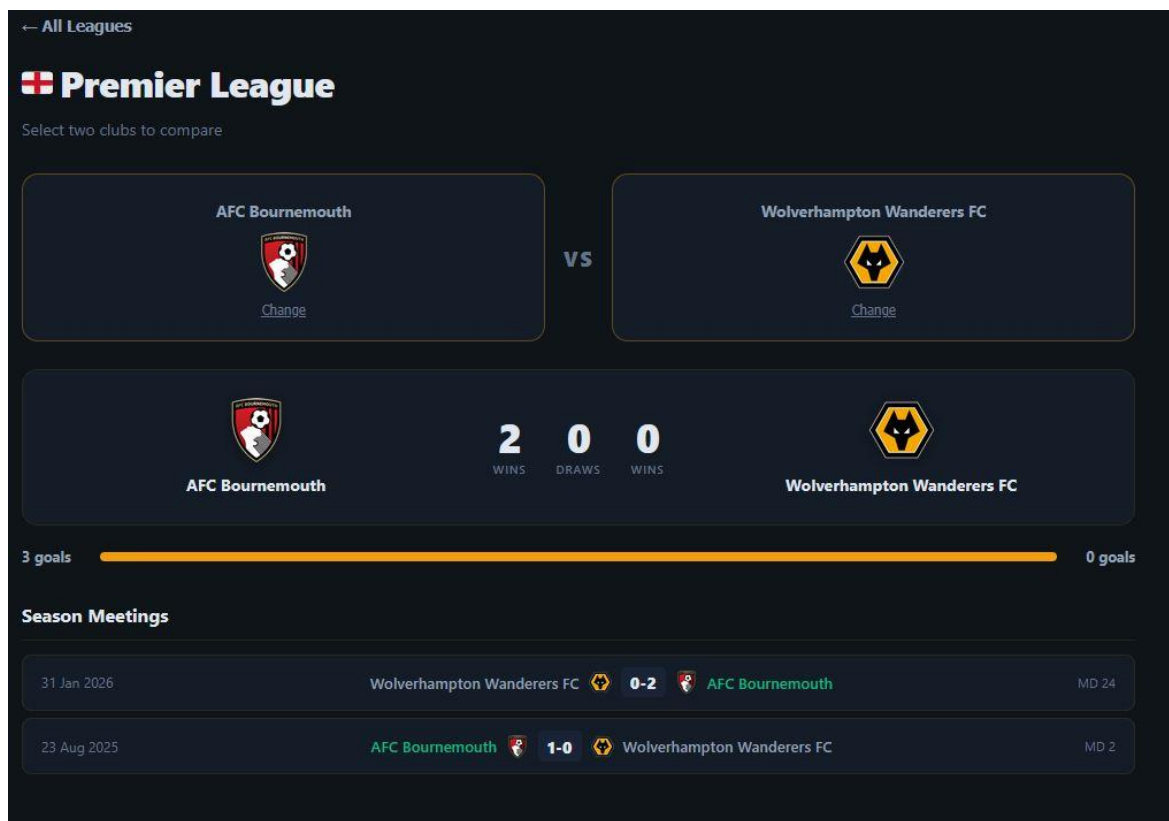
6.21.att. Administratora panelis – ziņojumi

H2H salīdzinājuma lapā (skat. 6.22.att.) lietotājs vispirms izvēlas līgu no saraksta, pēc tam ekrānā parādās divu komandu izvēles panelis ar visām attiecīgās līgas komandām. Noklikšķinot uz divām komandām, tiek ielādēts salīdzinājums.



6.22.att. H2H – komandu izvēles lapa

H2H salīdzinājuma rezultātu skatā (skat. 6.23.att.) blakus redzamas abu izvēlēto komandu statistikas: uzvaras, neizšķirti, zaudējumi, gūtie un ielaistie vārti un citi rādītāji. Tiek rādīta arī abu komandu savstarpējo spēļu vēsture.



6.23.att. H2H – salīdzinājuma rezultāti

Prognozes līdertabulā (skat. 6.24.att.) redzami visi lietotāji, kuri ir iesnieguši vismaz vienu prognozi, sarindoti pēc uzkrātajiem punktiem. Katram lietotājam redzama vieta sarakstā,

lietotājs, aizvadīto prognožu skaits, precīzo (3 punkti), pareizo (1 punkts) un nepareizo prognožu skaits un kopējie punkti.

Prediction Leaderboard						
Ranked by points — 3pts for exact score, 1pt for correct result						
#	PLAYER	PLAYED	🎯 EXACT	✓ CORRECT	✗ WRONG	PTS
1	Bloke you	2	0	2	0	2
2	ak00285	1	0	1	0	1
3	krabis1	1	0	0	1	0

6.24.att. Prognozes līdertabula

7. PROGRAMMATŪRAS LIETOJAMĪBAS TESTĒŠANA

Tika pārbaudītas piecas galvenās sistēmas funkcionālās prasības (PR-01, PR-02, PR-04, PR-05 un PR-06). Katrai sagatavoti divi līdz četri testa gadījumi, ar pareiziem un nepareiziem ievaddatiem. Rezultāti apkopoti 7.1. tabulā.

7.1. tabula

Programmatūras lietojamības testēšanas rezultāti

Lietotāja loma	Prasības ID	Apraksts	Ievaddati / situācija	Sagaidāmais rezultāts	Statuss
Viesis	PR-01	Lietotāja reģistrācija	Korekti ievaddati: unikāls lietotājvārds, derīgs e-pasts, parole $\geq$ 8 rakstzīmes	Konts izveidots, lietotājs novirzīts uz pieslēgšanās lapu	Nokārtots
Viesis	PR-01	Lietotāja reģistrācija	E-pasta adrese jau reģistrēta datubāzē	Kļūdas ziņojums: "E-pasts jau tiek izmantots"	Nokārtots
Viesis	PR-01	Lietotāja reģistrācija	Parole īsāka par 8 rakstzīmēm	Kļūdas ziņojums: "Parole pārāk īsa"	Nokārtots
Reģistrēts lietotājs	PR-02	Lietotāja pieslēgšanās	Pareizs e-pasts un parole	Veiksmīga pieslēgšanās, JWT tokens saglabāts localStorage	Nokārtots
Reģistrēts lietotājs	PR-02	Lietotāja pieslēgšanās	Korekts e-pasts, nepareiza parole	Kļūdas ziņojums: "Nepareizs lietotājvārds vai parole"	Nokārtots
Bloķēts lietotājs	PR-02	Lietotāja pieslēgšanās	Korekti akreditācijas dati, konts bloķēts	Kļūdas ziņojums: "Konts ir bloķēts"	Nokārtots
Autorizēts lietotājs	PR-04	Izslases komandas pievienošana	Lietotājs izvēlas komandu un nospiež "Pievienot izlasei"	Komanda saglabāta favourites tabulā, redzama panelī	Nokārtots
Autorizēts lietotājs	PR-04	Izslases komandas pievienošana	Lietotājam jau ir iecienīta komanda; izvēlas citu un saglabā	Iepriekšējā komanda aizstāta ar jauno (upsert), izlasē paliek viena	Nokārtots
Viesis	PR-04	Izslases komandas pievienošana	Neautorizēts lietotājs mēģina pievienot komandu	Serveris atgriež 401 Unauthorized	Nokārtots
Autorizēts lietotājs	PR-05	Prognozes iesniegšana	Ievadīta korekta prognoze: 2 : 1	Prognoze saglabāta, apstiprinājums saskarnē	Nokārtots
Autorizēts lietotājs	PR-05	Prognozes iesniegšana	Ievadīts negatīvs vārtu skaits: -1 : 0	Kļūdas ziņojums: "Vārtu skaitlis ārpus diapazona"	Nokārtots

Autorizēts lietotājs	PR-05	Proгноzes iesniegšana	Spēlei jau eksistē prognoze; ievadīta jauna vērtība	Iepriekšējā prognoze atjaunināta (upsert logika)	Nokārtots
Autorizēts lietotājs	PR-06	Komentāra pievienošana	Ievadīts parastais teksts, garums < 500 rakstzīmes	Komentārs saglabāts un uzreiz redzams bez atkārtotas ielādes	Nokārtots
Autorizēts lietotājs	PR-06	Komentāra pievienošana	Komentārs satur HTML tagu: <script>alert(1)</script>	HTML tagi izfiltrēti ar sanitize-html; saglabāts tīrs teksts	Nokārtots
Autorizēts lietotājs	PR-06	Komentāra pievienošana	Komentāra lauks tukšs	Kļūdas ziņojums: "Komentārs nevar būt tukšs"	Nokārtots
Viesis	PR-06	Komentāra pievienošana	Neautorizēts lietotājs mēģina pievienot komentāru	Serveris atgriež 401 Unauthorized	Nokārtots

NOBEIGUMS

Kvalifikācijas darba ietvaros izstrādāta futbola statistikas tīmekļa lietojumprogramma "Footbalyze". Izstrādē izmantots Vue 3 klienta pusē, Express.js servera pusē un MySQL datubāzē. Sistēma ir izvietota publiskā serverī un pieejama adresē <https://footbalyze.rvtdev.tech>.

Uzdevuma nostādnē definētās funkcionālās prasības ir pilnībā realizētas. PR-01 (reģistrācija) un PR-02 (pieslēgšanās) nodrošinātas ar bcrypt paroles šifrēšanu un JWT autentifikāciju. PR-03 (statistikas apskate) realizēta, integrējot football-data.org API ar datu attēlošanu komandu, spēlētāju, sacensību un spēļu skatā. PR-04 (izlases pārvaldība) un PR-05 (prognozes) darbojas ar upsert loģiku. PR-06 (komentāri) ietver XSS sanitizāciju ar sanitize-html un ziņošanas mehānismu. PR-07 (profila pārvaldība) ļauj mainīt lietotājevārdu, paroli un augšupielādēt avatāru. PR-08 (administratora funkcijas) nodrošina lietotāju bloķēšanu, lomu maiņu, kontu dzēšanu un ziņoto komentāru pārvaldību.

Turpmākai attīstībai ir vairāki konkrēti virzieni. Pirmkārt, var ieviest WebSocket savienojumu reāllaika rezultātu atjauninājumiem: pašlaik lapa jāatsvaidzina manuāli, lai redzētu pašreizējo spēles stāvokli. Otrkārt, prognožu sistēmu var paplašināt ar privātiem turnīriem, kur lietotāji piedalās savu draugu grupā ar kopīgu tabulu un balvām. Treškārt, lietotājiem var piedāvāt personalizētus paziņojumus par iecienīto komandu spēļu rezultātiem, izmantojot e-pasta vai push paziņojumu mehānismu. Šie uzlabojumi paplašinātu lietotāju iesaisti bez būtiskām izmaiņām pašreizējā arhitektūrā.

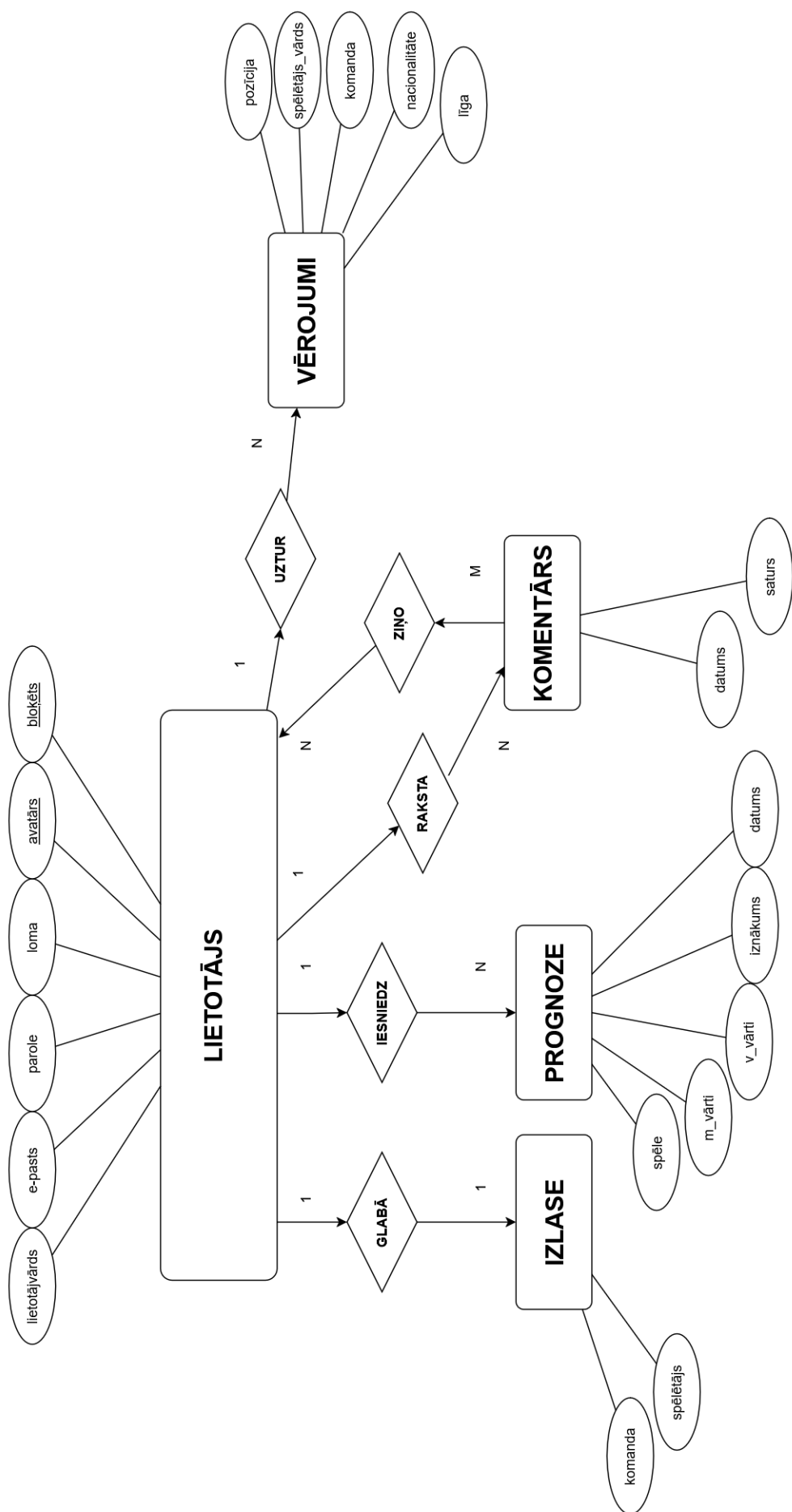
INFORMĀCIJAS AVOTI

1. Fielding, R.T. Architectural Styles and the Design of Network-based Software Architectures. – Irvine: University of California, 2000. – 162 lpp.
2. bcrypt.js. bcrypt.js npm package. – <https://www.npmjs.com/package/bcryptjs>. – (Resurss apskatīts 15.04.2026.)
3. Express.js. Express 5.x API Reference. – <https://expressjs.com/en/5x/api.html>. – (Resurss apskatīts 15.04.2026.)
4. express-rate-limit. express-rate-limit npm package. – <https://www.npmjs.com/package/express-rate-limit>. – (Resurss apskatīts 15.04.2026.)
5. football-data.org. Football Data API Documentation. – <https://www.football-data.org/documentation/quickstart>. – (Resurss apskatīts 15.04.2026.)
6. jsonwebtoken. jsonwebtoken npm package. – <https://www.npmjs.com/package/jsonwebtoken>. – (Resurss apskatīts 15.04.2026.)
7. multer. multer npm package. – <https://www.npmjs.com/package/multer>. – (Resurss apskatīts 15.04.2026.)
8. MySQL. MySQL 8.4 Reference Manual. – <https://dev.mysql.com/doc/refman/8.4/en/>. – (Resurss apskatīts 15.04.2026.)
9. Node.js. Node.js v18 Documentation. – <https://nodejs.org/en/docs>. – (Resurss apskatīts 15.04.2026.)
10. OWASP. Cross Site Scripting Prevention Cheat Sheet. – https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html. – (Resurss apskatīts 15.04.2026.)
11. sanitize-html. sanitize-html npm package. – <https://www.npmjs.com/package/sanitize-html>. – (Resurss apskatīts 15.04.2026.)
12. Vite. Vite Documentation. – <https://vite.dev/guide/>. – (Resurss apskatīts 15.04.2026.)
13. Vue Router. Vue Router 4 Documentation. – <https://router.vuejs.org/guide/>. – (Resurss apskatīts 15.04.2026.)
14. Vue.js. Vue 3 Guide. – <https://vuejs.org/guide/introduction.html>. – (Resurss apskatīts 15.04.2026.)

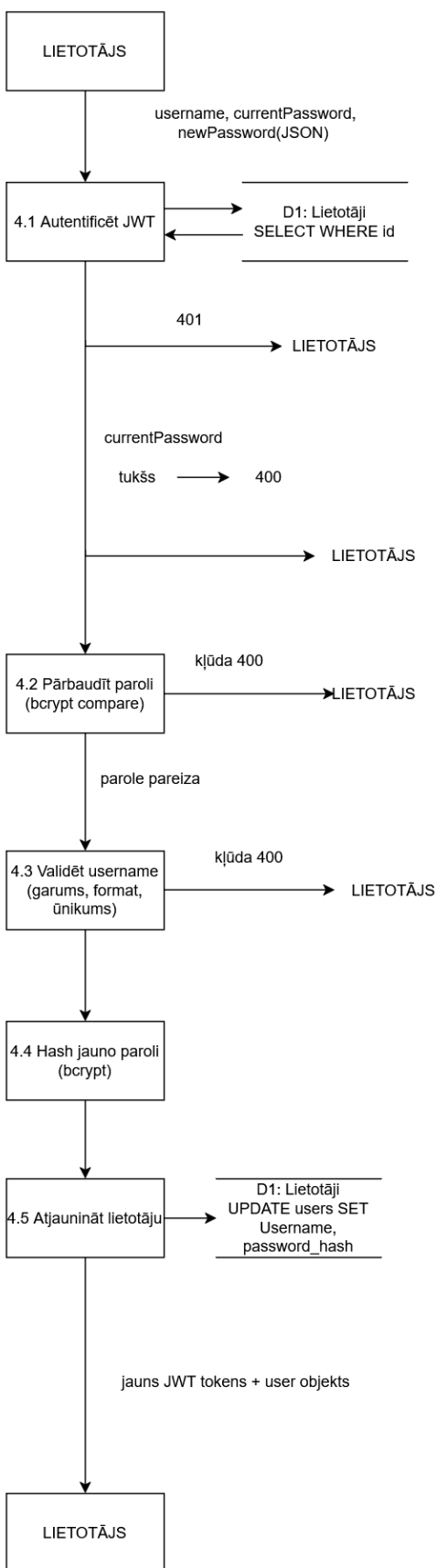
15. Vuex. Vuex 4 Documentation. – <https://vuex.vuejs.org/guide/>. – (Resurss apskatīts 15.04.2026.)

PIELIKUMI

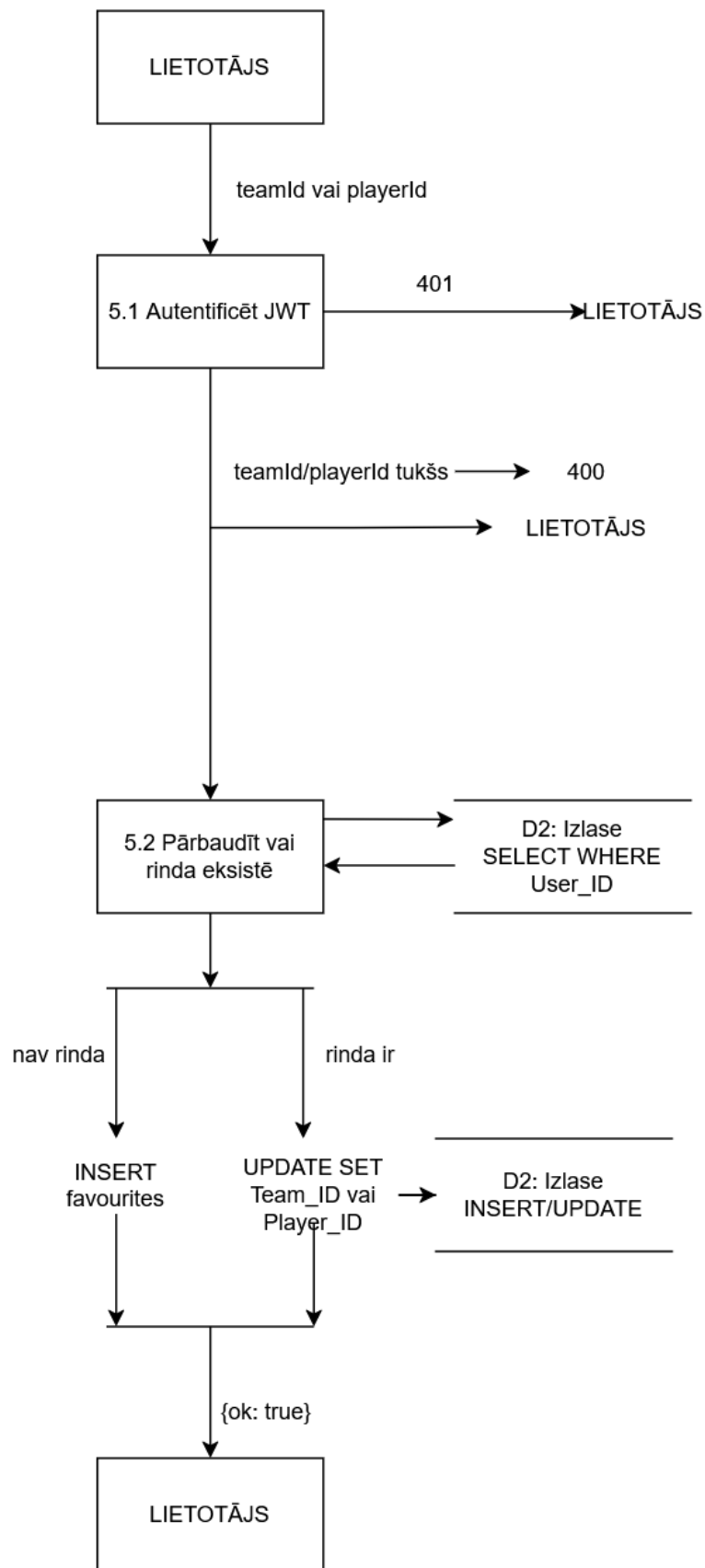
Footbalyze sistēmas ER modelis



Profila rediģēšanas un izlases pārvaldības moduļu datu plūsmas diagrammas



Profila rediģēšanas moduļa datu plūsmas diagramma



Izlases pārvaldības moduļa datu plūsmas diagramma

Galveno moduļu pirmkods

backend/auth.js – autentifikācijas middleware

Fails auth.js satur divus middleware: authenticate pārbauda JWT Bearer tokenu un bloķēšanas statusu, requireAdmin pārbauda, vai lietotāja loma ir "admin". Authenticate tiek pievienots visiem aizsargātajiem maršrutiem kā pirmā apstrādes funkcija pirms biznesa loģikas.

```
const jwt = require('jsonwebtoken');
const pool = require('./db');

function getSecret() {
  return process.env.JWT_SECRET;
}

async function authenticate(req, res, next) {
  const auth = req.headers.authorization;
  if (!auth || !auth.startsWith('Bearer '))
    return res.status(401).json({ error: 'Unauthorized' });
  try {
    req.user = jwt.verify(auth.slice(7), getSecret());
  } catch {
    return res.status(401).json({ error: 'Invalid token' });
  }
  try {
    const [rows] = await pool.query(
      'SELECT banned FROM users WHERE User_ID = ?', [req.user.id]
    );
    if (rows[0]?.banned)
      return res.status(403).json({ error: 'Account banned' });
  } catch { /* allow through if column missing */ }
  next();
}

function requireAdmin(req, res, next) {
  if (req.user?.role?.toLowerCase() !== 'admin')
    return res.status(403).json({ error: 'Forbidden' });
  next();
}

module.exports = { getSecret, authenticate, requireAdmin };
```

backend/favourites.js – izlases maršruti

Fails favourites.js realizē trīs API maršrutus izlases pārvaldībai. GET /api/favourites atgriež lietotāja saglabāto komandu un spēlētāju. POST /api/favourites/team un POST /api/favourites/player izmanto INSERT ... ON DUPLICATE KEY UPDATE loģiku, lai vienā darbībā gan izveidotu, gan atjauninātu ierakstu. Katrā lietotāja rindā drīkst būt tikai viena komanda un viens spēlētājs.

```
const express = require('express');
const pool = require('./db');
const { authenticate } = require('./auth');
```

```

const router = express.Router();

// GET /api/favourites
router.get('/', authenticate, async (req, res) => {
  const [rows] = await pool.query(
    'SELECT Team_ID, Player_ID FROM favourites WHERE User_ID = ? LIMIT 1',
    [req.user.id]
  );
  const fav = rows[0] || {};
  res.json({ team: fav.Team_ID || null, player: fav.Player_ID || null });
});

// POST /api/favourites/team
router.post('/team', authenticate, async (req, res) => {
  const { teamId } = req.body;
  if (!teamId) return res.status(400).json({ error: 'teamId required' });
  await pool.query(
    'INSERT INTO favourites (User_ID, Team_ID) VALUES (?, ?) ' +
    'ON DUPLICATE KEY UPDATE Team_ID = VALUES(Team_ID)',
    [req.user.id, teamId]
  );
  res.json({ ok: true });
});

// POST /api/favourites/player
router.post('/player', authenticate, async (req, res) => {
  const { playerId } = req.body;
  if (!playerId) return res.status(400).json({ error: 'playerId required' });
  await pool.query(
    'INSERT INTO favourites (User_ID, Player_ID) VALUES (?, ?) ' +
    'ON DUPLICATE KEY UPDATE Player_ID = VALUES(Player_ID)',
    [req.user.id, playerId]
  );
  res.json({ ok: true });
});

module.exports = router;

```

backend/server.js – lietojumprogrammas ieejas punkts

Fails server.js konfigurē Express lietotni: reģistrē middleware (CORS, JSON, statiskos failus), pievieno ātruma ierobežošanu autentifikācijas maršrutiem un reģistrē visus API maršrutus. Startējoties, izpilda datubāzes migrācijas un aktivizē prognožu rezultātu aprēķinu.

```

require('dotenv').config();
const express = require('express');
const cors = require('cors');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const rateLimit = require('express-rate-limit');
const path = require('path');
const pool = require('./db');
const { validatePassword } = require('./validators');
const footballRouter = require('./football');
const usersRouter = require('./users');
const favouritesRouter = require('./favourites');
const { router: predictionsRouter, resolveOutcomes } = require('./predictions');
const watchlistRouter = require('./watchlist');
const adminRouter = require('./admin');
const commentsRouter = require('./comments');
const profilesRouter = require('./profiles');

```

```

const app = express();
app.use(cors({ origin: process.env.FRONTEND_URL || 'http://localhost:5173' }));
app.use(express.json());
app.use('/uploads', express.static(path.join(__dirname, 'uploads')));

const SECRET = process.env.JWT_SECRET;
if (!SECRET) throw new Error('JWT_SECRET environment variable is not set');

const authLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 20,
  message: { error: 'Too many attempts, please try again later.' },
  standardHeaders: true,
  legacyHeaders: false,
});

app.post('/api/register', authLimiter, async (req, res) => {
  const { username, email, password } = req.body;
  const role = 'user';
  if (!username || !email || !password)
    return res.status(400).json({ field: null, message: 'All fields are required.' });
  if (username.length < 3 || username.length > 20)
    return res.status(400).json({ field: 'username', message: 'Username must be between 3 and 20 characters.' });
  if (!/^[a-zA-Z0-9_]+$/.test(username))
    return res.status(400).json({ field: 'username', message: 'Username can only contain letters, numbers, and underscores.' });
  if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email))
    return res.status(400).json({ field: 'email', message: 'Please enter a valid email address.' });
  const pwErr = validatePassword(password);
  if (pwErr) return res.status(400).json(pwErr);
  try {
    const [existing] = await pool.query(
      'SELECT Username, Email FROM users WHERE Username = ? OR Email = ?',
      [username, email]
    );
    if (existing.length > 0) {
      const taken = existing[0];
      if (taken.Username === username)
        return res.status(409).json({ field: 'username', message: 'That username is already taken.' });
      return res.status(409).json({ field: 'email', message: 'An account with that email already exists.' });
    }
    const hashedPassword = await bcrypt.hash(password, 10);
    const [result] = await pool.query(
      'INSERT INTO users (Username, Email, Password, password_hash, Role) VALUES (?, ?, ?, ?, ?)',
      [username, email, hashedPassword, hashedPassword, role]
    );
    res.status(201).json({ id: result.insertId, username, email, role });
  } catch (err) {
    console.error('Registration error:', err.message);
    res.status(500).json({ field: null, message: 'Registration failed. Please try again.' });
  }
});

app.post('/api/login', authLimiter, async (req, res) => {
  const { email, password } = req.body;
  if (!email || !password)
    return res.status(400).json({ message: 'Email and password are required.' });
  try {
    const [rows] = await pool.query('SELECT * FROM users WHERE Email = ?', [email]);
  }
});

```

```

    const user = rows[0];
    if (!user) return res.status(401).json({ message: 'No account found with that email.' });
    const match = await bcrypt.compare(password, user.password_hash);
    if (!match) return res.status(401).json({ message: 'Incorrect password.' });
    if (user.banned) return res.status(403).json({ message: 'Your account has been banned.' });
    const token = jwt.sign(
      { id: user.User_ID, username: user.Username, role: user.Role?.trim() },
      SECRET, { expiresIn: '7d' }
    );
    res.json({ token, user: { id: user.User_ID, username: user.Username, email: user.Email, role: user.Role?.trim() } });
  } catch (err) {
    console.error('Login error:', err.message);
    res.status(500).json({ message: 'Server error during login. Please try again.' });
  }
});

app.use('/api', footballRouter);
app.use('/api/user', usersRouter);
app.use('/api/favourites', favouritesRouter);
app.use('/api/predictions', predictionsRouter);
app.use('/api/watchlist', watchlistRouter);
app.use('/api/admin', adminRouter);
app.use('/api/comments', commentsRouter);
app.use('/api/users', profilesRouter);

app.use((_req, res) => res.status(404).json({ error: 'Not found' }));

module.exports = app;

if (require.main === module) {
  const PORT = process.env.PORT || 3000;
  app.listen(PORT, async () => {
    console.log(`Backend running at http://localhost:${PORT}`);
    await runMigrations();
    resolveOutcomes();
    setInterval(resolveOutcomes, 15 * 60 * 1000);
  });
}

```

backend/db.js – datubāzes savienojuma pūls

Fails db.js izveido mysql2 savienojumu pūlu, izmantojot vides mainīgos. Pūls tiek eksportēts un izmantots visos maršrutu failos.

```

const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: process.env.DB_HOST || 'localhost',
  user: process.env.DB_USER || 'root',
  password: process.env.DB_PASSWORD || '',
  database: process.env.DB_NAME || 'fullstack_db',
  waitForConnections: true,
  connectionLimit: 10
});

pool.getConnection()
  .then(connection => {
    console.log('Successfully connected to the database');
    connection.release();
  })

```

```
.catch(error => {
  console.error('Error connecting to the database:', error);
});
```

```
module.exports = pool;
```

backend/validators.js – paroles validācija

Fails validators.js eksportē validatePassword funkciju, ko izmanto reģistrācijas un iestatījumu maršruti, lai pārbaudītu paroles stiprumu.

```
function validatePassword(password, field = 'password') {
  if (password.length < 8)
    return { field, message: 'Password must be at least 8 characters.' };
  if (password.length > 72)
    return { field, message: 'Password must be 72 characters or fewer.' };
  if (!/[A-Z]/.test(password))
    return { field, message: 'Password must contain at least one uppercase letter.' };
  if (!/[0-9]/.test(password))
    return { field, message: 'Password must contain at least one number.' };
  return null;
}
```

```
module.exports = { validatePassword };
```

backend/comments.js – komentāru maršruti

Fails comments.js pārvalda komentāru lasīšanu, rakstīšanu, ziņošanu un dzēšanu. XSS aizsardzība realizēta ar sanitize-html pirms saglabāšanas datubāzē.

```
const express = require('express');
const sanitizeHtml = require('sanitize-html');
const pool = require('./db');
const { authenticate } = require('./auth');

const router = express.Router();

router.get('/counts', async (req, res) => {
  const ids = (req.query.matchIds || '').split(',').map(n => parseInt(n, 10)).filter(Boolean);
  if (!ids.length) return res.json({});
  try {
    const [rows] = await pool.query(
      `SELECT Match_ID, COUNT(*) AS cnt FROM comments WHERE Match_ID IN (${ids.map(() => '?').join(',')}) GROUP BY Match_ID`,
      ids
    );
    const result = {};
    for (const r of rows) result[r.Match_ID] = r.cnt;
    res.json(result);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch counts' });
  }
});

router.get('/', async (req, res) => {
  const matchId = parseInt(req.query.matchId, 10);
  if (!matchId) return res.status(400).json({ error: 'matchId required' });
  try {
    const [rows] = await pool.query(
      `SELECT Comment_ID, User_ID, Username, Content, CreatedAt FROM comments WHERE Match_ID = ? ORDER BY CreatedAt ASC`,

```

```

        [matchId]
    );
    res.json(rows);
} catch (err) {
    res.status(500).json({ error: 'Failed to fetch comments' });
}
});

router.post('/', authenticate, async (req, res) => {
    const { matchId, content } = req.body;
    if (!matchId || !content?.trim()) return res.status(400).json({ error: 'matchId and content required' });
    const clean = sanitizeHtml(content.trim(), { allowedTags: [], allowedAttributes: {} });
    if (!clean) return res.status(400).json({ error: 'Comment content is empty after sanitization' });
    if (clean.length > 500) return res.status(400).json({ error: 'Comment too long (max 500 chars)' });
    try {
        const [result] = await pool.query(
            'INSERT INTO comments (User_ID, Username, Match_ID, Content) VALUES (?, ?, ?, ?)',
            [req.user.id, req.user.username, matchId, clean]
        );
        res.status(201).json({ Comment_ID: result.insertId, User_ID: req.user.id, Username: req.user.username, Match_ID: matchId, Content: clean, CreatedAt: new Date() });
    } catch (err) {
        res.status(500).json({ error: 'Failed to post comment' });
    }
});

router.post('/:id/report', authenticate, async (req, res) => {
    const id = parseInt(req.params.id, 10);
    try {
        const [rows] = await pool.query('SELECT User_ID FROM comments WHERE Comment_ID = ?', [id]);
        if (!rows.length) return res.status(404).json({ error: 'Comment not found' });
        if (rows[0].User_ID === req.user.id) return res.status(400).json({ error: 'Cannot report your own comment' });
        await pool.query('INSERT IGNORE INTO comment_reports (Comment_ID, Reporter_ID) VALUES (?, ?)', [id, req.user.id]);
        res.json({ ok: true });
    } catch (err) {
        res.status(500).json({ error: 'Failed to report comment' });
    }
});

router.delete('/:id', authenticate, async (req, res) => {
    const id = parseInt(req.params.id, 10);
    try {
        const [rows] = await pool.query('SELECT User_ID FROM comments WHERE Comment_ID = ?', [id]);
        if (!rows.length) return res.status(404).json({ error: 'Comment not found' });
        if (rows[0].User_ID !== req.user.id && req.user.role?.toLowerCase() !== 'admin') return res.status(403).json({ error: 'Forbidden' });
        await pool.query('DELETE FROM comment_reports WHERE Comment_ID = ?', [id]);
        await pool.query('DELETE FROM comments WHERE Comment_ID = ?', [id]);
        res.json({ ok: true });
    } catch (err) {
        res.status(500).json({ error: 'Failed to delete comment' });
    }
});

module.exports = router;

```

backend/predictions.js – prognožu maršruti un rezultātu aprēķins

Fails predictions.js pārvalda prognožu iesniegšanu, līdertabulu un automātisko rezultātu aprēķinu. resolveOutcomes tiek izsaukts ik 15 minūtes, lai atjauninātu pabeigto spēļu prognožu statusus.

```
const express = require('express');
const pool = require('./db');
const { getMatch } = require('./footballApi');
const { authenticate } = require('./auth');

const router = express.Router();

router.get('/', authenticate, async (req, res) => {
  try {
    const [rows] = await pool.query(
      'SELECT Match_ID, HomeGoals, AwayGoals, Outcome, CreatedAt FROM predictions WHERE User_ID = ?',
      [req.user.id]
    );
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch predictions' });
  }
});

router.post('/', authenticate, async (req, res) => {
  const { matchId, homeGoals, awayGoals } = req.body;
  if (matchId == null || homeGoals == null || awayGoals == null)
    return res.status(400).json({ error: 'matchId, homeGoals, awayGoals required' });
  const hg = parseInt(homeGoals, 10);
  const ag = parseInt(awayGoals, 10);
  if (!Number.isFinite(hg) || !Number.isFinite(ag) || hg < 0 || ag < 0 || hg > 20 || ag > 20)
    return res.status(400).json({ error: 'Goals must be integers 0-20' });
  try {
    await pool.query(
      `INSERT INTO predictions (User_ID, Match_ID, HomeGoals, AwayGoals)
      VALUES (?, ?, ?, ?)
      ON DUPLICATE KEY UPDATE HomeGoals = VALUES(HomeGoals), AwayGoals =
      VALUES(AwayGoals)`,
      [req.user.id, matchId, hg, ag]
    );
    res.json({ ok: true });
  } catch (err) {
    res.status(500).json({ error: 'Failed to save prediction' });
  }
});

router.get('/leaderboard', async (req, res) => {
  try {
    const [rows] = await pool.query(`
      SELECT u.Username, COUNT(*) AS total,
      SUM(CASE WHEN p.Outcome = 'exact' THEN 3
      WHEN p.Outcome = 'correct' THEN 1 ELSE 0 END) AS points,
      SUM(p.Outcome = 'exact') AS exact_count,
      SUM(p.Outcome = 'correct') AS correct_count,
      SUM(p.Outcome = 'wrong') AS wrong_count
      FROM predictions p JOIN users u ON p.User_ID = u.User_ID
      WHERE p.Outcome IS NOT NULL
      GROUP BY p.User_ID, u.Username ORDER BY points DESC, exact_count DESC
    `);
  }
});
```

```

    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch leaderboard' });
  }
});

async function resolveOutcomes() {
  let conn;
  try {
    conn = await pool.getConnection();
    const [unresolved] = await conn.query(
      'SELECT Prediction_ID, User_ID, Match_ID, HomeGoals, AwayGoals FROM predictions
WHERE Outcome IS NULL'
    );
    if (!unresolved.length) return;
    const matchIds = [...new Set(unresolved.map(p => p.Match_ID))];
    const results = {};
    for (const id of matchIds) {
      try { results[id] = await getMatch(id); } catch {}
      await new Promise(r => setTimeout(r, 700));
    }
    for (const pred of unresolved) {
      const match = results[pred.Match_ID];
      if (!match || match.status !== 'FINISHED') continue;
      const actualHome = match.score?.fullTime?.home;
      const actualAway = match.score?.fullTime?.away;
      if (actualHome == null || actualAway == null) continue;
      let outcome;
      if (pred.HomeGoals === actualHome && pred.AwayGoals === actualAway) {
        outcome = 'exact';
      } else {
        const predResult = Math.sign(pred.HomeGoals - pred.AwayGoals);
        const actualResult = Math.sign(actualHome - actualAway);
        outcome = predResult === actualResult ? 'correct' : 'wrong';
      }
      await conn.query('UPDATE predictions SET Outcome = ? WHERE Prediction_ID = ?',
[outcome, pred.Prediction_ID]);
    }
  } catch (err) {
    console.error('resolveOutcomes error:', err.message);
  } finally {
    if (conn) conn.release();
  }
}

module.exports = { router, resolveOutcomes };

```

backend/watchlist.js – vērošanas saraksta maršruti

Fails watchlist.js nodrošina spēlētāju pievienošanu un noņemšanu no vērošanas saraksta.

Saglabā spēlētāja metadatus (vārds, komanda, pozīcija, tautība).

```

const express = require('express');
const pool = require('./db');
const { authenticate } = require('./auth');

const router = express.Router();

router.get('/', authenticate, async (req, res) => {
  try {
    const [rows] = await pool.query(
      'SELECT Player_ID, Player_Name, Team_ID, Team_Name, Position, DetailedPosition,
Nationality, LeagueCode FROM watchlist WHERE User_ID = ? ORDER BY CreatedAt DESC',

```



```

    [req.user.id]
  );
  res.json(rows);
} catch (err) {
  res.status(500).json({ error: 'Failed to fetch watchlist' });
}
});

router.post('/', authenticate, async (req, res) => {
  const { playerId, playerName, teamId, teamName, position, detailedPosition,
    nationality, leagueCode } = req.body;
  if (!playerId) return res.status(400).json({ error: 'playerId required' });
  try {
    await pool.query(
      `INSERT INTO watchlist (User_ID, Player_ID, Player_Name, Team_ID, Team_Name,
        Position, DetailedPosition, Nationality, LeagueCode)
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        ON DUPLICATE KEY UPDATE
          Player_Name = VALUES(Player_Name), Team_Name = VALUES(Team_Name),
          Position = VALUES(Position), DetailedPosition = VALUES(DetailedPosition),
          Nationality = VALUES(Nationality), LeagueCode = VALUES(LeagueCode)`,
      [req.user.id, playerId, playerName || null, teamId || null, teamName || null,
        position || null, detailedPosition || null, nationality || null, leagueCode ||
        null]
    );
    res.json({ ok: true });
  } catch (err) {
    res.status(500).json({ error: 'Failed to add to watchlist' });
  }
});

router.delete('/:playerId', authenticate, async (req, res) => {
  const playerId = parseInt(req.params.playerId, 10);
  if (!playerId) return res.status(400).json({ error: 'playerId required' });
  try {
    await pool.query('DELETE FROM watchlist WHERE User_ID = ? AND Player_ID = ?',
      [req.user.id, playerId]);
    res.json({ ok: true });
  } catch (err) {
    res.status(500).json({ error: 'Failed to remove from watchlist' });
  }
});

module.exports = router;

```

backend/admin.js – administratora maršruti

Fails admin.js nodrošina administratora funkcijas: lietotāju saraksta apskate, lomas maiņa, bloķēšana, dzēšana un ziņoto komentāru pārvaldība. Visi maršruti aizsargāti ar authenticate un requireAdmin middleware.

```

const express = require('express');
const pool = require('./db');
const { authenticate, requireAdmin } = require('./auth');

const router = express.Router();
const guard = [authenticate, requireAdmin];

router.get('/users', guard, async (req, res) => {
  try {
    const [rows] = await pool.query(
      'SELECT User_ID, Username, Email, Role, banned FROM users ORDER BY User_ID ASC'
    );
  }
});

```

```

    });
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch users' });
  }
});

router.patch('/users/:id/role', guard, async (req, res) => {
  const id = parseInt(req.params.id, 10);
  const { role } = req.body;
  if (!['user', 'admin'].includes(role)) return res.status(400).json({ error: 'Role must be "user" or "admin"' });
  if (id === req.user.id) return res.status(400).json({ error: 'Cannot change your own role' });
  try {
    await pool.query('UPDATE users SET Role = ? WHERE User_ID = ?', [role, id]);
    res.json({ ok: true });
  } catch (err) {
    res.status(500).json({ error: 'Failed to update role' });
  }
});

router.patch('/users/:id/ban', guard, async (req, res) => {
  const id = parseInt(req.params.id, 10);
  if (id === req.user.id) return res.status(400).json({ error: 'Cannot ban yourself' });
  const { banned } = req.body;
  try {
    await pool.query('UPDATE users SET banned = ? WHERE User_ID = ?', [banned ? 1 : 0, id]);
    res.json({ ok: true });
  } catch (err) {
    res.status(500).json({ error: 'Failed to update ban status' });
  }
});

router.delete('/users/:id', guard, async (req, res) => {
  const id = parseInt(req.params.id, 10);
  if (id === req.user.id) return res.status(400).json({ error: 'Cannot delete your own account' });
  try {
    await pool.query('DELETE FROM predictions WHERE User_ID = ?', [id]);
    await pool.query('DELETE FROM comment_reports WHERE Reporter_ID = ?', [id]);
    await pool.query('DELETE FROM comments WHERE User_ID = ?', [id]);
    await pool.query('DELETE FROM watchlist WHERE User_ID = ?', [id]);
    await pool.query('DELETE FROM favourites WHERE User_ID = ?', [id]);
    await pool.query('DELETE FROM users WHERE User_ID = ?', [id]);
    res.json({ ok: true });
  } catch (err) {
    res.status(500).json({ error: 'Failed to delete user' });
  }
});

router.get('/reports', guard, async (req, res) => {
  try {
    const [rows] = await pool.query(`
      SELECT c.Comment_ID, c.Username, c.Content, c.Match_ID, c.CreatedAt,
        COUNT(r.Report_ID) AS report_count
      FROM comments c JOIN comment_reports r ON r.Comment_ID = c.Comment_ID
      GROUP BY c.Comment_ID ORDER BY report_count DESC, c.CreatedAt DESC
    `);
    res.json(rows);
  } catch (err) {
    res.status(500).json({ error: 'Failed to fetch reports' });
  }
});

```

```
});

router.delete('/reports/:commentId', guard, async (req, res) => {
  const id = parseInt(req.params.commentId, 10);
  try {
    await pool.query('DELETE FROM comment_reports WHERE Comment_ID = ?', [id]);
    res.json({ ok: true });
  } catch (err) {
    res.status(500).json({ error: 'Failed to dismiss report' });
  }
});

module.exports = router;
```

backend/users.js – profila iestatījumu maršruti

Fails users.js pārvalda lietotāja profila atjaunināšanu: lietotājvārda un paroles maiņu un avatāra augšupielādi. Veiksmīgas atjaunināšanas gadījumā tiek izsniegts jauns JWT tokens ar atjaunināto lietotājvārdu.

```
const express = require('express');
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');
const multer = require('multer');
const path = require('path');
const pool = require('./db');
const { authenticate } = require('./auth');
const { validatePassword } = require('./validators');

const router = express.Router();
const SECRET = process.env.JWT_SECRET;

const avatarStorage = multer.diskStorage({
  destination: (req, file, cb) => cb(null, path.join(__dirname, 'uploads')),
  filename: (req, file, cb) => {
    const ext = path.extname(file.originalname).toLowerCase() || '.jpg';
    cb(null, `avatar_${req.user.id}_${Date.now()}${ext}`);
  },
});

const uploadAvatar = multer({
  storage: avatarStorage,
  limits: { fileSize: 3 * 1024 * 1024 },
  fileFilter: (req, file, cb) => {
    if (!file.mimetype.startsWith('image/')) return cb(new Error('Images only'));
    cb(null, true);
  },
});

router.get('/me', authenticate, async (req, res) => {
  try {
    const [rows] = await pool.query('SELECT avatar_url FROM users WHERE User_ID = ?',
[req.user.id]);
    res.json({ avatarUrl: rows[0]?.avatar_url || null });
  } catch (err) {
    res.status(500).json({ message: 'Server error' });
  }
});

router.post('/avatar', authenticate, uploadAvatar.single('avatar'), async (req, res) =>
{
```

```

    if (!req.file) return res.status(400).json({ message: 'No file uploaded' });
    const avatarUrl = `/uploads/${req.file.filename}`;
    try {
      await pool.query('UPDATE users SET avatar_url = ? WHERE User_ID = ?', [avatarUrl, req.user.id]);
      res.json({ avatarUrl });
    } catch (err) {
      res.status(500).json({ message: 'Failed to save avatar' });
    }
  });

router.put('/settings', authenticate, async (req, res) => {
  const userId = req.user.id;
  const { username, currentPassword, newPassword } = req.body;
  if (!currentPassword) return res.status(400).json({ field: 'currentPassword', message: 'Current password is required.' });
  try {
    const [rows] = await pool.query('SELECT * FROM users WHERE User_ID = ?', [userId]);
    const user = rows[0];
    if (!user) return res.status(404).json({ message: 'User not found.' });
    const valid = await bcrypt.compare(currentPassword, user.password_hash);
    if (!valid) return res.status(400).json({ field: 'currentPassword', message: 'Current password is incorrect.' });
    if (username && username !== user.Username) {
      if (username.length < 3 || username.length > 20)
        return res.status(400).json({ field: 'username', message: 'Username must be 3-20 characters.' });
      const [taken] = await pool.query('SELECT User_ID FROM users WHERE Username = ? AND User_ID != ?', [username, userId]);
      if (taken.length) return res.status(400).json({ field: 'username', message: 'Username already taken.' });
    }
    const newUsername = username || user.Username;
    let newHash = user.password_hash;
    if (newPassword) {
      const pwErr = validatePassword(newPassword, 'newPassword');
      if (pwErr) return res.status(400).json(pwErr);
      newHash = await bcrypt.hash(newPassword, 10);
    }
    await pool.query('UPDATE users SET Username = ?, Email = ?, password_hash = ? WHERE User_ID = ?', [newUsername, user.Email, newHash, userId]);
    const token = jwt.sign({ id: userId, username: newUsername, role: user.Role?.trim() }, SECRET, { expiresIn: '7d' });
    res.json({ token, user: { id: userId, username: newUsername, email: user.Email, role: user.Role?.trim() } });
  } catch (err) {
    res.status(500).json({ message: 'Server error. Please try again.' });
  }
});

module.exports = router;

```

backend/footballApi.js – football-data.org API klients ar kešatmiņu

Fails footballApi.js realizē visus pieprasījumus uz football-data.org API. Kešatmiņa novērš API limitu pārsniegšanu; inflight deduplication novērš vairākus vienlaicīgus pieprasījumus par vienu un to pašu resursu.

```

const axios = require('axios');

const BASE = 'https://api.football-data.org/v4';
const KEY = process.env.FOOTBALL_DATA_API_KEY;

```

```

const cache = {};
const inflight = {};

function cached(key, ttlMs, fetcher) {
  const now = Date.now();
  if (cache[key] && cache[key].expiresAt > now)
    return Promise.resolve(cache[key].data);
  if (inflight[key]) return inflight[key];
  inflight[key] = fetcher().then(data => {
    cache[key] = { data, expiresAt: now + ttlMs };
    delete inflight[key];
    return data;
  }).catch(err => {
    delete inflight[key];
    throw err;
  });
  return inflight[key];
}

const http = axios.create({
  baseURL: BASE,
  headers: { 'X-Auth-Token': KEY },
});

const MIN = 60_000;
const HOUR = 60 * MIN;
const DAY = 24 * HOUR;

const COMPETITIONS = [
  { code: 'PL', name: 'Premier League', country: 'England', flag: '🇬🇧' },
  { code: 'PD', name: 'La Liga', country: 'Spain', flag: '🇪🇸' },
  { code: 'BL1', name: 'Bundesliga', country: 'Germany', flag: '🇩🇪' },
  { code: 'SA', name: 'Serie A', country: 'Italy', flag: '🇮🇹' },
  { code: 'FL1', name: 'Ligue 1', country: 'France', flag: '🇫🇷' },
  { code: 'ELC', name: 'Championship', country: 'England', flag: '🇬🇧' },
  { code: 'PPL', name: 'Primeira Liga', country: 'Portugal', flag: '🇵🇹' },
  { code: 'DED', name: 'Eredivisie', country: 'Netherlands', flag: '🇳🇱' },
  { code: 'BSA', name: 'Brasileirao Serie A', country: 'Brazil', flag: '🇧🇷' },
  { code: 'CL', name: 'UEFA Champions League', country: 'Europe', flag: '🇪🇺' },
];

function getCompetitions() { return COMPETITIONS; }

function getStandings(code) {
  return cached(`standings:${code}`, 6 * HOUR, async () => {
    const res = await http.get(`/competitions/${code}/standings`);
    return res.data;
  });
}

function getMatches(code, options = {}) {
  const params = {};
  if (options.season) params.season = options.season;
  if (options.status) params.status = options.status;
  const cacheKey = `matches:${code}:${JSON.stringify(params)}`;
  return cached(cacheKey, 30 * MIN, async () => {
    const res = await http.get(`/competitions/${code}/matches`, { params });
    return res.data;
  });
}

function getTeams(code) {

```

```

    return cached(`teams:${code}`, DAY, async () => {
      const res = await http.get(`/competitions/${code}/teams`);
      return res.data;
    });
  }

function getScorers(code, limit = 50) {
  return cached(`scorers:${code}:${limit}`, 6 * HOUR, async () => {
    const res = await http.get(`/competitions/${code}/scorers`, { params: { limit } });
    return res.data;
  });
}

function getTeamDetail(teamId) {
  return cached(`team:${teamId}`, DAY, async () => {
    const res = await http.get(`/teams/${teamId}`);
    return res.data;
  });
}

function getMatch(matchId) {
  return cached(`match:${matchId}`, 5 * MIN, async () => {
    const res = await http.get(`/matches/${matchId}`);
    return res.data;
  });
}

module.exports = { getCompetitions, getStandings, getMatches, getTeams, getScorers,
  getTeamDetail, getMatch };

```

backend/football.js – futbola datu maršruti

Fails football.js eksponē visus publiski pieejamos futbola datu API maršrutus: līgas, komandas, spēlētāji, spēles, tabulas, vārtu guvēji, H2H un statistika. Dati tiek normalizēti pirms nosūtīšanas klientam.

```

const express = require('express');
const fdApi = require('./footballApi');

const router = express.Router();
const SEASON = parseInt(process.env.CURRENT_SEASON, 10) || 2025;

const DOMESTIC_LEAGUES = new Set(['PL', 'PD', 'BL1', 'SA', 'FL1', 'ELC', 'PPL', 'DED', 'BSA']);
const SUPPORTED_LEAGUES = new Set(['PL', 'PD', 'BL1', 'SA', 'FL1', 'ELC', 'PPL', 'DED', 'BSA', 'CL']);
const LEAGUE_NAMES = {
  PL: 'Premier League', PD: 'La Liga', BL1: 'Bundesliga', SA: 'Serie A',
  FL1: 'Ligue 1', ELC: 'Championship', PPL: 'Primeira Liga', DED: 'Eredivisie',
  BSA: 'Brasileirao', CL: 'Champions League',
};

const DETAILED_POSITION_MAP = {
  'goalkeeper': 'Goalkeeper', 'centre-back': 'Centre-Back', 'left-back': 'Left-Back',
  'right-back': 'Right-Back', 'wing-back': 'Wing-Back', 'defensive midfield':
'Defensive Mid',
  'central midfield': 'Central Mid', 'attacking midfield': 'Attacking Mid',
  'left winger': 'Left Winger', 'right winger': 'Right Winger',
  'centre-forward': 'Centre-Forward', 'striker': 'Striker',
  'defence': 'Defender', 'midfield': 'Midfielder', 'offence': 'Forward', 'offense':
'Forward',

```

```

};

const BROAD_CATEGORY = {
  'Goalkeeper': 'Goalkeeper', 'Centre-Back': 'Defender', 'Left-Back': 'Defender',
  'Right-Back': 'Defender', 'Wing-Back': 'Defender', 'Defensive Mid': 'Midfielder',
  'Central Mid': 'Midfielder', 'Attacking Mid': 'Midfielder', 'Left Winger': 'Forward',
  'Right Winger': 'Forward', 'Centre-Forward': 'Forward', 'Striker': 'Forward',
  'Defender': 'Defender', 'Midfielder': 'Midfielder', 'Forward': 'Forward',
};

function detailedPosition(pos) {
  if (!pos || pos === 'null') return null;
  return DETAILED_POSITION_MAP[pos.toLowerCase()] || null;
}

function mapPosition(pos) {
  if (!pos || pos === 'null') return 'Unknown';
  const detailed = detailedPosition(pos);
  if (detailed && BROAD_CATEGORY[detailed]) return BROAD_CATEGORY[detailed];
  const p = pos.toLowerCase();
  if (p.includes('goalkeeper')) return 'Goalkeeper';
  if (p.includes('back') || p.includes('defender') || p.includes('defence')) return
'Defender';
  if (p.includes('midfield')) return 'Midfielder';
  if (p.includes('winger') || p.includes('forward') || p.includes('striker')) return
'Forward';
  return 'Unknown';
}

function calcAge(dob) {
  if (!dob) return null;
  const birth = new Date(dob);
  const now = new Date();
  let age = now.getFullYear() - birth.getFullYear();
  const m = now.getMonth() - birth.getMonth();
  if (m < 0 || (m === 0 && now.getDate() < birth.getDate())) age--;
  return age;
}

function splitName(fullName) {
  if (!fullName) return { Name: 'Unknown', Surname: '' };
  const parts = fullName.trim().split(' ');
  if (parts.length === 1) return { Name: parts[0], Surname: '' };
  return { Name: parts.slice(0, -1).join(' '), Surname: parts[parts.length - 1] };
}

function normalizeTeam(t) {
  return {
    Team_ID: t.id, Team_Name: t.name, Crest: t.crest || null,
    Coach_name: t.coach?.name || null, Coach_nationality: t.coach?.nationality || null,
    Country: t.area?.name || 'England',
  };
}

function normalizeMatch(m) {
  const ft = m.score?.fullTime;
  const Score = (ft && ft.home !== null && ft.away !== null) ? `${ft.home}-${ft.away}` :
null;
  return {
    Match_ID: m.id, MatchDate: m.utcDate, Status: m.status,
    Matchday: m.matchday ?? null, Stage: m.stage ?? null, Score,
    HomeTeam: m.homeTeam?.name, HomeTeamID: m.homeTeam?.id, HomeCrest:
m.homeTeam?.crest || null,

```

```

      AwayTeam: m.awayTeam?.name, AwayTeamID: m.awayTeam?.id, AwayCrest:
m.awayTeam?.crest || null,
    };
  }

router.get('/leagues', (req, res) => res.json(fdApi.getCompetitions()));

router.get('/leagues/:code/standings', async (req, res) => {
  try { res.json(await fdApi.getStandings(req.params.code)); }
  catch (err) { res.status(err.response?.status || 500).json({ error: 'Failed to fetch
standings' }); }
});

router.get('/leagues/:code/matches', async (req, res) => {
  try { res.json(await fdApi.getMatches(req.params.code, { season: req.query.season
})); }
  catch (err) { res.status(err.response?.status || 500).json({ error: 'Failed to fetch
matches' }); }
});

router.get('/leagues/:code/teams', async (req, res) => {
  try { res.json(await fdApi.getTeams(req.params.code)); }
  catch (err) { res.status(err.response?.status || 500).json({ error: 'Failed to fetch
teams' }); }
});

router.get('/leagues/:code/scorers', async (req, res) => {
  try { res.json(await fdApi.getScorers(req.params.code)); }
  catch (err) { res.status(err.response?.status || 500).json({ error: 'Failed to fetch
scorers' }); }
});

router.get('/teams/:id', async (req, res) => {
  const id = Number(req.params.id);
  try {
    const teamData = await fdApi.getTeamDetail(id);
    const comps = teamData.runningCompetitions || [];
    const leagueCode = (comps.find(c => DOMESTIC_LEAGUES.has(c.code)) || comps.find(c
=> SUPPORTED_LEAGUES.has(c.code)))?.code || 'PL';
    const [matchData, standingsData] = await Promise.all([
      fdApi.getMatches(leagueCode, { season: SEASON }),
      fdApi.getStandings(leagueCode).catch(() => null),
    ]);
    const posOrder = ['Goalkeeper', 'Defender', 'Midfielder', 'Forward'];
    const players = (teamData.squad || []).
      .map(p => {
        const { Name, Surname } = splitName(p.name);
        return { Player_ID: p.id, Name, Surname, Position: mapPosition(p.position),
          DetailedPosition: detailedPosition(p.position), Nationality: p.nationality ||
null,
          Age: calcAge(p.dateOfBirth), Team_ID: id };
      })
      .filter(p => p.Position !== 'Unknown')
      .sort((a, b) => posOrder.indexOf(a.Position) - posOrder.indexOf(b.Position));
    const recentMatches = (matchData.matches || []).
      .filter(m => m.status === 'FINISHED' && (m.homeTeam?.id === id || m.awayTeam?.id
=== id))
      .sort((a, b) => new Date(b.utcDate) - new Date(a.utcDate))
      .slice(0, 5).map(normalizeMatch);
    let tableRow = null;
    if (standingsData) {
      const total = (standingsData.standings || []).find(s => s.type === 'TOTAL');
      const row = total?.table?.find(r => r.team.id === id);
      if (row) tableRow = { Position: row.position, P: row.playedGames, W: row.won,

```



```

        D: row.draw, L: row.lost, GF: row.goalsFor, GA: row.goalsAgainst, Pts:
row.points };
    }
    res.json({ ...normalizeTeam(teamData), players, recentMatches, leagueCode,
        leagueName: LEAGUE_NAMES[leagueCode] || leagueCode, tableRow });
    } catch (err) {
        res.status(err.response?.status || 500).json({ error: 'Failed to fetch team' });
    }
});

router.get('/players', async (req, res) => {
    const code = req.query.code || 'PL';
    try {
        const data = await fdApi.getTeams(code);
        const players = [];
        for (const t of (data.teams || [])) {
            for (const p of (t.squad || [])) {
                const { Name, Surname } = splitName(p.name);
                players.push({ Player_ID: p.id, Name, Surname, Position:
mapPosition(p.position),
                    DetailedPosition: detailedPosition(p.position), Age: calcAge(p.dateOfBirth),
                    Nationality: p.nationality || null, Team_ID: t.id, Team_Name: t.name });
            }
        }
        res.json(players.filter(p => p.Position !== 'Unknown'));
    } catch (err) {
        res.status(500).json({ error: 'Failed to fetch players' });
    }
});

router.get('/players/:id/profile', async (req, res) => {
    const playerId = parseInt(req.params.id, 10);
    const teamId = parseInt(req.query.teamId, 10);
    const code = req.query.code || 'PL';
    if (!teamId) return res.status(400).json({ error: 'teamId required' });
    try {
        const [teamData, matchData] = await Promise.all([
            fdApi.getTeamDetail(teamId), fdApi.getMatches(code, { season: SEASON }),
        ]);
        const rawPlayer = (teamData.squad || []).find(p => p.id === playerId);
        if (!rawPlayer) return res.status(404).json({ error: 'Player not found in squad'
});
        const position = mapPosition(rawPlayer.position);
        const { Name, Surname } = splitName(rawPlayer.name);
        const teamMatches = (matchData.matches || []).filter(m =>
            m.status === 'FINISHED' && (m.homeTeam.id === teamId || m.awayTeam.id === teamId)
        );
        let stats = {};
        if (position === 'Goalkeeper') {
            let cleanSheets = 0, conceded = 0;
            for (const m of teamMatches) {
                const ft = m.score?.fullTime;
                if (!ft || ft.home == null) continue;
                const ga = m.homeTeam.id === teamId ? ft.away : ft.home;
                conceded += ga;
                if (ga === 0) cleanSheets++;
            }
            stats = { cleanSheets, conceded, played: teamMatches.length };
        } else {
            const scorersData = await fdApi.getScorers(code, 100);
            const entry = (scorersData.scorers || []).find(s => s.player.id === playerId);
            stats = { goals: entry?.goals ?? 0, assists: entry?.assists ?? 0,
                penalties: entry?.penalties ?? 0, playedMatches: entry?.playedMatches ?? 0 };
        }
        const recentMatches = [...teamMatches]

```

```

        .sort((a, b) => new Date(b.utcDate) - new Date(a.utcDate))
        .slice(0, 5).map(normalizeMatch);
    res.json({ Player_ID: playerId, Name, Surname, FullName: rawPlayer.name,
        Position: position, DetailedPosition: detailedPosition(rawPlayer.position),
        Nationality: rawPlayer.nationality || null, Age: calcAge(rawPlayer.dateOfBirth),
        Team_ID: teamId, Team_Name: teamData.name, Team_Crest: teamData.crest || null,
        leagueCode: code, stats, recentMatches });
} catch (err) {
    res.status(err.response?.status || 500).json({ error: 'Failed to fetch player
profile' });
}
});

router.get('/matches', async (req, res) => {
    const code = req.query.code || 'PL';
    const upcoming = req.query.status === 'upcoming';
    try {
        const data = await fdApi.getMatches(code, { season: SEASON });
        const matches = (data.matches || []).
            .filter(m => upcoming ? (m.status === 'SCHEDULED' || m.status === 'TIMED') :
m.status === 'FINISHED')
            .map(normalizeMatch);
        res.json(matches);
    } catch (err) {
        res.status(500).json({ error: 'Failed to fetch matches' });
    }
});

router.get('/h2h', async (req, res) => {
    const code = req.query.code || 'PL';
    const team1Id = parseInt(req.query.team1Id, 10);
    const team2Id = parseInt(req.query.team2Id, 10);
    if (!team1Id || !team2Id) return res.status(400).json({ error: 'team1Id and team2Id
required' });
    try {
        const data = await fdApi.getMatches(code, { season: SEASON });
        const meetings = (data.matches || []).filter(m =>
            m.status === 'FINISHED' &&
            ((m.homeTeam.id === team1Id && m.awayTeam.id === team2Id) ||
            (m.homeTeam.id === team2Id && m.awayTeam.id === team1Id))
        ).sort((a, b) => new Date(b.utcDate) - new Date(a.utcDate));
        let team1Wins = 0, team2Wins = 0, draws = 0, team1Goals = 0, team2Goals = 0;
        for (const m of meetings) {
            const ft = m.score?.fullTime;
            if (!ft || ft.home == null) continue;
            const isTeam1Home = m.homeTeam.id === team1Id;
            const t1g = isTeam1Home ? ft.home : ft.away;
            const t2g = isTeam1Home ? ft.away : ft.home;
            team1Goals += t1g; team2Goals += t2g;
            if (t1g > t2g) team1Wins++;
            else if (t2g > t1g) team2Wins++;
            else draws++;
        }
        res.json({ meetings: meetings.map(normalizeMatch),
            record: { team1Wins, team2Wins, draws, team1Goals, team2Goals } });
    } catch (err) {
        res.status(500).json({ error: 'Failed to fetch head-to-head data' });
    }
});

router.get('/all-teams', async (req, res) => {
    const codes = ['PL', 'PD', 'BL1', 'SA', 'FL1', 'ELC', 'PPL', 'DED', 'BSA'];
    const settled = await Promise.allSettled(codes.map(code =>
fdApi.getTeams(code).then(data => ({ code, data })))));
    const results = [];

```

```

    for (const outcome of settled) {
      if (outcome.status === 'rejected') continue;
      const { code, data } = outcome.value;
      for (const t of (data.teams || []))
        results.push({ ...normalizeTeam(t), leagueCode: code, leagueName:
LEAGUE_NAMES[code] || code });
    }
    const seen = new Set();
    res.json(results.filter(t => { if (seen.has(t.Team_ID)) return false;
seen.add(t.Team_ID); return true; }));
  });
}

```

```
module.exports = router;
```

backend/profiles.js – publiskā profila maršruts

Fails profiles.js atgriež cita lietotāja publisko profilu pēc lietotājvārda: avatāru, iecienīto komandu, iecienīto spēlētāju un vērošanas sarakstu.

```

const express = require('express');
const pool = require('./db');

const router = express.Router();

router.get('/:username', async (req, res) => {
  const { username } = req.params;
  try {
    const [users] = await pool.query(
      'SELECT User_ID, Username, avatar_url FROM users WHERE Username = ?', [username]
    );
    if (!users.length) return res.status(404).json({ error: 'User not found' });
    const user = users[0];
    const [[favRows], [watchlist]] = await Promise.all([
      pool.query(
        'SELECT Team_ID, Player_ID, Player_Name, Player_Team_ID, Player_Team_Name,
Position, DetailedPosition, Nationality, LeagueCode FROM favourites WHERE User_ID = ?
LIMIT 1',
        [user.User_ID]
      ),
      pool.query(
        'SELECT Player_ID, Player_Name, Team_ID, Team_Name, Position, DetailedPosition,
Nationality, LeagueCode FROM watchlist WHERE User_ID = ? ORDER BY CreatedAt DESC',
        [user.User_ID]
      ),
    ]);
    const fav = favRows[0] || {};
    const favTeamId = fav.Team_ID || null;
    const favPlayerId = fav.Player_ID || null;
    let favPlayerMeta = null;
    if (favPlayerId) {
      if (fav.Player_Name) {
        favPlayerMeta = { Player_ID: favPlayerId, Player_Name: fav.Player_Name,
          Team_ID: fav.Player_Team_ID, Team_Name: fav.Player_Team_Name,
          Position: fav.Position, DetailedPosition: fav.DetailedPosition,
          Nationality: fav.Nationality, LeagueCode: fav.LeagueCode };
      } else {
        const ownEntry = watchlist.find(w => w.Player_ID === favPlayerId);
        if (ownEntry) {
          favPlayerMeta = ownEntry;
        } else {
          const [anyRows] = await pool.query(
            'SELECT Player_ID, Player_Name, Team_ID, Team_Name, Position,
DetailedPosition, Nationality, LeagueCode FROM watchlist WHERE Player_ID = ? LIMIT 1',
            [favPlayerId]
          );
        }
      }
    }
  }
}

```

```

        );
        favPlayerMeta = anyRows[0] || null;
    }
}
res.json({ username: user.Username, avatarUrl: user.avatar_url || null,
    favTeamId, favPlayerId, favPlayerMeta, watchlist });
} catch (err) {
    console.error('Public profile error:', err.message);
    res.status(500).json({ error: 'Failed to fetch profile' });
}
});

module.exports = router;

```

Frontend pirmkods

frontend/src/main.js – lietotnes inicializācija

Fails main.js inicializē Vue lietotni: pievieno i18n, Vuex store un Vue Router, kā arī ielādē flag emoji polyfill Windows pārlūku atbalstam.

```
import './assets/main.css'
import { polyfillCountryFlagEmojis } from 'country-flag-emoji-polyfill'
polyfillCountryFlagEmojis()

import { createApp } from 'vue'
import { createI18n } from 'vue-i18n'
import App from './App.vue'
import router from './router'
import store from './store/store.js'
import en from './locales/en.js'
import lv from './locales/lv.js'

const i18n = createI18n({
  legacy: false,
  locale: localStorage.getItem('locale') || 'en',
  fallbackLocale: 'en',
  messages: { en, lv },
})

const app = createApp(App)
app.use(router)
app.use(store)
app.use(i18n)
app.mount('#app')
```

frontend/src/router/index.js – maršrutētājs

Fails index.js definē visus frontend maršrutus un navigācijas sardzes, kas aizsargā privātus maršrutus un novirza neautorizētus lietotājus.

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import store from '../store/store.js'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    { path: '/', name: 'home', component: HomeView },
    { path: '/dashboard', name: 'dashboard', component: () =>
import('../views/UserDashboard.vue') },
    { path: '/teams', name: 'team-details', component: () =>
import('../views/TeamDetails.vue') },
    { path: '/table', name: 'table', component: () =>
import('../views/TableView.vue') },
    { path: '/leagues', name: 'leagues', component: () =>
import('../views/LeagueView.vue') },
    { path: '/teams/:id', name: 'team-squad', component: () =>
import('../views/TeamView.vue') },
    { path: '/matches', name: 'match-details', component: () =>
import('../views/MatchDetails.vue') },
```

```

    { path: '/h2h',          name: 'h2h',          component: () =>
import('../views/H2HView.vue') },
    { path: '/leaderboard', name: 'leaderboard',   component: () =>
import('../views/LeaderboardView.vue') },
    { path: '/players',     name: 'player-profile', component: () =>
import('../views/PlayerProfile.vue') },
    { path: '/players/:id', name: 'player-view',   component: () =>
import('../views/PlayerView.vue') },
    { path: '/admin',       name: 'admin-panel',   component: () =>
import('../views/AdminPanel.vue') },
    { path: '/login',      name: 'Login',         component: () =>
import('../views/LoginView.vue') },
    { path: '/register',   name: 'Register',      component: () =>
import('../views/RegisterView.vue') },
    { path: '/account',    name: 'Account',       component: () =>
import('../views/AccountView.vue') },
    { path: '/settings',   name: 'Settings',      component: () =>
import('../views/SettingsView.vue') },
    { path: '/users/:username', name: 'user-profile', component: () =>
import('../views/PublicProfile.vue') },
    { path: '/*',          name: 'not-found',      component: () =>
import('../views/NotFoundView.vue') },
  ],
})

router.beforeEach((to, from, next) => {
  const storedUser = store.state.user
  const isLoggedIn = !!storedUser
  const isAdmin = storedUser?.role?.toLowerCase() === 'admin'
  if (['/account', '/dashboard', '/settings'].includes(to.path) && !isLoggedIn) {
    next('/login')
  } else if (to.path === '/admin' && !isAdmin) {
    next('/')
  } else if (['/login', '/register'].includes(to.path) && isLoggedIn) {
    next('/dashboard')
  } else {
    next()
  }
})

export default router

```

frontend/src/store/store.js – Vuex stāvokļa pārvaldība

Fails store.js pārvalda autentifikācijas stāvokli: lietotāja datus, JWT tokenu localStorage un avatar URL atjaunināšanu.

```

import { createStore } from 'vuex'

export default createStore({
  state() {
    let user = null
    try { user = JSON.parse(localStorage.getItem('user')) } catch {}
    return { user }
  },
  actions: {
    login({ commit }, userData) {
      commit('setUser', userData)
      localStorage.setItem('user', JSON.stringify(userData))
    },
    logout({ commit }) {
      commit('clearUser')
      localStorage.removeItem('token')
      localStorage.removeItem('user')
    }
  }
})

```

```

    },
  },
  mutations: {
    setUser(state, user)    { state.user = user },
    clearUser(state)        { state.user = null },
    updateUser(state, updates) {
      state.user = { ...state.user, ...updates }
      localStorage.setItem('user', JSON.stringify(state.user))
    },
    setAvatar(state, avatarUrl) {
      state.user = { ...state.user, avatarUrl }
      localStorage.setItem('user', JSON.stringify(state.user))
    },
  },
})

```

frontend/src/services/api.js – Axios HTTP klients

Fails api.js konfigurē Axios instanci ar bāzes URL un pieprasījumu interceptoru, kas automātiski pievieno JWT tokenu katram pieprasījumam.

```

import axios from 'axios'

const api = axios.create({
  baseURL: import.meta.env.VITE_API_URL || '/api',
  headers: { 'Cache-Control': 'no-cache' },
})

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token')
  if (token) config.headers['Authorization'] = `Bearer ${token}`
  return config
})

export default api

```

frontend/src/utils/football.js – futbola palīgfunkcijas

Fails football.js eksportē tautību karogu atbilstību tabulu, pozīciju klasifikācijas funkcijas un pozīciju saīsinājumus, ko izmanto vairāki Vue komponenti.

```

export const NATIONALITY_FLAGS = {
  'England': '🇬🇧', 'Scotland': '🏴󠁧󠁢󠁥󠁮󠁧󠁿', 'Wales': '🏴󠁧󠁢󠁷󠁿',
  'Spain': '🇪🇸', 'France': '🇫🇷', 'Germany': '🇩🇪', 'Italy': '🇮🇹',
  'Portugal': '🇵🇹', 'Netherlands': '🇳🇱', 'Belgium': '🇧🇪', 'Brazil': '🇧🇷',
  'Argentina': '🇦🇷', 'Norway': '🇳🇴', 'Denmark': '🇩🇰', 'Sweden': '🇸🇪',
  'Poland': '🇵🇱', 'Croatia': '🇭🇷', 'Serbia': '🇷🇸', 'Austria': '🇦🇹',
  'Switzerland': '🇨🇭', 'Greece': '🇬🇷', 'Turkey': '🇹🇷', 'Ukraine': '🇺🇦',
  'Ghana': '🇬🇭', 'Nigeria': '🇳🇮', 'Senegal': '🇸🇳', 'Morocco': '🇲🇦',
  'South Korea': '🇰🇷', 'Japan': '🇯🇵', 'United States': '🇺🇸',
  'Republic of Ireland': '🇮🇪', 'Kosovo': '🇰🇲', 'Albania': '🇦🇱',
}

export function nationalityFlag(nat) {
  return NATIONALITY_FLAGS[nat] || '🏴󠁧󠁢󠁥󠁮󠁧󠁿'
}

export function posClass(pos) {
  if (!pos) return 'unknown'
  const p = pos.toLowerCase()
  if (p === 'goalkeeper') return 'gk'
  if (p === 'defender') return 'def'
}

```

```

    if (p === 'midfielder') return 'mid'
    if (p === 'forward')    return 'fwd'
    return 'unknown'
  }

  const DETAIL_ABBR = {
    'goalkeeper': 'GK', 'centre-back': 'CB', 'left-back': 'LB', 'right-back': 'RB',
    'wing-back': 'WB', 'defensive mid': 'DM', 'central mid': 'CM', 'attacking mid': 'AM',
    'left winger': 'LW', 'right winger': 'RW', 'centre-forward': 'CF', 'striker': 'ST',
    'defender': 'DEF', 'midfielder': 'MID', 'forward': 'FWD',
  }

  export function posAbbr(detailed) {
    if (!detailed) return '?'
    return DETAIL_ABBR[detailed.toLowerCase()] || detailed
  }

```

frontend/src/App.vue – galvenais lietotnes komponents

Fails App.vue satur navigācijas joslu ar darbvirsma un mobilā skata izkārtojumu, valodas pārslēgšanu, lietotāja izvēlni un RouterView satura attēlošanai.

```

<template>
  <div id="app">
    <header class="app-header">
      <div class="header-inner">
        <RouterLink to="/" class="brand">Footbalyze</RouterLink>
        <nav class="navbar desktop-nav">
          <RouterLink to="/" class="nav-link">{{ $t('nav.home')
        }}</RouterLink>
          <RouterLink to="/teams" class="nav-link">{{ $t('nav.teams')
        }}</RouterLink>
          <RouterLink to="/table" class="nav-link">{{ $t('nav.table')
        }}</RouterLink>
          <RouterLink to="/leagues" class="nav-link">{{ $t('nav.leagues')
        }}</RouterLink>
          <RouterLink to="/matches" class="nav-link">{{ $t('nav.matches')
        }}</RouterLink>
          <RouterLink to="/players" class="nav-link">{{ $t('nav.players')
        }}</RouterLink>
          <RouterLink to="/h2h" class="nav-link">{{ $t('nav.h2h')
        }}</RouterLink>
          <RouterLink to="/leaderboard" class="nav-link">{{ $t('nav.leaderboard')
        }}</RouterLink>
          <RouterLink v-if="isLoggedIn" to="/dashboard" class="nav-link">{{
        $t('nav.dashboard') }}</RouterLink>
        <div class="nav-utils">
          <SearchBar />
          <button class="lang-toggle" @click="toggleLocale">{{ locale === 'en' ? 'LV'
        : 'EN' }}</button>
          <div class="account-dropdown-wrapper">
            <button class="account-btn" @click="onAccountClick">
              {{ isLoggedIn ? user.username : $t('nav.account') }}
            </button>
            <div v-if="showDropdown" class="dropdown-menu">
              <template v-if="!isLoggedIn">
                <button class="dropdown-item" @click="loginClick">{{ $t('nav.login')
              }}</button>
                <button class="dropdown-item" @click="registerClick">{{
              $t('nav.register') }}</button>
              </template>
              <template v-else>
                <RouterLink class="dropdown-item" to="/account" @click="showDropdown
              = false">{{ $t('nav.myAccount') }}</RouterLink>

```



```

        <RouterLink class="dropdown-item" to="/settings" @click="showDropdown
= false">{{ $t('nav.settings') }}</RouterLink>
        <RouterLink v-if="user?.role?.toLowerCase() === 'admin'"
class="dropdown-item dropdown-item--admin" to="/admin" @click="showDropdown = false">{{
$t('nav.adminPanel') }}</RouterLink>
        <button class="dropdown-item dropdown-item--danger"
@click="logout">{{ $t('nav.logout') }}</button>
    </template>
</div>
</div>
</nav>
<div class="mobile-controls">
    <SearchBar />
    <button class="hamburger" :class="{ open: menuOpen }" @click="menuOpen =
!menuOpen">
        <span></span><span></span><span></span>
    </button>
</div>
</div>
<nav v-if="menuOpen" class="mobile-drawer">
    <RouterLink to="/" class="mob-link" @click="closeMenu">{{
$t('nav.home') }}</RouterLink>
    <RouterLink to="/teams" class="mob-link" @click="closeMenu">{{
$t('nav.teams') }}</RouterLink>
    <RouterLink to="/table" class="mob-link" @click="closeMenu">{{
$t('nav.table') }}</RouterLink>
    <RouterLink to="/matches" class="mob-link" @click="closeMenu">{{
$t('nav.matches') }}</RouterLink>
    <RouterLink to="/leaderboard" class="mob-link" @click="closeMenu">{{
$t('nav.leaderboard') }}</RouterLink>
    <div class="mob-divider"></div>
    <template v-if="!isLoggedIn">
        <RouterLink to="/login" class="mob-link" @click="closeMenu">{{
$t('nav.login') }}</RouterLink>
        <RouterLink to="/register" class="mob-link" @click="closeMenu">{{
$t('nav.register') }}</RouterLink>
    </template>
    <template v-else>
        <RouterLink to="/account" class="mob-link" @click="closeMenu">{{
$t('nav.myAccount') }}</RouterLink>
        <RouterLink to="/dashboard" class="mob-link" @click="closeMenu">{{
$t('nav.dashboard') }}</RouterLink>
        <RouterLink to="/settings" class="mob-link" @click="closeMenu">{{
$t('nav.settings') }}</RouterLink>
        <RouterLink v-if="user?.role?.toLowerCase() === 'admin'" to="/admin"
class="mob-link mob-link--admin" @click="closeMenu">{{ $t('nav.adminPanel')
}}</RouterLink>
        <button class="mob-link mob-link--danger" @click="logoutMobile">{{
$t('nav.logout') }}</button>
    </template>
    <div class="mob-divider"></div>
    <button class="mob-link" @click="toggleLocale">{{ locale === 'en' ? 'Latviski'
: 'English' }}</button>
</nav>
</header>
<main class="app-main"><RouterView /></main>
</div>
</template>

<script setup>
import { computed, ref, onMounted, onBeforeUnmount, watch } from 'vue'
import { useStore } from 'vuex'
import { useRouter, useRoute } from 'vue-router'
import { useI18n } from 'vue-i18n'
import SearchBar from './components/SearchBar.vue'

```

```

const store = useStore()
const router = useRouter()
const route = useRoute()
const { locale } = useI18n()

function toggleLocale() {
  locale.value = locale.value === 'en' ? 'lv' : 'en'
  localStorage.setItem('locale', locale.value)
}

const isLoggedIn = computed(() => !!store.state.user)
const user = computed(() => store.state.user)
const showDropdown = ref(false)
const menuOpen = ref(false)

watch(() => route.path, () => { menuOpen.value = false })

function closeMenu() { menuOpen.value = false }
function onAccountClick() {
  if (isLoggedIn.value) { router.push('/account'); showDropdown.value = false }
  else showDropdown.value = !showDropdown.value
}
function loginClick() { showDropdown.value = false; router.push('/login') }
function registerClick() { showDropdown.value = false; router.push('/register') }
function logout() { showDropdown.value = false; store.dispatch('logout');
router.push('/') }
function logoutMobile() { closeMenu(); store.dispatch('logout'); router.push('/') }

function handleOutsideClick(e) {
  const wrapper = document.querySelector('.account-dropdown-wrapper')
  if (wrapper && !wrapper.contains(e.target)) showDropdown.value = false
}
onMounted(() => document.addEventListener('mousedown', handleOutsideClick))
onBeforeUnmount(() => document.removeEventListener('mousedown', handleOutsideClick))
</script>

```

frontend/src/views/LoginView.vue – pieteikšanās skats

Fails LoginView.vue realizē pieteikšanās formu ar e-pasta un paroles laukiem. Veiksmīgas pieteikšanās gadījumā JWT žetons tiek saglabāts localStorage un lietotājs tiek novirzīts uz personīgo informācijas paneli.

```

<template>
  <div class="auth-page">
    <div class="auth-card">
      <h2 v-if="!user">{{ $t('login.title') }}</h2>
      <h2 v-else>{{ $t('login.welcome', { name: user.username }) }}</h2>

      <form v-if="!user" @submit.prevent="handleLogin" class="auth-form">
        <div class="field">
          <label for="email">{{ $t('login.email') }}</label>
          <input
            id="email"
            v-model="email"
            type="email"
            placeholder="you@example.com"
            required
            autocomplete="email"
          />
        </div>
      </form>
    </div>
  </div>

```

```

    <div class="field">
      <label for="password">{{ $t('login.password') }}</label>
      <input
        id="password"
        v-model="password"
        type="password"
        :placeholder="$t('login.password')"
        required
        autocomplete="current-password"
      />
    </div>

    <button type="submit" class="btn-primary" :disabled="loading">
      {{ loading ? $t('login.submitting') : $t('login.submit') }}
    </button>
  </form>

  <div v-if="user" class="logged-in-actions">
    <RouterLink to="/dashboard" class="btn-primary">{{ $t('login.toDashboard')
  }}</RouterLink>
    <button class="btn-ghost" @click="logout">{{ $t('login.logout') }}</button>
  </div>

  <p v-if="errorMessage" class="error-msg">{{ errorMessage }}</p>

  <p v-if="!user" class="auth-switch">
    {{ $t('login.noAccount') }}
    <RouterLink to="/register">{{ $t('nav.register') }}</RouterLink>
  </p>
</div>
</template>

<script setup>
import { ref, computed } from 'vue'
import { useStore } from 'vuex'
import { useRouter } from 'vue-router'
import { useI18n } from 'vue-i18n'
import api from '../services/api'

const store = useStore()
const router = useRouter()
const { t } = useI18n()

const email = ref('')
const password = ref('')
const loading = ref(false)
const errorMessage = ref('')

const user = computed(() => store.state.user)

async function handleLogin() {
  loading.value = true
  errorMessage.value = ''
  try {
    const response = await api.post('/login', {
      email: email.value,
      password: password.value
    })
    const { token, user: loggedInUser } = response.data

    localStorage.setItem('token', token)
    localStorage.setItem('user', JSON.stringify(loggedInUser))
  }
}

```

```

    store.dispatch('login', loggedInUser)

    email.value = ''
    password.value = ''
    router.push('/dashboard')
  } catch (error) {
    errorMessage.value = error.response?.data?.message || t('login.error')
  } finally {
    loading.value = false
  }
}

function logout() {
  store.dispatch('logout')
}
</script>

<style scoped>
.auth-page {
  display: flex;
  align-items: center;
  justify-content: center;
  min-height: calc(100vh - 60px);
  padding: 2rem 1rem;
}

.auth-card {
  width: 100%;
  max-width: 400px;
  background: #141c27;
  border: 1px solid rgba(245, 158, 11, 0.15);
  border-radius: var(--radius-lg);
  padding: 2.25rem;
  box-shadow: 0 8px 32px rgba(0, 0, 0, 0.4);
}

h2 {
  font-size: 1.6rem;
  font-weight: 800;
  text-align: center;
  margin-bottom: 1.75rem;
  color: var(--text-primary);
  letter-spacing: -0.02em;
}

.auth-form {
  display: flex;
  flex-direction: column;
  gap: 1.1rem;
}

.field {
  display: flex;
  flex-direction: column;
  gap: 0.4rem;
}

.field label {
  font-size: 0.82rem;
  font-weight: 600;
  color: var(--text-secondary);
  letter-spacing: 0.02em;
}

```

```

}

input {
  background: rgba(255, 255, 255, 0.06);
  border: 1px solid rgba(255, 255, 255, 0.1);
  border-radius: var(--radius);
  color: var(--text-primary);
  padding: 0.65rem 0.9rem;
  font-size: 0.95rem;
  transition: border-color 0.2s, background 0.2s;
  width: 100%;
}
input::placeholder { color: var(--text-muted); }
input:focus {
  outline: none;
  border-color: rgba(245, 158, 11, 0.6);
  background: rgba(245, 158, 11, 0.04);
}

.btn-primary {
  width: 100%;
  margin-top: 0.4rem;
  text-align: center;
  display: block;
}
.btn-ghost {
  width: 100%;
  margin-top: 0.5rem;
}

.logged-in-actions { display: flex; flex-direction: column; gap: 0.5rem; }

.error-msg {
  color: var(--lose);
  font-size: 0.875rem;
  margin-top: 0.75rem;
  text-align: center;
}

.auth-switch {
  text-align: center;
  font-size: 0.875rem;
  color: var(--text-secondary);
  margin-top: 1.25rem;
}
.auth-switch a {
  color: #f59e0b;
  text-decoration: none;
  font-weight: 600;
}
.auth-switch a:hover { text-decoration: underline; }
</style>

```

frontend/src/views/RegisterView.vue – reģistrācijas skats

Fails RegisterView.vue realizē reģistrācijas formu ar tiešraides validāciju: lietotājbārdam jābūt no 3 līdz 20 rakstzīmēm (burti, cipari, pasvītra), parolei jāsatvismaz 8 rakstzīmes, lielo burtu un ciparu. Kļūdas tiek parādītas katram laukam atsevišķi.

```

<template>
  <div class="auth-page">
    <div class="auth-card">

```

```

<h2>{{ $t('register.title') }}</h2>

<form @submit.prevent="handleRegister" class="auth-form" novalidate>

  <!-- Username -->
  <div class="field" :class="{ 'field--error': fieldErrors.username }">
    <label for="username">{{ $t('register.username') }}</label>
    <input
      id="username"
      v-model="username"
      type="text"
      :placeholder="$t('register.usernamePlaceholder')"
      autocomplete="username"
    />
    <span v-if="fieldErrors.username" class="field-error">{{ fieldErrors.username
  }}</span>
    <ul v-else class="requirements">
      <li :class="req(username.length >= 3 && username.length <= 20)">{{
$t('register.reqChars') }}</li>
      <li :class="req(/^[a-zA-Z0-9_]*$/ .test(username) && username.length >
0)">{{ $t('register.reqAlphanum') }}</li>
    </ul>
  </div>

  <!-- Email -->
  <div class="field" :class="{ 'field--error': fieldErrors.email }">
    <label for="email">{{ $t('register.email') }}</label>
    <input
      id="email"
      v-model="email"
      type="email"
      :placeholder="$t('register.emailPlaceholder')"
      autocomplete="email"
    />
    <span v-if="fieldErrors.email" class="field-error">{{ fieldErrors.email
  }}</span>
  </div>

  <!-- Password -->
  <div class="field" :class="{ 'field--error': fieldErrors.password }">
    <label for="password">{{ $t('register.password') }}</label>
    <input
      id="password"
      v-model="password"
      type="password"
      :placeholder="$t('register.passwordPlaceholder')"
      autocomplete="new-password"
    />
    <span v-if="fieldErrors.password" class="field-error">{{ fieldErrors.password
  }}</span>
    <ul class="requirements">
      <li :class="req(password.length >= 8)">{{ $t('register.reqLength') }}</li>
      <li :class="req(/[A-Z]/ .test(password))">{{ $t('register.reqUpper') }}</li>
      <li :class="req(/[0-9]/ .test(password))">{{ $t('register.reqNumber')
  }}</li>
    </ul>
  </div>

  <p v-if="generalError" class="error-msg">{{ generalError }}</p>

  <button type="submit" class="btn-primary" :disabled="loading || !canSubmit">
    {{ loading ? $t('register.submitting') : $t('register.submit') }}
  </button>
</form>

```

```

    <p v-if="successMessage" class="success-msg">{{ successMessage }}</p>

    <p class="auth-switch">
      {{ $t('register.haveAccount') }}
      <RouterLink to="/login">{{ $t('nav.login') }}</RouterLink>
    </p>
  </div>
</div>
</template>

<script setup>
import { ref, computed } from 'vue'
import { useRouter } from 'vue-router'
import { useI18n } from 'vue-i18n'
import api from '../services/api'

const router = useRouter()
const { t } = useI18n()

const username = ref('')
const email = ref('')
const password = ref('')
const loading = ref(false)
const generalError = ref('')
const successMessage = ref('')
const fieldErrors = ref({ username: '', email: '', password: '' })

const usernameValid = computed(() =>
  username.value.length >= 3 && username.value.length <= 20 && /^[a-zA-Z0-9_]+$/ .test(username.value)
)
const emailValid = computed(() => /^[^\s@]+@[^\s@]+\.[^\s@]+$/ .test(email.value))
const passwordValid = computed(() =>
  password.value.length >= 8 && /[A-Z]/ .test(password.value) && /[0-9]/ .test(password.value)
)
const canSubmit = computed(() => usernameValid.value && emailValid.value && passwordValid.value)

function req(met) { return met ? 'req-met' : 'req-unmet' }

async function handleRegister() {
  generalError.value = ''
  fieldErrors.value = { username: '', email: '', password: '' }
  if (!canSubmit.value) return
  loading.value = true
  try {
    await api.post('/register', { username: username.value, email: email.value,
password: password.value })
    successMessage.value = t('register.success')
    username.value = ''
    email.value = ''
    password.value = ''
    setTimeout(() => router.push('/login'), 1500)
  } catch (error) {
    const data = error.response?.data
    if (data?.field && data?.message) {
      fieldErrors.value[data.field] = data.message
    } else if (data?.message) {
      generalError.value = data.message
    } else {
      generalError.value = t('register.failed')
    }
  }
}

```

```

    }
  } finally {
    loading.value = false
  }
}
</script>

<style scoped>
.auth-page {
  display: flex;
  align-items: center;
  justify-content: center;
  min-height: calc(100vh - 60px);
  padding: 2rem 1rem;
}

.auth-card {
  width: 100%;
  max-width: 420px;
  background: #141c27;
  border: 1px solid rgba(245, 158, 11, 0.15);
  border-radius: var(--radius-lg);
  padding: 2.25rem;
  box-shadow: 0 8px 32px rgba(0, 0, 0, 0.4);
}

h2 {
  font-size: 1.6rem;
  font-weight: 800;
  text-align: center;
  margin-bottom: 1.75rem;
  color: var(--text-primary);
  letter-spacing: -0.02em;
}

.auth-form { display: flex; flex-direction: column; gap: 1.1rem; }

.field { display: flex; flex-direction: column; gap: 0.35rem; }

.field label {
  font-size: 0.82rem;
  font-weight: 600;
  color: var(--text-secondary);
  letter-spacing: 0.02em;
}

input {
  background: rgba(255, 255, 255, 0.06);
  border: 1px solid rgba(255, 255, 255, 0.1);
  border-radius: var(--radius);
  color: var(--text-primary);
  padding: 0.65rem 0.9rem;
  font-size: 0.95rem;
  transition: border-color 0.2s, background 0.2s;
  width: 100%;
  box-sizing: border-box;
}

input::placeholder { color: var(--text-muted); }
input:focus {
  outline: none;
  border-color: rgba(245, 158, 11, 0.6);
  background: rgba(245, 158, 11, 0.04);
}

.field--error input { border-color: var(--lose); }

```



```

.field-error { font-size: 0.8rem; color: var(--lose); margin-top: 0.1rem; }

.requirements {
  list-style: none;
  padding: 0;
  margin: 0.2rem 0 0;
  display: flex;
  flex-direction: column;
  gap: 0.2rem;
}
.requirements li {
  font-size: 0.78rem;
  padding-left: 1.25rem;
  position: relative;
  transition: color 0.2s;
}
.requirements li::before { content: 'x'; position: absolute; left: 0; font-size: 0.7rem; font-weight: 700; }
.req-unmet { color: var(--text-muted); }
.req-met { color: var(--accent-green-hover); }
.req-met::before { content: 'v' !important; color: var(--accent-green-hover); }

.btn-primary {
  width: 100%;
  margin-top: 0.4rem;
}

.error-msg { color: var(--lose); font-size: 0.875rem; margin-top: 0.25rem; text-align: center; }
.success-msg { color: #fbbf24; font-size: 0.875rem; margin-top: 0.75rem; text-align: center; }

.auth-switch { text-align: center; font-size: 0.875rem; color: var(--text-secondary); margin-top: 1.25rem; }
.auth-switch a { color: #f59e0b; text-decoration: none; font-weight: 600; }
.auth-switch a:hover { text-decoration: underline; }
</style>

```

frontend/src/views/LeaderboardView.vue – līderu saraksta skats

Fails LeaderboardView.vue attēlo prognožu līderu sarakstu. Ja rezultātu vēl nav, tiek parādīts informatīvs tukšs stāvoklis ar punktu skaitīšanas noteikumiem: precīzs rezultāts dod 3 punktus, pareizs iznākums bez precīza rezultāta dod 1 punktu. Aizpildīts skats rāda tabulu ar rindu katram lietotājam, ieskaitot pašreizējo lietotāju.

```

<template>
  <div class="lb-page">

    <div class="page-header">
      <h1>{{ $t('leaderboard.title') }}</h1>
      <p class="page-sub">{{ $t('leaderboard.sub') }}</p>
    </div>

    <div v-if="loading" class="loading">{{ $t('leaderboard.loading') }}</div>

    <div v-else-if="!rows.length" class="empty-state">
      <div class="empty-icon">trophy</div>
      <h2 class="empty-title">{{ $t('leaderboard.noResults') }}</h2>
      <p class="empty-sub">{{ $t('leaderboard.noResultsSub') }}</p>
    </div>
  </div>
</template>

```

```

<div class="how-it-works">
  <p class="how-label">{{ $t('leaderboard.howTitle') }}</p>
  <div class="how-rows">
    <div class="how-row">
      <span class="how-icon exact">3pts</span>
      <div>
        <strong>{{ $t('leaderboard.exact') }}</strong>
        <span class="how-pts">{{ $t('leaderboard.exactPts') }}</span>
      </div>
      <p class="how-desc">{{ $t('leaderboard.exactDesc') }}</p>
    </div>
    <div class="how-row">
      <span class="how-icon correct">1pt</span>
      <div>
        <strong>{{ $t('leaderboard.correct') }}</strong>
        <span class="how-pts">{{ $t('leaderboard.correctPts') }}</span>
      </div>
      <p class="how-desc">{{ $t('leaderboard.correctDesc') }}</p>
    </div>
    <div class="how-row">
      <span class="how-icon wrong">0pts</span>
      <div>
        <strong>{{ $t('leaderboard.wrong') }}</strong>
        <span class="how-pts">{{ $t('leaderboard.wrongPts') }}</span>
      </div>
      <p class="how-desc">{{ $t('leaderboard.wrongDesc') }}</p>
    </div>
  </div>
</div>

  <router-link to="/dashboard" class="cta-btn">{{ $t('leaderboard.makePredictions') }}</router-link>
</div>

<div v-else class="lb-card">
  <table class="lb-table">
    <thead>
      <tr>
        <th class="col-rank">#</th>
        <th>{{ $t('leaderboard.player') }}</th>
        <th class="col-num col-hide-mobile">{{ $t('leaderboard.played') }}</th>
        <th class="col-num col-hide-mobile">{{ $t('leaderboardCols.exact') }}</th>
        <th class="col-num col-hide-mobile">{{ $t('leaderboardCols.correct') }}</th>
        <th class="col-num col-hide-mobile">{{ $t('leaderboardCols.wrong') }}</th>
        <th class="col-pts">{{ $t('leaderboard.pts') }}</th>
      </tr>
    </thead>
    <tbody>
      <tr>
        <td class="col-rank">
          <span class="rank-badge" :class="`rank-${i + 1}`">{{ i + 1 }}</span>
        </td>
        <td>
          <div class="col-user">
            <RouterLink :to="`/users/${row.Username}`" class="lb-username">{{
row.Username }}</RouterLink>
            <span v-if="row.Username === currentUsername" class="you-badge">{{
$t('leaderboard.you') }}</span>
          </div>
        </td>
      </tr>
    </tbody>
  </table>
</div>

```

```

        </td>
        <td class="col-num col-hide-mobile">{{ row.total }}</td>
        <td class="col-num col-exact col-hide-mobile">{{ row.exact_count }}</td>
        <td class="col-num col-correct col-hide-mobile">{{ row.correct_count
    }}</td>

        <td class="col-num col-wrong col-hide-mobile">{{ row.wrong_count }}</td>
        <td class="col-pts">
            <span class="pts-val">{{ row.points }}</span>
        </td>
    </tr>
</tbody>
</table>
</div>

</div>
</template>

<script setup>
import { ref, computed, onMounted } from 'vue'
import { RouterLink } from 'vue-router'
import { useStore } from 'vuex'
import api from '../services/api'

const store = useStore()
const currentUsername = computed(() => store.state.user?.username)

const rows = ref([])
const loading = ref(true)

onMounted(async () => {
    try {
        const res = await api.get('/predictions/leaderboard')
        rows.value = res.data
    } catch (e) {
        console.error(e)
    } finally {
        loading.value = false
    }
})
</script>

<style scoped>
.lb-page {
    max-width: 700px;
    margin: 0 auto;
    padding: 2rem 1rem 4rem;
    display: flex;
    flex-direction: column;
    gap: 1.5rem;
}

.page-header h1 {
    font-size: 1.75rem;
    font-weight: 800;
    color: var(--text-primary);
}

.page-sub {
    color: var(--text-muted);
    font-size: 0.875rem;
    margin-top: 0.25rem;
}

.loading {

```

```

    text-align: center;
    color: var(--text-muted);
    font-style: italic;
    font-size: 0.9rem;
    margin-top: 2rem;
}

.empty-state {
    display: flex;
    flex-direction: column;
    align-items: center;
    gap: 0.75rem;
    padding: 3rem 1rem;
    text-align: center;
}

.empty-icon { font-size: 2.5rem; }
.empty-title {
    font-size: 1.25rem;
    font-weight: 700;
    color: var(--text-primary);
    margin: 0;
}

.empty-sub {
    color: var(--text-muted);
    font-size: 0.875rem;
    max-width: 340px;
    line-height: 1.5;
    margin: 0;
}

.how-it-works {
    margin-top: 1.25rem;
    width: 100%;
    max-width: 420px;
    background: var(--bg-surface);
    border: 1px solid var(--border);
    border-radius: var(--radius-lg);
    padding: 1.25rem 1.5rem;
    text-align: left;
}

.how-label {
    font-size: 0.68rem;
    font-weight: 700;
    text-transform: uppercase;
    letter-spacing: 0.08em;
    color: var(--text-muted);
    margin: 0 0 1rem;
}

.how-rows { display: flex; flex-direction: column; gap: 0.75rem; }
.how-row {
    display: grid;
    grid-template-columns: 2rem 1fr;
    grid-template-rows: auto auto;
    column-gap: 0.75rem;
    align-items: start;
}

.how-icon {
    grid-row: 1 / 3;
    display: flex;
    align-items: center;
    justify-content: center;
    width: 2rem;
    height: 2rem;
    border-radius: 50%;
    font-size: 0.875rem;
}

```

```

    font-weight: 800;
    border: 1px solid;
}
.how-icon.exact { background: rgba(16,185,129,0.12); color: var(--win); border-color:
rgba(16,185,129,0.3); }
.how-icon.correct { background: rgba(245,158,11,0.12); color: var(--accent-hover);
border-color: rgba(245,158,11,0.3); }
.how-icon.wrong { background: rgba(239,68,68,0.12); color: var(--lose); border-color:
rgba(239,68,68,0.3); }
.how-row > div {
    display: flex;
    align-items: center;
    gap: 0.5rem;
}
.how-row strong { font-size: 0.875rem; color: var(--text-primary); }
.how-pts {
    margin-left: auto;
    font-size: 0.75rem;
    font-weight: 700;
    color: var(--accent-hover);
    background: var(--accent-muted);
    border: 1px solid rgba(245,158,11,0.25);
    border-radius: 999px;
    padding: 1px 8px;
}
.how-desc { font-size: 0.78rem; color: var(--text-muted); margin: 0; grid-column: 2; }

.cta-btn {
    margin-top: 1rem;
    display: inline-block;
    padding: 0.6rem 1.4rem;
    background: var(--accent);
    color: #0f1419;
    border-radius: var(--radius);
    font-size: 0.875rem;
    font-weight: 600;
    text-decoration: none;
    transition: background 0.15s;
}
.cta-btn:hover { background: var(--accent-hover); }

.lb-card {
    background: var(--bg-surface);
    border: 1px solid var(--border);
    border-radius: var(--radius-lg);
    overflow: hidden;
}

.lb-table {
    width: 100%;
    border-collapse: collapse;
    font-size: 0.875rem;
}

.lb-table thead tr {
    background: var(--bg-surface-2);
    border-bottom: 1px solid var(--border);
}

.lb-table th {
    padding: 0.65rem 1rem;
    text-align: left;
    font-size: 0.68rem;
    font-weight: 700;
}

```

```

    text-transform: uppercase;
    letter-spacing: 0.06em;
    color: var(--text-muted);
}

.lb-table td {
    padding: 0.75rem 1rem;
    border-bottom: 1px solid var(--border);
    vertical-align: middle;
}

.lb-table tbody tr:last-child td { border-bottom: none; }
.lb-table tbody tr:hover td { background: rgba(255,255,255,0.02); }
.row-self td { background: rgba(245,158,11,0.04); }

.col-rank { width: 48px; }
.col-num { text-align: center; color: var(--text-secondary); }
.col-pts { text-align: center; }
.col-user { display: flex; align-items: center; gap: 0.5rem; }

.col-exact { color: var(--win) !important; font-weight: 700; }
.col-correct { color: var(--accent-hover) !important; font-weight: 600; }
.col-wrong { color: var(--lose) !important; }

.rank-badge {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    width: 24px;
    height: 24px;
    border-radius: 50%;
    font-size: 0.72rem;
    font-weight: 800;
    background: var(--bg-surface-2);
    color: var(--text-muted);
    border: 1px solid var(--border);
}

.rank-1 { background: rgba(234,179,8,0.15); color: #facc15; border-color:
rgba(234,179,8,0.4); }
.rank-2 { background: rgba(148,163,184,0.15); color: #94a3b8; border-color:
rgba(148,163,184,0.4); }
.rank-3 { background: rgba(180,83,9,0.15); color: #f97316; border-color:
rgba(180,83,9,0.4); }

.lb-username {
    font-weight: 600;
    color: var(--text-primary);
    text-decoration: none;
    transition: color 0.15s;
}

.lb-username:hover { color: var(--accent-hover); }

.you-badge {
    font-size: 0.62rem;
    font-weight: 700;
    background: var(--accent-muted);
    color: var(--accent-hover);
    border: 1px solid var(--accent);
    border-radius: 999px;
    padding: 1px 6px;
}

.pts-val {

```

```
    font-size: 1rem;
    font-weight: 800;
    color: var(--accent-hover);
}

@media (max-width: 560px) {
  .col-hide-mobile { display: none; }
}
</style>
```